Digital Systems Project Assignment 1
Digital Watch: Settable Time

Done By: Julian Tan
Student Number: 269962
ELEN20006 Digital Systems, 2026 Semester 2

What values do variables hold ?

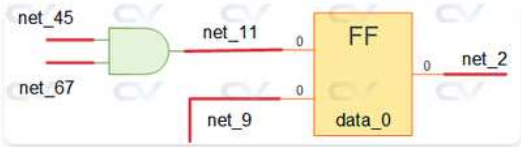
https://chipverify.com/verilog/verilog-data-types

Almost all data-types can only have one of the four different values as given below except for `real` and `event` data types.

0	represents a logic zero, or a false condition
1	represents a logic one, or a true condition
x	represents an unknown logic value (can be zero or one)
z	represents a high-impedance state

Nets

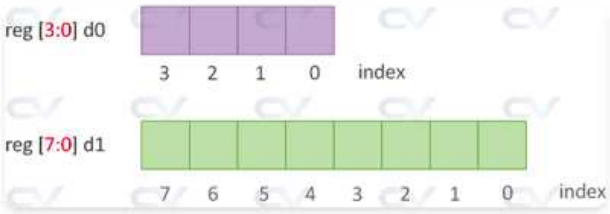
Nets are used to connect between hardware entities like logic gates and hence do not store any value on its own. In the image shown below, a net called `net_11` is used to connect between the output of the AND gate to the first input of the flip-flop called `data_0`. In a similar way, the two inputs of the AND gate are connected to nets `net_45` and `net_67`.



Nets and Variables

Nets and *variables* are the two main groups of data types which represent different hardware structures and differ in the way they are assigned and retain values.

Variables



A variable on the other hand is an abstraction of a data storage element and can hold values. A flip-flop is a good example of a storage element.

https://chipverify.com/systemverilog/systemverilog-data-type-logic

logic: A 4-State Data Type

`logic` is a 4-state data type — its value set (`0` , `1` , `X` , `Z`) is identical to `reg` 's. It isn't, by itself, a variable or a net; it's just the type. When you write a plain declaration like `logic a;` with no other keyword, SystemVerilog implicitly treats that as `var logic a;` — a **variable** of type `logic` .

5. Development Cycle

The golden rule of software development — which applies equally to Verilog development — is to write a *small* amount of code, compile and test it thoroughly, finalise its documentation, and then leave it alone unless you need to fix a bug down the track. We use this approach to build small, reusable modules. Near the end, when these modules must be assembled, we shift to a feature-layered strategy: add a small feature, compile and test thoroughly, then move on to the next feature.

Thousands of lines of automated test code have been written for you. In real life, you or your team would be responsible for developing and maintaining this test code. As the saying goes, *"if it is not tested, it is broken."* Test code is critical: it improves reliability, prevents costly defects, makes changes safer, and serves as living documentation.

Each write-compile-test loop follows what we call the *development cycle*. This cycle is exemplified below.

Conventions A good project has an agreed set of conventions.

Each module should live in its own file within the `rtl/` subdirectory, with the filename matching the module name. Names should be descriptive and should only be abbreviated when the full name would be overly long. Well-established abbreviations such as `clk` and `rst` are acceptable, provided they are used consistently.

Naming conventions are as follows. (Commonly, `localparam` constants are also capitalised, but our particular linter disagrees.)

Kind	Convention	Example
Modules and signals	`snake_case`	`next_state`, `tick_enable`
`parameter` constants	`ALL_CAPS`	`MAX`, `WIDTH`
`localparam` constants	`CamelCase`	`TicksPerSecond`

5.1. Module Specification

For each module, you will be given its declaration together with a description of the required behaviour. Follow the specification exactly, or the automated tests will fail.

5.2. Write the Module

Whenever you are asked to write a new module — starting now, for the `seven_segment` specification above — do the following:

5.3. Lint, Compile then Generate Waveforms

In this assignment we use the terminal for four things (in the next assignment, it will also be used to run a formal verification tool):

1. More thorough linting (`verilator -Wall --lint-only`).
2. Compiling code (`iverilog -Wall -g2012`).
3. Running test benches (`iverilog -Wall -g2012`, then `./a.out`).
4. Running the automated test suite (`pytest`).

Lint Run: `verilator -Wall --lint-only rtl/seven_segment.sv`

It reports 20 warnings before stopping. These are warnings rather than errors: the code is valid SystemVerilog and will compile. **Do not mistake "warning" for "harmless": a warning usually means the tool has noticed you doing something you did not intend.** Most of the warnings below correspond to genuine bugs. Even the one that does not, noted as it arises, should still be eliminated: being explicit about your intent is good practice, and the goal is always zero warnings.

Report 1

Explain why Verilator accepts `if (blank)` but not `if (ACTIVE_LOW)`.

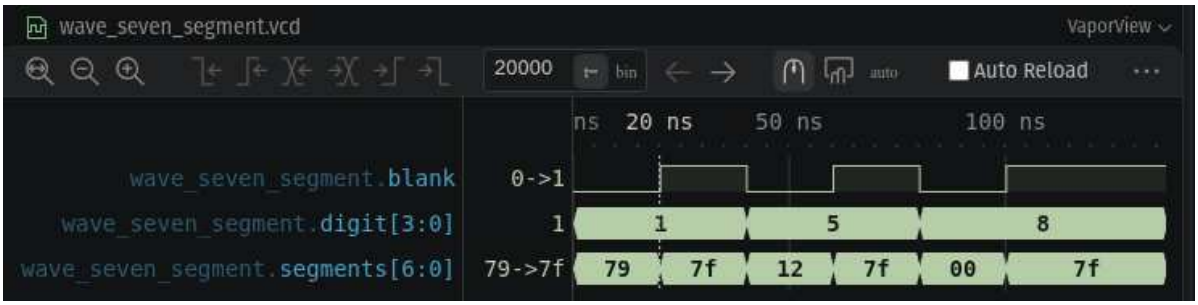
```
%Warning-WIDTHTRUNC: rtl/seven_segment.sv:42:32: Logical operator COND expects 1 bit on
the Conditional Test, but Conditional Test's VARREF 'ACTIVE_LOW' generates 32 bits.
                                                : ... note: In instance 'seven_segment'
   42 |   assign segments = ACTIVE_LOW ? !segments_active_high : segments_active_high;
      |                                ^
```

The if-conditional statement requires its expression to resolve to either True (1) or False (0). In other words, a binary outcome from 1-bit. It does not encode three or more outcomes by itself.

ACTIVE_LOW is declared as an `int`, which is a 2-state, 32-bit signed integer, and it can contain values other than 0 or 1. Although any non-zero values gets treated as True, the if condition still expects only single-bits and not 32-bits for the conditional tests



```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/seven_segment.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/seven_segment.sv
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/seven_segment.sv tb/wave_seven_segment.v
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_seven_segment.vcd opened for output.
tb/wave_seven_segment.v:41: $finish called at 120000 (1ps)
root@4034a70d84cc:/workspace#
```



5.4. Run the Automated Tests

From the terminal, run `pytest tests/test_seven_segment.py` to test only this module, or `pytest` to test all modules. Tests for unwritten modules will be skipped.

Check that all tests for `tests/test_seven_segment.py` pass. If they are reported as skipped, the file is either misnamed or not in the `rtl` subdirectory.

If any tests fail, work through the following steps:

- Scroll up through the `pytest` output to identify which test failed.
- Read the comments in `tests/cocotb/tb_seven_segment.py` to understand what the failing test checks.
- Focus on the corresponding part of your module.

If you are still stuck, see the next section.

```
root@4034a70d84cc:/workspace# pytest tests/test_seven_segment.py
========================== test session starts ==========================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 2 items

tests/test_seven_segment.py ..                                    [100%]

=========================== 2 passed in 0.57s ===========================
root@4034a70d84cc:/workspace#
```

5.5. Write Your Own Test Benches

If your module is still not working, write your own test bench. Although the error dump from `pytest` is verbose, and much of it may be unintelligible, scroll through it for messages that make sense. There will usually be a comment explaining what the test was checking, followed by a statement of why it failed. Target that same behaviour in your test bench: your aim is to reproduce the fault.

If reproducing the faulty behaviour requires many steps, simplify it to the fewest steps that still exhibit the fault. Focus on one bug at a time.

Once you have a test bench that reproduces the bug, trace your code through the same steps. This should reveal where your code behaves unexpectedly.

The other side of the coin is defensive programming. As this project emphasises, a module should be both small *and* simple: if the code cannot be understood in seconds rather than minutes, decompose it into smaller modules. When further decomposition is not feasible, adopt a feature-layering approach and add code gradually, one feature at a time, for the same reason.

Using small, focused modules is best practice because it keeps debugging manageable: a small module has fewer possible sources of error, and its input and state combinations grow much more slowly, which helps you avoid a combinatorial explosion in what you need to reason about and test.

5.6. Fulfil Report and Oral Preparation Obligations

The following obligations apply to every module, including `seven_segment`.

Report 2

For every module, include a screenshot of the waveforms and briefly explain why they are correct, pointing to specific features in the traces.

Additional Oral Preparation 1

For every module, be able to describe its behaviour precisely for all input combinations.

Additional Oral Preparation 2

Be able to explain the development cycle and the project conventions in detail.

5.7. Backup

After each module passes all its tests, back up your code and report.

Report 3

At the start of your report, describe your backup strategy.

I aleady have an existing GitHub account so I'll be using that as my cloud-based backup repository along with my local backup
The Original is forked onto my personal GitHub account and renamed as `fpga-digital-watch-2026Sem2-JT` to differentiate from the original. Both the original and forked repos are downloaded onto my computer.
All the coding is done on the downloaded original local repo. Once the coding session is done for the day, the modules are copied over to the downloaded forked local repo. From there I use my git system to add, commit with signiture, and push to my GitHub's forked repo for archival.

Development Cycle
- Write a small amount of code, compile and test thoroughly, finalise its documentation, backup, and leave it alone unless there's a bug down the line.
- Build using modules, small chunks of code that are reusable, each providing a reusable feature

Conventions (Coding Styles) are to be followed to ensure standardisation and consistency. This allows team members (if any) to understand each others' work without ambiguity

6. Time Display

By the end of this section, the seven-segment displays will show hours, minutes, and seconds, incrementing once per second like a clock. Switches select faster rates. Time is in 24-hour format.

6.1. State Encoding

Before writing any code, we must decide how to represent the current time in hardware. There are at least four reasonable choices.

1. **One counter (clock cycles).** Count the total number of rising edges since midnight. With a 50 MHz clock, a rising edge occurs every 20 ns.
2. **One counter (seconds).** Count the total number of seconds elapsed since midnight.
3. **Three counters.** Maintain separate counters for hours (0–23), minutes (0–59), and seconds (0–59).
4. **Six counters.** Maintain a separate counter for each display digit: the tens and units of hours, minutes, and seconds.

The choice of state encoding directly influences the number of flip-flops required, the complexity of the next-state logic, and the complexity of the output logic.

The encoding choice also influences the time-setting hardware developed later. Time-setting works by cycling through each unit (seconds, minutes, and hours) and letting the user increment or decrement each independently.

This project adopts the three-counter encoding: separate counters for hours, minutes, and seconds.

Additional Oral Preparation 3

Be able to explain all four state encoding options and the trade-offs between them, and justify why the three-counter encoding was chosen.

1 Day = 24 Hours = 1440 Minutes = 86400 Seconds

Option 1: One Counter (Clock Cycles)
CLOCK_50 is 50MHz clock, this means if the Counter is counting total number of rising edges since midnight, there are (50 x 10^6) * (86400) rising edges = 4.32 x 10^12 rising edges.
Therefore, the Counter needs log_2_(4.32x10^12) = 41.97 = 42-bit length Rising-Edge Counter
Total FlipFlops required = 42
Need to perform integer division by 50 x 10^6 on the counter to get the seconds. Although the remainder (by Modulus 50 x 10^6) can give precise decimal seconds, it is usually not needed as there are only 6 SSEG displays on the DE1-SoC board, making it superfluous and unusable.

Option 2: One Counter (Seconds)
Counter requires a Modulo-50MHz counter to add 1 to itself. Max value is 86400 (seconds).
Seconds Counter needs log_2_(86400) = 16.40 = 17-bit length Seconds Counter
Total FF required = 17
Requires continuous assignment logic to factor states of Seconds Counter into required HH:MM:SS.

Option 3: Three Counters (Hours, Minutes and Seconds)
Separate Counters for Hours (0-23), Minutes (0-59) and Seconds (0-59).
- Seconds Counter needs Modulo-60 to reset to 0 and adds 1 to Minutes Counter.
-- Needs log_2_(59) = 5.88 = 6-bit length Seconds Counter.
- Minutes Counter similar to Seconds Counter: Modulo-60 to reset to 0 and adds 1 to Hours Counter.
-- Needs log_2_(59) = 5.88 = 6-bit length Minutes Counter.
- Hours Counter needs Modulo-24 counter and resets to 0 for next day.
-- Needs log_2_(23) = 4.52 = 5-bit length Hours Counter.
- Total FF required = 6 + 6 + 5 = 17
- The 3 Counters encode the HH:MM:SS states directly. No need for additional logic

Option 4: Six Counters (Tens and Ones Counters for Hours, Minutes and Seconds)
Can use parametrised Counters, just need different values for each of the 6 Counters
Max 5/9 for Tens/Ones for Minutes and Seconds. FF = 4+4+4+4 = 16
Max 2/9 for Tens/Ones for Hours, with additional logic condition of 2/3. FF = 2+4 = 6
Total FF required = 16 + 6 = 22
Although only 2 parametrised modules are required (Tens and Minutes) to represent HH:MM:SS, additional logic is required to ensure HH:MM:SS properly loops back to 0, with HH having more.

In this assignment we use the terminal for four things (in the next assignment, it will also be used to run a formal verification tool):

1. More thorough linting (`verilator -Wall --lint-only`).
2. Compiling code (`iverilog -Wall -g2012`).
3. Running test benches (`iverilog -Wall -g2012`, then `./a.out`).
4. Running the automated test suite (`pytest`).

6.2. A Parametrised Up-Down Counter with Enable

You must follow the development cycle for every module (Section 5). Among other things, this includes eliminating all Verilator lint warnings and errors, generating and including waveforms in your report, and ensuring that all automated tests pass.

```
module up_down_counter #(
    parameter int MAX   = 2,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic enable,
    input logic up,
    output logic [WIDTH-1:0] count
);
```

The `count` is initialised to 0.

When `enable` is low, `count` does not change.

When `enable` is high, `count` increments if `up` is high and decrements if `up` is low, wrapping from `MAX` to 0 on increment and from 0 to `MAX` on decrement.

Rationale To edit time conveniently, we must be able to increment and decrement the count.

Implementation Notes It is conventional[1] to gate `enable` inside the `always_ff` block, even though it could instead be absorbed into the next-state logic.

```
always_ff @(posedge clk) if (enable) count <= next_count;
```

Use `always_comb` for the next-state logic.

Verilator will flag bit-width mismatches; use SystemVerilog's cast notation to adjust the width of a value explicitly.

[1] This mirrors real flip-flops, many of which have a dedicated clock-enable pin.

rtl > up_down_counter.sv > Verible > up_down_counter

```sv
// Up_Down_Counter
// To act as a standard Counter, whose modulus value is determined by the
// MAX parameter argument that was passed into it. The counter will count up
// when the up input is high, and count down when the up input is low. The
// counter will wrap back to 0 when the maximum value is reached, and wrap
// back to the maximum value when the minimum value is reached for up and
// down counting, respectively.
//
// Parameters:
//   MAX    - Maximum count value (default: 2).
//   WIDTH  - Width of the count output (default: 2).
//
// Ports:
//   clk    - Clock input.
//   enable - When high, the counter is enabled.
//   up     - When high, the counter counts up; when low, counts down.
//

`timescale 1ns / 1ps

module up_down_counter #(
    parameter int MAX   = 2,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic enable,
    input logic up,
    output logic [WIDTH-1:0] count = WIDTH'(0)
);

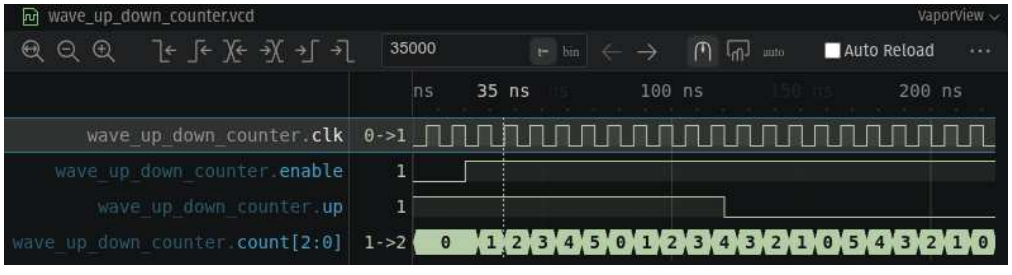
  // Declare local parameters and signals
  // Cast input `MAX` parameter to local `Max` parameter
  localparam logic [WIDTH-1:0] Max = WIDTH'(MAX);
  logic [WIDTH-1:0] next_count;

  // State, and Output Logic
  always_ff @(posedge clk) if (enable) count <= next_count;

  // Next state logic
  always_comb begin
    if (up) begin  // up=1 -> Count up
      if (count < Max) next_count = count + WIDTH'(1);
      else next_count = WIDTH'(0);  // Wrap to 0 when Max
    end else begin  // else up=0 -> Count down
      if (count > WIDTH'(0)) next_count = count - WIDTH'(1);
      else next_count = Max;  // Wrap to Max when 0
    end
  end

```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/up_down_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/up_down_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/up_down_counter.sv tb/wave_up_down_counter.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_up_down_counter.vcd opened for output.
tb/wave_up_down_counter.v:26: $finish called at 220000 (1ps)
root@4034a70d84cc:/workspace#
```



When counter is 'enabled' and is going 'up', the `count` increments as expected. When 'down', the count, decrements as expected

the counter ticks on every rising edge when enabled

```
root@4034a70d84cc:/workspace# pytest tests/test_up_down_counter.py
========================== test session starts ==========================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 3 items

tests/test_up_down_counter.py ...                                  [100%]

========================== 3 passed in 0.93s ==========================
root@4034a70d84cc:/workspace#
```

6.3. Binary to Binary-Coded Decimal

```
module binary_to_bcd (
    input  logic [6:0] bin,   // binary input, 0-99
    output logic [3:0] tens,  // decimal tens digit (BCD)
    output logic [3:0] ones   // decimal ones digit (BCD)
);
```

For any input value `bin` in the range 0 to 99, the output `tens` represents the decimal tens digit of `bin`, and the output `ones` represents the decimal ones digit.

Rationale Displaying hours, minutes and seconds on seven-segment displays requires a binary to binary-coded decimal converter.

> **Report 4**
>
> Explain what binary-coded decimal (BCD) is. You may need to research it.

Implementation Notes The linter expects the same bit width on both sides of a division. Since `bin` is 7 bits wide, we must write `bin / 7'd10`.

Writing `assign tens = bin / 7'd10` will still produce a warning: the LHS (left-hand side) is 4 bits wide, but the RHS (right-hand side) is 7 bits wide. As we genuinely want to discard the leading bits, we must explicitly convert the result to 4 bits.

```
assign tens = 4'(bin / 7'd10);
```

This may seem pedantic when you want to finish the assignment quickly, but in mission-critical designs, you must always be conscious of bit width. The linter forces you to be explicit about your intent.

When viewing the waveforms, set the viewer format for `bin` to decimal.

https://www.electronics-tutorials.ws/binary/binary-coded-decimal.html

What is a Binary Coded Decimal Number?

Binary Coded Decimal (BCD) is another type of binary encoding used in digital and electronic circuits. Unlike the standard binary numbering system, which converts the entire number, BCD converts each digit individually, making it ideal for 7-segment displays and input keyboards.

BCD For Driving Output Displays

Nevertheless, the use of a BCD coding system in both microelectronics and computer systems is particularly useful in situations where the binary coded decimal is intended to be displayed on one or more 7-segment LED or LCD displays and there are many popular integrated circuits available that are configured to give a BCD output or outputs.

Binary Coded Decimal Summary

We have seen here that **Binary Coded Decimal** or **BCD** is simply the 4-bit binary code representation of a decimal number. Each decimal digit replaced in the integer and fractional parts with its binary equivalent. BCD Code uses four bits to represent the 10 decimal digits of 0 to 9.

The main advantage of the **Binary Coded Decimal** system is that it is a fast and efficient system to convert the decimal numbers into binary numbers as compared to the pure binary system. But the BCD code is wasteful as many of the 4-bit states (10-to-16) are not used.

This because with 4 bits, we can represent sixteen numbers (0000 to 1111) in binary. However, in the BCD code only the first ten are used (0000 to 1001). The remaining six code combinations, i.e., 1010 to 1111 (A to F in Hex) are invalid in the binary coded decimal numbering format.

Binary-Coded Decimal (BCD) is the 4-bit binary code representation of a decimal number

rtl > binary_to_bcd.sv > ...

```sv
// Binary to Binary-Coded Decimal (BCD) Converter
// Displaying hours, minutes, and seconds on seven-segement displays
// requires a binary to binary-coded decimal (BCD) converter.
// This module converts a 7-bit binary input (0-99) into two
// 4-bit BCD outputs representing the tens and ones digits.
//
// Note: Module allows conversion up to d'127 even though the
// maximum expected input is d'99. The BCD outputs will be d'12 and
// d'7 for an input of d'127.
//
// Parameters:
//   None
//
// Ports:
//   bin       [6:0]   - Binary input b'(0-110_0011) for d'(0-99).
//   tens      [3:0]   - BCD output for the tens digit.
//   ones      [3:0]   - BCD output for the ones digit.
//

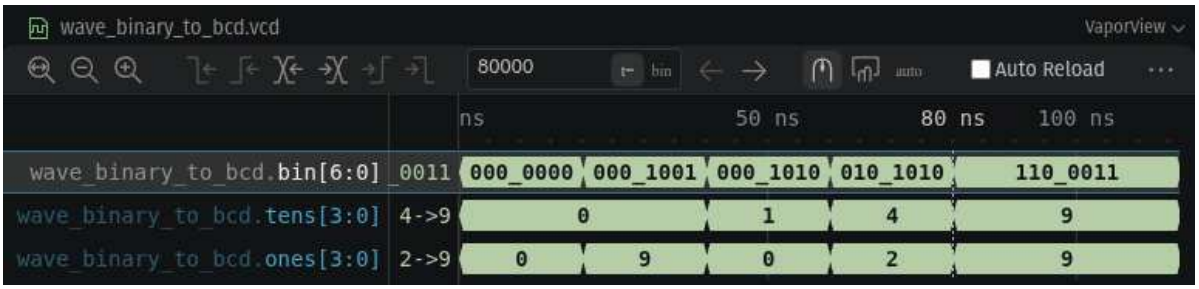
`timescale 1ns / 1ps

module binary_to_bcd (
    input  logic [6:0] bin,   // binary input, b'(0-110_0011)
    output logic [3:0] tens,  // decimal tens digit (BCD)
    output logic [3:0] ones   // decimal ones digit (BCD)
);

  assign tens = 4'(bin / 7'd10);  // Calculate the tens digit
  assign ones = 4'(bin % 7'd10);  // Calculate the ones digit

endmodule
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/binary_to_bcd.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/binary_to_bcd.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/binary_to_bcd.sv tb/wave_binary_to_bcd.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_binary_to_bcd.vcd opened for output.
tb/wave_binary_to_bcd.v:37: $finish called at 100000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_binary_to_bcd.py
========================= test session starts ==========================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_binary_to_bcd.py .                                    [100%]

========================== 1 passed in 0.31s ===========================
root@4034a70d84cc:/workspace#
```

Output `tens` needs integer division by 10 to get the tens unit
Output `ones` need modulus decimal 10 to get the remainder for the ones unit
Input `bin` is 7-width, capable of holding (2^7 - 1) a maximum number 127. So it is possible to output 10, 11 and 12 since the output width of `tens` is 4 (2^4-1) giving max 15 base 10. Whether subsequent modules are able to utilise this is up to the design.

6.4. Cascaded Hour-Minute-Second Counter with Enable

```
module hms_counter #(
    parameter int N_HOURS   = 24,  // number of hours
    parameter int N_MINUTES = 60,  // number of minutes
    parameter int N_SECONDS = 60,  // number of seconds

    // Output port widths
    parameter int W_HOURS   = 5,
    parameter int W_MINUTES = 6,
    parameter int W_SECONDS = 6
) (
    input logic clk,
    input logic enable,
    output logic [W_HOURS-1:0] hours,
    output logic [W_MINUTES-1:0] minutes,
    output logic [W_SECONDS-1:0] seconds
);
```

When `enable` is low, all outputs retain their values.

When `enable` is high, `seconds` increments on each rising clock edge. On a seconds rollover (when `seconds` is at `N_SECONDS-1` and `enable` is high), `seconds` wraps to zero and `minutes` increments. Likewise, on a minutes rollover, `minutes` wraps to zero and `hours` increments; on an hours rollover, `hours` wraps to zero.

Rationale We introduced parameters for testing, while deliberately keeping the module name specific rather than generic. Smaller parameter values do more than speed up simulation: they make bugs easier to find, since the test bench is shorter and debugging is more manageable. Had we intended this module to be reused as a general cascaded counter, we would have given it a more generic name instead. Because its name is specific, it is fine to instantiate `hms_counter` without overriding any parameters: the defaults already implement exactly what the name promises.

Implementation Notes Follow these conventions in the implementation:

- Declare two `logic` variables named `second_rollover` and `minute_rollover`.
- Instantiate three `up_down_counter` modules called `u_hour`, `u_minute`, and `u_second`. The `u_` prefix, short for unit, marks these as module instances and avoids name clashes with signals.

To eliminate linter warnings, we must use width-adjusted parameter versions. The following contains a subtle bug:

```
// Incorrect!
localparam logic [W_MINUTES-1:0] NMinutes = W_MINUTES'(N_MINUTES);
localparam logic [W_SECONDS-1:0] NSeconds = W_SECONDS'(N_SECONDS);
```

The widths passed to the module are suitable for storing numbers from 0 to $N-1$, but the code above stores N using the same number of bits. In edge cases, such as $N = 2$, it stores 0 instead of 2 due to truncation that you explicitly authorised.

Because the code needs $N-1$ rather than N, the solution is straightforward:

```
localparam logic [W_MINUTES-1:0] MaxMinutes = W_MINUTES'(N_MINUTES - 1);
localparam logic [W_SECONDS-1:0] MaxSeconds = W_SECONDS'(N_SECONDS - 1);
```

Careful bit-width tracking is vital: writing a logic condition like `x == y+1` is dangerous because `y+1` might wrap around to zero. This danger exists in other programming languages too, but it is most apparent in hardware design, where we work with small numbers of bits. The extra effort to be precise with widths costs minutes and can save hours of debugging.

For linting and compilation, include all required files.

```
verilator -Wall --lint-only rtl/hms_counter.sv rtl/up_down_counter.sv
iverilog -Wall -g2012 rtl/hms_counter.sv rtl/up_down_counter.sv tb/
    wave_hms_counter.v
```

rtl > hms_counter.sv > SystemVerilog - Language Support > hms_counter

```
// HMS Counter
// This module implements a Hours-Minutes-Seconds (HMS) counter using three
// up-down counters. The counter increments the seconds, minutes, and hours
// based on the enable signal and rolls over when reaching the maximum values.
//
// The hours, minutes and seconds are `ticked up` by the up_down_counter
// modules which are instantiated for each time unit. The up_down_counter
// modules are the ones that handle the wrap back to 0 when the maximum value
// is reached, as determined by the MAX parameter argument that was passed.
// into it. Up is always up.
// The rollover is handled here, at the Next-State Logic section, which
// checks if the seconds or minutes have reached their maximum values and sets
// the corresponding rollover signals accordingly.
//
// Parameters:
//   N_HOURS:             - Number of hours (default: 24).
//   N_MINUTES:           - Number of minutes (default: 60).
//   N_SECONDS:           - Number of seconds (default: 60).
//   W_HOURS:             - Width of the hours output (default: 5).
//   W_MINUTES:           - Width of the minutes output (default: 6).
//   W_SECONDS:           - Width of the seconds output (default: 6).
//
// Ports:
//   clk:                    - Clock input.
//   enable:                 - When high, the counter is enabled.
//   hours [W_HOURS-1:0]:    - Output for hours   (0 to N_HOURS-1).
//   minutes [W_MINUTES-1:0]: - Output for minutes (0 to N_MINUTES-1).
//   seconds [W_SECONDS-1:0]: - Output for seconds (0 to N_SECONDS-1).
//

`timescale 1ns / 1ps

module hms_counter #(
    parameter int N_HOURS   = 24,  // number of hours
    parameter int N_MINUTES = 60,  // number of minutes
    parameter int N_SECONDS = 60,  // number of seconds

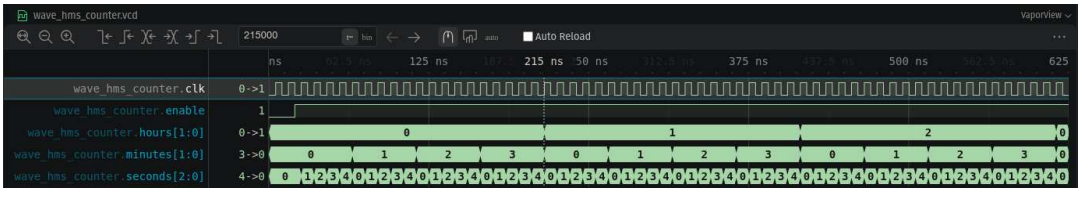
    // Output Port Widths
    parameter int W_HOURS   = 5,  // bit length 5'b for hours (0-23)
    parameter int W_MINUTES = 6,  // bit length 6'b for minutes (0-59)
    parameter int W_SECONDS = 6   // bit length 6'b for seconds (0-59)
) (
    input  logic                clk,
    input  logic                enable,
    output logic [  W_HOURS-1:0] hours,    // = W_HOURS'(0),
    output logic [W_MINUTES-1:0] minutes,  // = W_MINUTES'(0),
    output logic [W_SECONDS-1:0] seconds   // = W_SECONDS'(0)
);

```

rtl > hms_counter.sv > ...

```
47      output logic [W_SECONDS-1:0] seconds   // = W_SECONDS'(0)
48  );
49
50      // Declare Local Parameters and Signals
51      localparam logic [W_MINUTES-1:0] MaxMinutes = W_MINUTES'(N_MINUTES - 1);
52      localparam logic [W_SECONDS-1:0] MaxSeconds = W_SECONDS'(N_SECONDS - 1);
53      logic second_rollover, minute_rollover;
54
55      // State Logic
56      up_down_counter #(
57          .MAX  (N_HOURS - 1),
58          .WIDTH(W_HOURS)
59      ) u_hour (
60          .clk(clk),
61          .enable(minute_rollover),
62          .up(1'b1),
63          .count(hours)
64      );
65
66      up_down_counter #(
67          .MAX  (N_MINUTES - 1),
68          .WIDTH(W_MINUTES)
69      ) u_minute (
70          .clk(clk),
71          .enable(second_rollover),
72          .up(1'b1),
73          .count(minutes)
74      );
75
76      up_down_counter #(
77          .MAX  (N_SECONDS - 1),
78          .WIDTH(W_SECONDS)
79      ) u_second (
80          .clk(clk),
81          .enable(enable),
82          .up(1'b1),
83          .count(seconds)
84      );
85
86      // Next-State Logic
87      always_comb begin
88        if (minutes == MaxMinutes && seconds == MaxSeconds) minute_rollover = 1'b1
89        else minute_rollover = 1'b0;
90        if (seconds == MaxSeconds) second_rollover = 1'b1;
91        else second_rollover = 1'b0;
92      end
93
94  endmodule
95
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/hms_counter.sv rtl/up_down_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/hms_counter.sv rtl/up_down_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/hms_counter.sv rtl/up_down_counter.sv tb/wave_hms_counter.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_hms_counter.vcd opened for output.
tb/wave_hms_counter.v:39: $finish called at 620000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_hms_counter.py
========================= test session starts ==========================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 3 items

tests/test_hms_counter.py ...                                    [100%]

========================== 3 passed in 0.97s ===========================
root@4034a70d84cc:/workspace#
```

This is where module parameters come in handy.
It allows reusability by allowing instantiation with different parameters using the same module.
So rather than writing three individual modules representing HH, MM and SS. Write a counter instead with parametrised maximum values (and bit widths)

6.5. Modulo N Counter

```
module mod_n_counter #(
    parameter int N = 4,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic rst,
    input logic enable,
    output logic [WIDTH-1:0] count
);
```

The output `count` is initially zero.

On a rising clock edge, `rst` takes priority over `enable`: a high `rst` clears `count` regardless of `enable`.

If only `enable` is high, `count` advances. When it reaches $N - 1$, the next enabled edge wraps it back to zero; otherwise it increments by one.

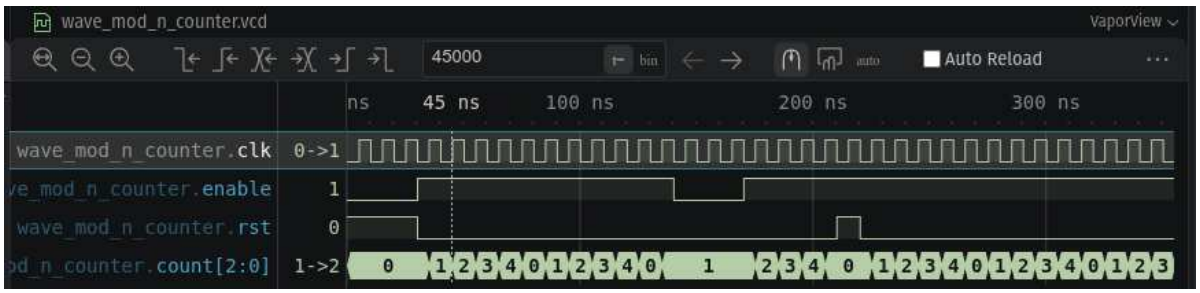
Rationale Even if `up_down_counter` had a reset pin, a dedicated module with a minimal interface makes the design intent clearer than one with several pins tied high or low.

Implementation Notes Refer to your previous modules for the techniques needed to suppress linter warnings.

It is customary[2] to place reset logic inside the flip-flop block rather than in the next-state logic. Use `always_ff @(posedge clk)` rather than `always @(posedge clk)`; the linter will not flag the latter, but `always_ff` makes the intent explicit.

```
always_ff @(posedge clk)
  if (rst) count <= '0;
  else if (enable) count <= next_count;
```

[2]This mirrors real flip-flops, many of which have a dedicated reset pin.



The testbench puts N as 5, so it instantiates a Modulo-5 counter. The counter is expected to range from 0 to 4, afterwhich it rollsover back to 0. This module also has a `rst` input signal that allows the counter to be asynchronously reset back to 0 regardless whether `enable` is HIGH.

rtl > mod_n_counter.sv > ...

```sv
// Modulo N Counter
// This module implements a modulo N counter that counts from 0 to N-1 and wraps
// around to 0 when it reaches N. The counter can be reset asynchronously and is
// enabled by an input signal.
//
// Parameters:
//   N      - Maximum count value (default: 4).
//   WIDTH  - Width of the count output (default: 2).
//
// Ports:
//   clk    - Clock input.
//   rst    - Asynchronous reset input.
//   enable - When high, the counter is enabled.
//   count  - Output count value (0 to N-1).
//

`timescale 1ns / 1ps

module mod_n_counter #(
    parameter int N = 4,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic rst,
    input logic enable,
    output logic [WIDTH-1:0] count = WIDTH'(0)
);

  // Declare local parameters and signals
  // Cast input `N` parameter to local `Max` parameter
  localparam logic [WIDTH-1:0] Max = WIDTH'(N - 1);
  logic [WIDTH-1:0] next_count;

  // State, and Output Logic
  always_ff @(posedge clk) begin
    if (rst) count <= WIDTH'(0);  // Asynchronous reset to 0
    else if (enable) count <= next_count;
  end

  // Next-State Logic
  always_comb begin
    if (count < Max) next_count = count + WIDTH'(1);
    else next_count = WIDTH'(0);  // Wrap to 0 when Max
  end

endmodule
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/mod_n_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/mod_n_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/mod_n_counter.sv tb/wave_mod_n_counter.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_mod_n_counter.vcd opened for output.
tb/wave_mod_n_counter.v:47: $finish called at 350000 (1ps)
root@4034a70d84cc:/workspace#
```

```
root@4034a70d84cc:/workspace# pytest tests/test_mod_n_counter.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 2 items

tests/test_mod_n_counter.py ..                                           [100%]

============================== 2 passed in 0.65s ===============================
root@4034a70d84cc:/workspace#
```

6.6. Restartable Rate Generator

```
module restartable_rate_generator #(
    parameter int CYCLE_COUNT = 2
) (
    input  logic clk,
    input  logic run,
    output logic tick
);
```

When run is low, tick goes low.

When run is high:

- After run has been high for exactly CYCLE_COUNT-1 rising edges, tick is high for exactly one clock cycle.
- Thereafter, after each period of run having remained high for CYCLE_COUNT rising edges, tick is again high for exactly one clock cycle.

The edge case CYCLE_COUNT = 1 is supported: tick follows run directly.

The module is to be implemented as a Moore FSM.

Rationale This module drives the watch's one-second timekeeping and is reused during time editing, where holding a button causes repeated increments or decrements. Supporting CYCLE_COUNT = 1 ensures the design remains functional when the watch is configured to tick every clock cycle, which is useful for debugging.

There is no functional reason for this module to be a Moore FSM; the requirement is imposed purely as a learning opportunity.

Implementation Notes There is no elegant way to write this module directly: it is just complex enough that a direct implementation becomes awkward. Instantiating a mod_n_counter avoids this, keeping the higher-level logic uncluttered.

In a Moore FSM, outputs depend only on state. Since tick must be low whenever run is sampled low, run must be captured as part of the state.

For CYCLE_COUNT > 1, the remaining state is a counter with modulus CYCLE_COUNT. Its width is an internal concern — it does not affect any port — and must be computed internally rather than imposed on the instantiating module.

```
localparam int CountWidth = $clog2(CYCLE_COUNT);
```

Report 5

Explain how localparam CounterWidth = $clog2(CYCLE_COUNT); works.

CYCLE_COUNT is a (unsigned) integer of width 32
$clog2(number) returns the logarithm of the input number at base 2, rounded to the next integer
It gives the bit width required to hold the input number as a binary string

When CYCLE_COUNT = 1, CounterWidth is zero, so no counter is needed; a generate statement handles this by conditionally omitting the instantiation. Blocks inside a generate statement must be named — here g_general and g_special — or the linter will warn.

```
generate
  if (CYCLE_COUNT > 1) begin : g_general
    // Put code for the general case here,
    // including counter instantiation
  end else begin : g_special
    // Put code for the special case here
  end
endgenerate
```

With readability in mind, we start with the high-level structure:

```
// Becomes high at the end of each cycle
logic tick_qualifier;

logic running = 1'b0;
always_ff @(posedge clk) running <= run;

assign tick = running && tick_qualifier;
```

It remains for the `generate` block to provide `tick_qualifier`. The special case is straightforward:

```
begin : g_special
  assign tick_qualifier = 1'b1;
end
```

A template for the general case follows, with the logic kept separate from the module instantiation.

```
begin : g_general
  localparam int CountWidth = $clog2(CYCLE_COUNT);

  logic rst_count;
  logic enable_count;
  logic [CountWidth-1:0] count;
  mod_n_counter #(
      .N(CYCLE_COUNT),
      .WIDTH(CountWidth)
  ) u_count (
      .clk(clk),
      .rst(rst_count),
      .enable(enable_count),
      .count(count)
  );

  assign rst_count = ...;        // Fill in
  assign enable_count = ...;     // Fill in

  assign tick_qualifier = ...;  // Fill in
end
```

To ensure the correct bit width, use `CountWidth'(CYCLE_COUNT - 1)` when needed.

Report 6

In addition to running the provided test bench, write your own test bench verifying correct behaviour when `CYCLE_COUNT = 1`.

Note The module's behaviour may appear asymmetric: the initial delay is `CYCLE_COUNT-1` rising edges, whereas subsequent ticks repeat every `CYCLE_COUNT` rising edges. However, if `run` is synchronised to the clock (i.e., it transitions just after a rising edge, as it would if driven from a flip-flop), the initial delay is also `CYCLE_COUNT` clock cycles, matching the repeat rate.

```systemverilog
// Restartable Rate Generator
// Generates a Mod-N counter that produces a tick at a specified rate when
// enabled. The rate is defined by the CYCLE_COUNT parameter. The generated
// Mod_CYCLE_COUNT counter can be restarted by deasserting and reasserting
// the run signal.
//
// Parameters:
//   CYCLE_COUNT: - Number of clock cycles per tick (default: 2).
//
// Ports:
//   clk:         - Clock input.
//   run:         - When high, the generated Mod_CYCLE_COUNT instant is
//                  enabled and produces ticks after successfully reaching
//                  each full cycle.
//   tick:        - Output tick signal, high when both run is high and the
//                  tick_qualifier is high.
//
// Local Variables:
//   CountWidth:    - Bit width of parameter CYCLE_COUNT.
//   rst_count:     - Reset signal for the generated `mod_n_counter` counter.
//   enable_count:  - Enable signal for the generated `mod_n_counter` counter.
//   count:         - Count output of the generated `mod_n_counter` counter.
//   tick_qualifier:- High when the generated `mod_n_counter` counter reaches
//                    its maximum value.
//   running:       - Indicates whether the rate generator is enabled based
//                    on input signal `run`.

`timescale 1ns / 1ps

module restartable_rate_generator #(
    parameter int CYCLE_COUNT = 2
) (
    input  logic clk,
    input  logic run,
    output logic tick
);

  // Declare Local Parameters and Signals
  // Get required bit width for the counter based on CYCLE_COUNT
  localparam int CountWidth = $clog2(CYCLE_COUNT);

  // Becomes high at the end of each cycle
  logic tick_qualifier;

  generate
    if (CYCLE_COUNT > 1) begin : g_general
      logic rst_count;
      logic enable_count;
      logic [CountWidth-1:0] count;
          .N(CYCLE_COUNT),
          .WIDTH(CountWidth)
      ) u_count (
          .clk(clk),
          .rst(rst_count),
          .enable(enable_count),
          .count(count)
      );

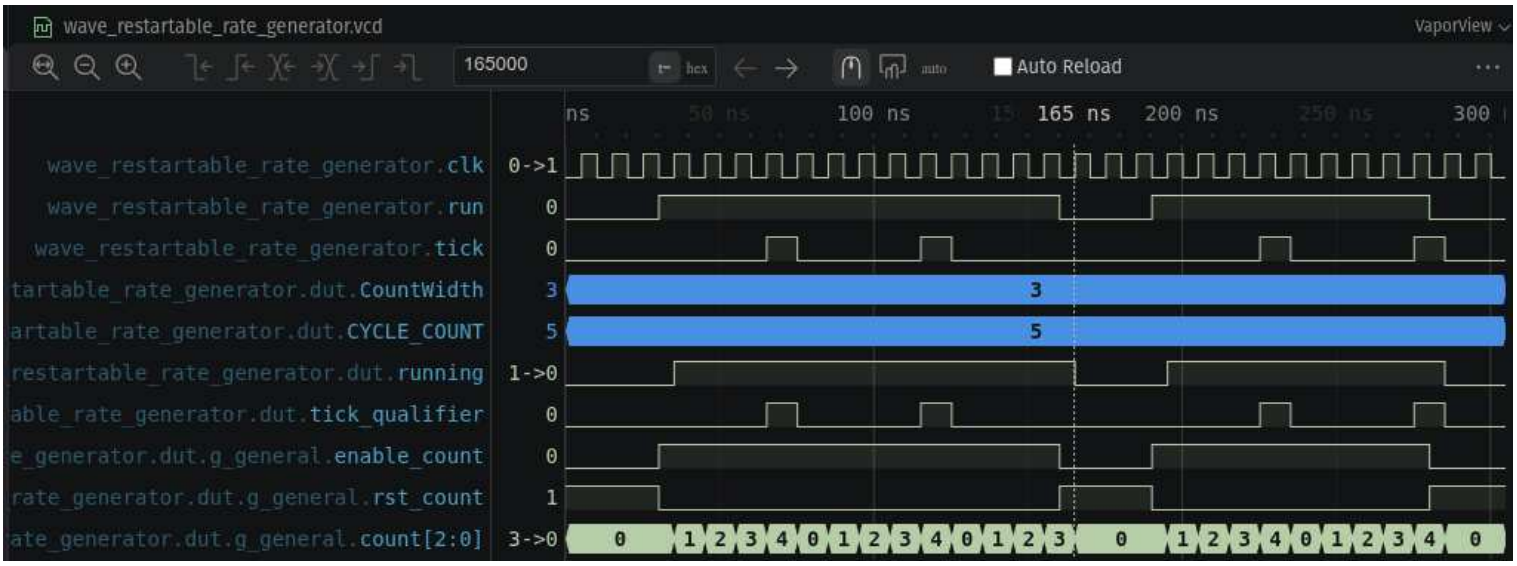
      assign rst_count = !run;
      assign enable_count = run;
      assign tick_qualifier = (count == CountWidth'(CYCLE_COUNT - 1));
    end else begin : g_special
      assign tick_qualifier = 1'b1;
    end
  endgenerate

  // Running state logic
  logic running = 1'b0;
  always_ff @(posedge clk) running <= run;

  // Output Logic
  assign tick = running && tick_qualifier;

endmodule
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/restartable_rate_generator.sv rtl/mod_n_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/restartable_rate_generator.sv rtl/mod_n_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/restartable_rate_generator.sv rtl/mod_n_counter.sv tb/wave_restartable_rate_generator.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_restartable_rate_generator.vcd opened for output.
tb/wave_restartable_rate_generator.v:33: $finish called at 300000 (1ps)
root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_restartable_rate_generator.py
================================== test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 6 items

tests/test_restartable_rate_generator.py ......                                  [100%]

=================================== 6 passed in 1.77s ===================================
(venv) root@4034a70d84cc:/workspace#
```

6.7. Variable-Speed Time Display

```
module top_time_display_v1 #(
    parameter int CYCLES_PER_SECOND = 50_000_000
) (
    input logic CLOCK_50,
    input logic [1:0] SW,
    output logic [6:0] HEX5,
    output logic [6:0] HEX4,
    output logic [6:0] HEX3,
    output logic [6:0] HEX2,
    output logic [6:0] HEX1,
    output logic [6:0] HEX0
);
```

SW[1:0]	Tick Rate
2'b00	1 Hz
2'b01	25 Hz
2'b10	1 kHz
2'b11	50 MHz

This top-level module for the DE1-SoC board displays the time on the seven-segment displays, initialised to 00:00:00, with the tick rate controlled by `SW[1:0]`.

Rationale The goal is to have a working design ready for the DE1-SoC board when you arrive at the lab. Designing and debugging in VS Code beforehand means it should work first time.

Implementation Notes The final code may be around 100 lines but is straightforward. Begin by instantiating:

- one `hms_counter`;
- three `restartable_rate_generators`, one for each of the slower tick rates;
- three `binary_to_bcds`, one each for hours, minutes and seconds; and
- six `seven_segments`, one for each of the digits.

If it is not immediately obvious what to connect to a port, create a named `logic` variable for it and assign it a value using an `assign` statement at the bottom of the module. This keeps instantiation separate from logic and improves readability, provided the variables are named clearly.

The outputs of `hms_counter` are narrower than the `bin` input of `binary_to_bcd`. Explicitly zero-extend them when making each connection, for example `.bin({2'b0, hours})`.

For each `restartable_rate_generator`, use `CYCLES_PER_SECOND` to derive the `CYCLE_COUNT` value. The default of `50_000_000` matches the DE1-SoC's 50 MHz clock; the test suite overrides it to a smaller value to keep simulation fast.

As the design grows, we can save time by telling `verilator` or `iverilog` to search `rtl/` for any modules it needs. For `iverilog`, we also specify `.sv` as the file extension to search. Note that `-Y .sv` must come before `-y rtl`.

```
verilator -Wall --lint-only -y rtl rtl/top_time_display_v1.sv
iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_top_time_display_v1.v
```

The automated tests check the 1 kHz tick rate but not the slower rates (25 Hz and 1 Hz); these can be verified by hand once the design is on the board.

The simulator may produce warnings labelled `Time: 0`, such as the following.

```
root@a374924768bc:/workspace# ./a.out
VCD info: dumpfile wave_top_time_display_v1.vcd opened for output.
WARNING: rtl/seven_segment.sv:30: value is unhandled for priority or
    unique case statement
         Time: 0 Scope: wave_top_time_display_v1.dut.u_HEX0
```

This is due to a race condition: the `unique case` statement in `seven_segment` must have executed before at least one of the `up_down_counter` instances has initialised its output, so some or all bits of `digit` are unknown. Since no case item matches an unknown value, the `unique` constraint is correctly reported as violated. There are three solutions.

1. Ignore `unique case` warnings labelled `Time: 0`.
2. Add a `$isunknown(digit)` check to handle unknown inputs explicitly.
3. Some simulators allow warnings at time 0 to be suppressed.

If these warnings truly bother you, add the following to `seven_segment`:

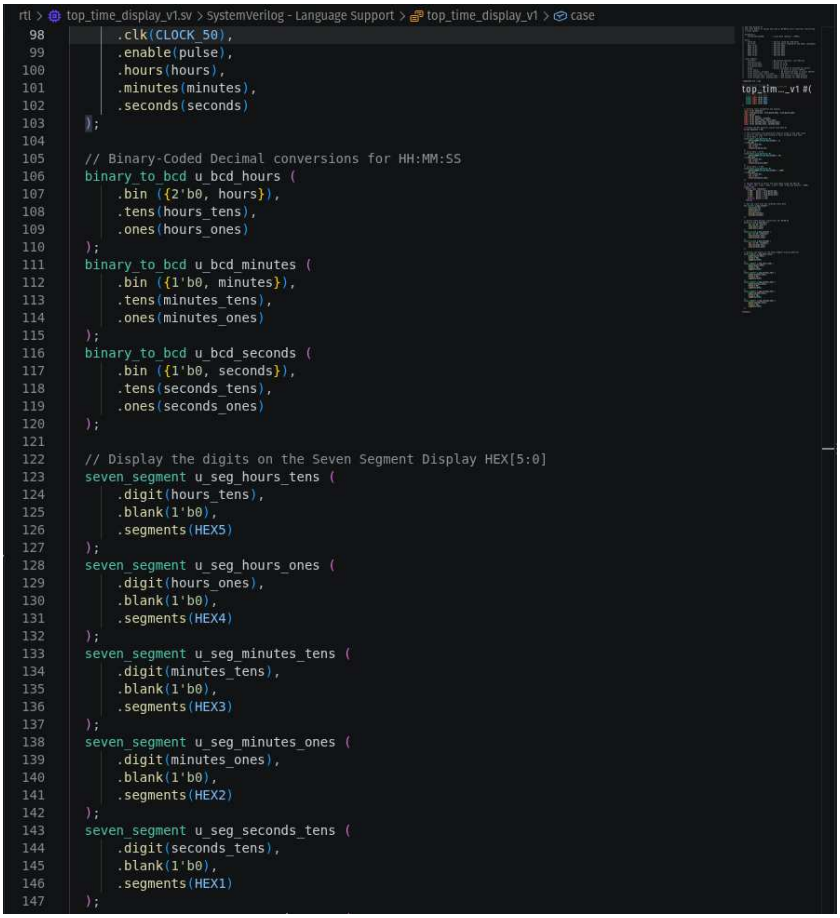
```
if (blank || $isunknown(digit)) segments_active_high = '0;
```

Quartus will ignore this construct, and since the warnings are a simulator quirk, adding extra logic to suppress them is not ideal. The most pragmatic solution is simply to ignore `unique case` warnings labelled `Time: 0`.





```
rtl > top_time_display_v1.sv > SystemVerilog - Language Support > top_time_display_v1 > { } seconds
50    logic [5:0] minutes, seconds;
51    logic [3:0] hours_tens, hours_ones;
52    logic [3:0] minutes_tens, minutes_ones;
53    logic [3:0] seconds_tens, seconds_ones;
54
55    // Assign the Mux Selector switch with SW[1:0]
56    assign selector = SW;
57
58    // The restartable rate generators need to align to the same clock
59    // while at the same time ticking to their assigned clock rate
60    // Clock Rate = 1 Hz
61    restartable_rate_generator #(
62        .CYCLE_COUNT(CYCLES_PER_SECOND / 1)
63    ) u_1hz_gen (
64        .clk (CLOCK_50),
65        .run (1'b1),
66        .tick(tick_pulse_1hz)
67    );
68    // Clock Rate = 25 Hz
69    restartable_rate_generator #(
70        .CYCLE_COUNT(CYCLES_PER_SECOND / 25)
71    ) u_25hz_gen (
72        .clk (CLOCK_50),
73        .run (1'b1),
74        .tick(tick_pulse_25hz)
75    );
76    // Clock Rate = 1 kHz
77    restartable_rate_generator #(
78        .CYCLE_COUNT(CYCLES_PER_SECOND / 1_000)
79    ) u_1khz_gen (
80        .clk (CLOCK_50),
81        .run (1'b1),
82        .tick(tick_pulse_1khz)
83    );
84
85    // The Mux Switch to select the Clock Rate using the SW[1:0]
86    // 2'b00 = 1Hz, 2'b01 = 25Hz, 2'b10 = 1kHz, 2'b11 and default = 50MHz
87    always_comb
88      unique case (selector)
89        2'b00:   pulse = tick_pulse_1hz;
90        2'b01:   pulse = tick_pulse_25hz;
91        2'b10:   pulse = tick_pulse_1khz;
92        2'b11:   pulse = 1'b1;
93        default: pulse = 1'b1;
94      endcase
95
96    // Run the clock with the selected Clock Rate
97    hms_counter u_hms_counter (
98        .clk(CLOCK_50),
99        .enable(pulse),
```

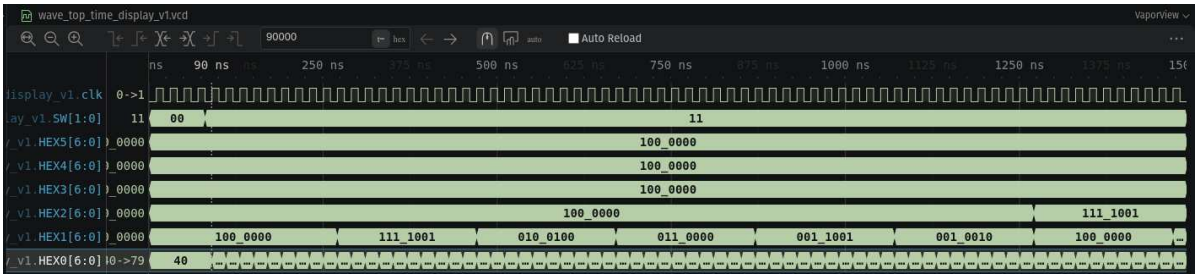


```
rtl > top_time_display_v1.sv > SystemVerilog - Language Support > top_time_display_v1 > case
98        .clk(CLOCK_50),
99        .enable(pulse),
100       .hours(hours),
101       .minutes(minutes),
102       .seconds(seconds)
103   );
104
105   // Binary-Coded Decimal conversions for HH:MM:SS
106   binary_to_bcd u_bcd_hours (
107       .bin ({2'b0, hours}),
108       .tens(hours_tens),
109       .ones(hours_ones)
110   );
111   binary_to_bcd u_bcd_minutes (
112       .bin ({1'b0, minutes}),
113       .tens(minutes_tens),
114       .ones(minutes_ones)
115   );
116   binary_to_bcd u_bcd_seconds (
117       .bin ({1'b0, seconds}),
118       .tens(seconds_tens),
119       .ones(seconds_ones)
120   );
121
122   // Display the digits on the Seven Segment Display HEX[5:0]
123   seven_segment u_seg_hours_tens (
124       .digit(hours_tens),
125       .blank(1'b0),
126       .segments(HEX5)
127   );
128   seven_segment u_seg_hours_ones (
129       .digit(hours_ones),
130       .blank(1'b0),
131       .segments(HEX4)
132   );
133   seven_segment u_seg_minutes_tens (
134       .digit(minutes_tens),
135       .blank(1'b0),
136       .segments(HEX3)
137   );
138   seven_segment u_seg_minutes_ones (
139       .digit(minutes_ones),
140       .blank(1'b0),
141       .segments(HEX2)
142   );
143   seven_segment u_seg_seconds_tens (
144       .digit(seconds_tens),
145       .blank(1'b0),
146       .segments(HEX1)
147   );
```



```
143   seven_segment u_seg_seconds_tens (
144       .digit(seconds_tens),
145       .blank(1'b0),
146       .segments(HEX1)
147   );
148   seven_segment u_seg_seconds_ones (
149       .digit(seconds_ones),
150       .blank(1'b0),
151       .segments(HEX0)
152   );
153
154   endmodule
155
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/top_time_display_v1.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/top_time_display_v1.sv
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/top_time_display_v1.sv:88: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/top_time_display_v1.sv tb/wave_top_time_display_v1.v
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/top_time_display_v1.sv:88: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_top_time_display_v1.vcd opened for output.
tb/wave_top_time_display_v1.v:34: $finish called at 1480000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_top_time_display_v1.py
================================== test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_top_time_display_v1.py .                                              [100%]

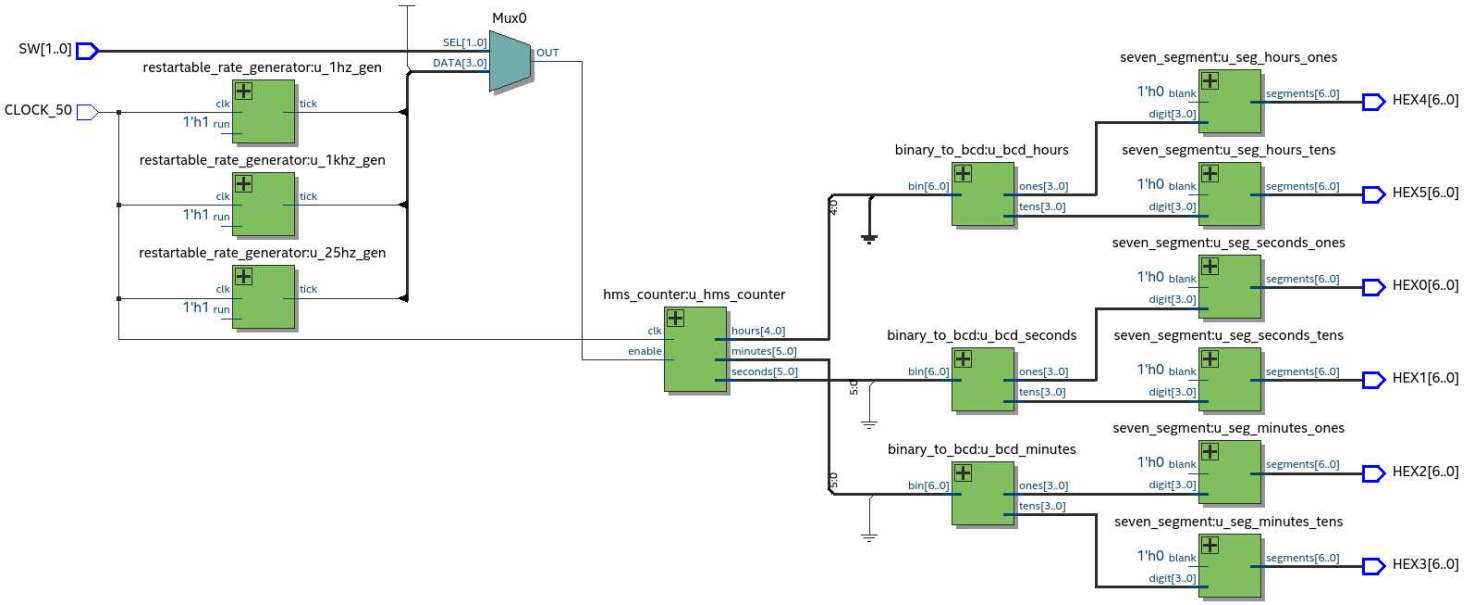
=================================== 1 passed in 0.30s ===================================
(venv) root@4034a70d84cc:/workspace#
```

6.8. Programming the Board

Skip to the next section if you are not in the lab; return here when you are ready to program the board.

Follow these steps to program the DE1-SoC:

1. Create a new Quartus project (e.g., `digital_watch`) with the correct settings for the DE1-SoC board.
2. Set the top-level module to `top_time_display_v1`.
3. Import assignments from `DE1_SoC.qsf`.
4. Create an `.SDC` file specifying the clock period.
5. Copy all your `rtl/*.sv` files into the Quartus project root directory (or under `rtl/` if you prefer).
6. Go to Project » Add/Remove Files and add these files to the project.
7. Compile and program the board.



7. Settable Watch

By the end of this section, we will have a settable watch: time displayed on the DE1-SoC board, incrementing once per second, with the ability to edit the time using the keys. Holding `KEY[3]` for at least one second enters edit mode, with the seconds digits flashing at 2 Hz with an 80% duty cycle. A second press of `KEY[3]` moves to the minutes digits, a third to the hours digits, and a fourth exits edit mode.

In edit mode, `KEY[1]` increments and `KEY[0]` decrements the flashing digits. Holding either key for more than half a second repeats the increment or decrement at 10 Hz.

7.1. Fixed-Frequency Fixed-Duty PWM Generator

```
module pwm_generator #(
    // Number of clock cycles in one PWM period
    parameter int PERIOD_CYCLES = 50_000_000,

    // Number of clock cycles output is high
    parameter int DUTY_CYCLES = 25_000_000
) (
    input  logic clk,
    input  logic rst,
    output logic pwm_out
);
```

The output `pwm_out` is periodic with a period of `PERIOD_CYCLES` clock cycles, high for the first `DUTY_CYCLES` cycles of each period and low for the remainder. When `rst` is asserted, the output restarts from the next rising clock edge.

Report 7

Explain why this module is called a PWM generator.

Rationale During edit mode, the display flashes to indicate which field is being edited. This module provides the PWM signal that drives the flashing.

Implementation Notes Instantiate a `mod_n_counter`. To see its port names while working, right-click `mod_n_counter.sv` in the Explorer panel and select *Open to the Side*.

Report 8

In `mod_n_counter`, `rst` takes priority over `enable`. Comment on the benefit of this design choice.

See 2 pages below

https://en.wikipedia.org/wiki/Pulse-width_modulation
https://www.sameskydevices.com/blog/pulse-width-modulation-pwm-how-it-works-and-why-its-essential-in-electronics

Pulse-width modulation (**PWM**), also known as **pulse-duration modulation** (**PDM**)[a] or **pulse-length modulation** (**PLM**),[1] is any method of representing a signal as a rectangular wave with a varying duty cycle (and for some methods also a varying period).

PWM facilitates the portion of the period for which the digital signal is active

Summary

Modulation is the application of control over a device or system. Pulse width modulation in particular, is an effective and efficient method used to control power in a wide variety of electronic devices. It allows for precise control and low-power operation when compared to other control technologies and is used across many applications to exert analog-like control using a digital signal.

rtl > pwm_generator.sv > ...

```
// Pulse Width Modulation (PWM) Generator
// Generates a PWM signal with a specified duty and period cycle. `pwm_out`
// signal is high for `DUTY_CYCLES` clock cycles and low for the remainder
// in `PERIOD_CYCLES` clock cycles. Local `count` signal generated by
// Modulo N Counter
//
// Parameters:
//   PERIOD_CYCLES:    - Number of clock cycles in one PWM period
//                       (default: 50_000_000)
//   DUTY_CYCLES:      - Number of clock cycles output is high
//                       (default: 25_000_000)
//
// Ports:
//   clk:              - Clock input
//   rst:              - Asynchronous reset input
//   pwm_out:          - PWM output signal, high when within duty cycle,
//                       low when outside duty cycle
//
// Local Variables:
//   CountWidth:       - Bit width of parameter PERIOD_CYCLES.

`timescale 1ns / 1ps

module pwm_generator #(
    // Number of clock cycles in one PWM period
    parameter int PERIOD_CYCLES = 50_000_000,

    // Number of clock cycles output is high
    parameter int DUTY_CYCLES = 25_000_000
) (
    input  logic clk,
    input  logic rst,
    output logic pwm_out
);

  // Declare local parameters and signals
  // (+1'b1) is needed here to cover edge cases when PERIOD_CYCLES are powers
  // of 2, otherwise there will be integer overflow when DUTY_CYCLES is equal
  // to PERIOD_CYCLES where localparam `DutyCycles` will equal to 0
  localparam int CountWidth = $clog2(PERIOD_CYCLES) + 1'b1;
  localparam logic [CountWidth-1:0] DutyCycles = CountWidth'(DUTY_CYCLES);
  logic [CountWidth-1:0] count;

  // Instaniate Modulo N Counter with N = PERIOD_CYCLES.
  mod_n_counter #(
      .N(PERIOD_CYCLES),
      .WIDTH(CountWidth)
  ) u_period_cycle_count (
      .clk(clk),
      .rst(rst),
      .enable(1'b1),
      .count(count)
  );

  // `pwm_out` is high as for period of DutyCycles within PeriodCycles,
  // past period of DutyCycles, `pwm_out` is low for rest of PeriodCycles.
  always_comb begin
    if (count < DutyCycles) pwm_out = 1'b1;
    else pwm_out = 1'b0;
  end

endmodule
```

```
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile user_wave_pwm_generator.vcd opened for output.
tb/user_wave_pwm_generator.v:29: $finish called at 440000 (1ps)
(venv) root@4034a70d84cc:/workspace# verilator -Wall -y rtl --lint-only rtl/pwm_generator.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/pwm_generator.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_pwm_generator.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_pwm_generator.vcd opened for output.
tb/wave_pwm_generator.v:29: $finish called at 410000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_pwm_generator.py
================================ test session starts ================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 7 items

tests/test_pwm_generator.py .......                                        [100%]

================================= 7 passed in 2.15s =================================
(venv) root@4034a70d84cc:/workspace#
```

Period Cycles = 10
Duty Cycles = 8
Thus, Pulse-Width Modulator is active for 80% of the period.
This PWM is a simple Modulo-(PERIOD_CYCLES) counter where as long as the counter is less than the stated DUTY_CYCLES, the `pwm-out` signal is HIGH.
DUTY_CYCLES is to be less than PERIOD_CYCLES if any modulation is to take place.
`rst` signal resets the count and thus lengthens the `pwm-out` signal

Report 8

```
34    // State, and Output Logic
35    always_ff @(posedge clk) begin
36      if (rst) count <= WIDTH'(0);  // Asynchronous reset to 0
37      else if (enable) count <= next_count;
38    end
39
```

If `enable` is checked first, `rst` cannot reset the counter as long as `enable` is high.
This forces the end-user to disable the counter first before being able to reset the counter and that means two button presses instead of one.

7.2. Rising-Edge Detector

```
module rising_edge_detector (
    input  logic clk,
    input  logic sig_in,
    output logic rise
);
```

The output `rise` is asserted immediately when `sig_in` transitions from low to high, and deasserted as soon as `sig_in` returns low or the next rising clock edge captures the high value, whichever comes first.

Rationale At 50 MHz, a button press spans millions of clock cycles. Without edge detection, a single press of the edit button could trigger millions of mode transitions. A rising edge detector converts the press into a single-cycle pulse, ensuring exactly one transition per press.

Unlike the other modules in this assignment, `rising_edge_detector` is a Mealy FSM — the industry-standard choice for edge detection — as it avoids a one-cycle output delay, is simpler to write, and produces output logic simple enough to be glitch-free.

rtl > rising_edge_detector.sv > ...

```
// Rising Edge Detector
// The output signal `rise` is asserted immediately when input signal `sig_in`
// transitions from low to high, and deasserted when `sig_in` becomes low OR
// when the next rising edge of the clock captures the `sig_in` high signal,
// whichever comes first.
//
// Parameters:
//   None
//
// Ports:
//   clk:       - Clock Input
//   sig_in:    - Input Signal
//   rise:      - Output Signal, a singular pulse within one clock cycle
//
// Local Variables:
//   prev:      - The State that captures the previous value of `sig_in`
//   next_prev: - Transitional state that captures the current
//                value of `sig_in`

`timescale 1ns / 1ps

module rising_edge_detector (
    input  logic clk,
    input  logic sig_in,
    output logic rise
);

  logic prev = 1'b0;  // State
  logic next_prev;

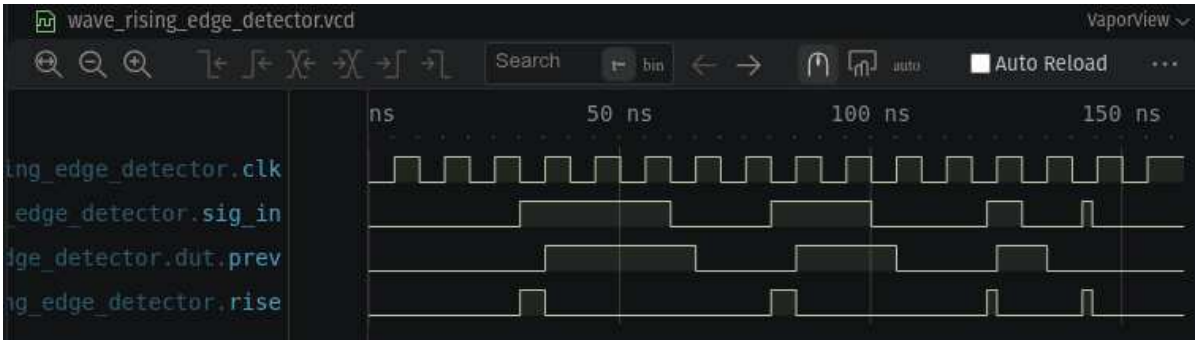
  // Flip-Flip
  always_ff @(posedge clk) begin
    prev <= next_prev;
  end

  // Next-State Logic
  assign next_prev = sig_in;

  // Output Logic
  assign rise = (!prev && sig_in);

endmodule
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/rising_edge_detector.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/rising_edge_detector.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_rising_edge_detector.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_rising_edge_detector.vcd opened for output.
tb/wave_rising_edge_detector.v:48: $finish called at 158000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_rising_edge_detector.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_rising_edge_detector.py .                                     [100%]

============================== 1 passed in 0.35s ===============================
(venv) root@4034a70d84cc:/workspace#
```

Output `rise` is HIGH only for one cycle.
This happens only when the input signal `sig_in` is HIGH for the current cycle and the module's previous rising edge state had a LOW in the `prev` signal
Once the `prev` signal turns HIGH on the rising edge by the HIGH `sig_in` input, the output `rise` becomes LOW.
Thus making this module a Mealy FSM, where the output is dependent on both the state and the input

7.3. Button Hold Detect

```
module button_hold_detect #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input  logic clk,
    input  logic button,
    output logic held
);
```

The output `held` is asserted on the clock edge at which `button` has been sampled high for `HOLD_CYCLES` consecutive rising edges, and deasserts on the clock edge after `button` goes low.

Rationale Entering edit mode requires the user to hold a button for a minimum duration: this module detects when that threshold has been reached.

Implementation Notes Instantiate a `mod_n_counter` and think carefully about its maximum count. The counter value is the entire FSM state: no additional state variables are needed.

It is more natural to reason about this counter in terms of its maximum value than its modulus `N`. Define `CountMax` explicitly to make the intent clear and avoid off-by-one errors.

```
localparam int CountMax = ... ; // Fill this in
localparam int CountWidth = $clog2(CountMax + 1);

logic count_rst;
logic count_enable;
logic [CountWidth-1:0] count;
mod_n_counter #(
    .N(CountMax + 1),
    .WIDTH(CountWidth)
) u_counter (
    .clk(clk),
    .rst(count_rst),
    .enable(count_enable),
    .count(count)
);
```

From here, derive the next-state and output logic. You may find it convenient for the next-state logic to depend on the output `held`: this is permissible.

Report 9

Explain why the next-state logic may depend on the output `held` without violating the Moore FSM constraint.

Use `CountWidth'(CountMax)` where needed to ensure correct bit widths.

Note As in Section 6.6, the hold duration is exactly `HOLD_CYCLES` clock cycles only if the button is synchronised to the clock.

Although the state `held` is used as input to determine the next state, the output condition still only depends on the state `held` and not on the `button` input. Thus not violating the Moore FSM constraint

rtl > button_hold_detect.sv > ...

```sv
// Button Hold Detect
// When the button is pressed for a specified number of clock cycles, returns
// HIGH as long as the button remains pressed.
//
// Parameters:
//    HOLD_CYCLES: - Number of clock cycles to check against to determine HIGH
//                   (default: 50_000_000)
//
// Ports:
//    clk:         - Clock input
//    button:      - Input button signal
//    held:        - Output signal, HIGH when button is HIGH for at least one
//                 - HOLD_CYCLES clock cycles, LOW otherwise
//
// Local Variables:
//    CountMax:       - To hold parameter value (HOLD_CYCLES - 1)
//    CountWidth:     - Bit width of CountMax
//    count_rst:      - Reset signal for `mod_n_counter` u_counter
//    count_enable:   - Enable signal for `mod_n_counter` u_counter
//    count:          - Count output of `mod_n_counter` u_counter
//    held_qualifier: - HIGH when count reaches CountMax, LOW otherwise

`timescale 1ns / 1ps

module button_hold_detect #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input  logic clk,
    input  logic button,
    output logic held = 1'b0
);

  localparam int CountMax = HOLD_CYCLES - 1;
  localparam int CountWidth = $clog2(CountMax + 1);

  logic count_rst;
  logic count_enable;
  logic [CountWidth-1:0] count;
  logic held_qualifier;

  mod_n_counter #(
      .N(CountMax + 1),
      .WIDTH(CountWidth)
  ) u_counter (
      .clk(clk),
      .rst(count_rst),
      .enable(count_enable),
      .count(count)
  );

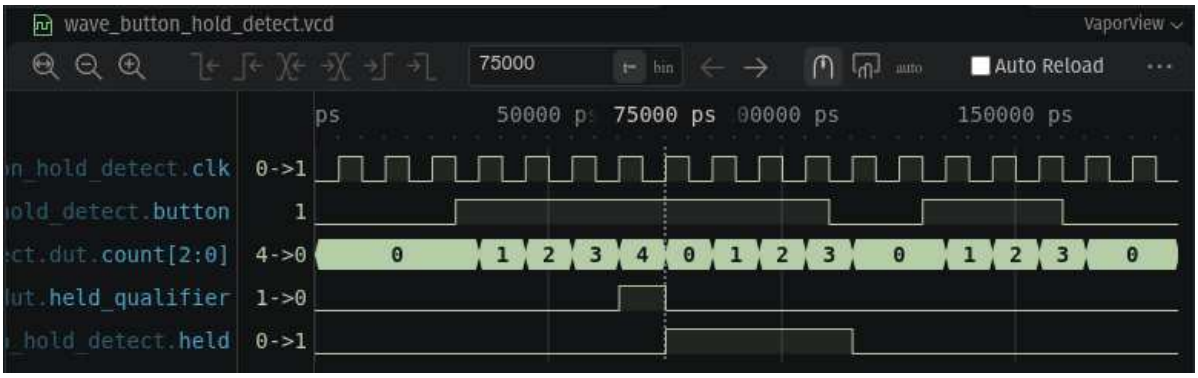
  // Next-State Logic
  assign count_rst = !button;
  assign count_enable = button;
  assign held_qualifier = (count == CountWidth'(CountMax));

  // Flip-Flop, Next-State Logic, and Output Logic
  always_ff @(posedge clk) begin
    if (button && held_qualifier) begin
      held <= 1'b1;
    end else if (button && held) begin
      held <= 1'b1;
    end else begin
      held <= 1'b0;
    end
  end

endmodule

```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/button_hold_detect.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/button_hold_detect.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_button_hold_detect.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_button_hold_detect.vcd opened for output.
tb/wave_button_hold_detect.v:34: $finish called at 180000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_button_hold_detect.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_button_hold_detect.py .                                       [100%]

============================== 1 passed in 0.36s ===============================
(venv) root@4034a70d84cc:/workspace#
```

7.4. Button Hold Pulse

This module is provided in full; nevertheless, go through the full development cycle: run `verilator`, `iverilog`, and the automated tests, and include the waveforms in your report.

```
module button_hold_pulse #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input  logic clk,
    input  logic button,
    output logic pulse
);

  logic held;

  button_hold_detect #(
      .HOLD_CYCLES(HOLD_CYCLES)
  ) u_detect (
      .clk(clk),
      .button(button),
      .held(held)
  );

  rising_edge_detector u_detector (
      .clk(clk),
      .sig_in(held),
      .rise(pulse)
  );

endmodule
```

rtl > button_hold_pulse.sv > ...

```sv
// Button Hold Pulse
// When the button is pressed for a specified number of clock cycles, returns
// HIGH for only one clock cycle, returns LOW while button remains pressed,
// and when released.
//
// Parameters:
//    HOLD_CYCLES: - Number of clock cycles to check against to determine HIGH
//                   (default: 50_000_000)
//
// Ports:
//    clk:         - Clock input
//    button:      - Input button signal
//    pulse:       - Output signal, HIGH for one clock cycle when button is
//                   HIGH for at least one HOLD_CYCLES clock cycles.
//
// Local Variables:
//    held:        - Wire between output .held from `button_hold_detect` and
//                   input .sig_in of `rising_edge_detector`

`timescale 1ns / 1ps

module button_hold_pulse #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input  logic clk,
    input  logic button,
    output logic pulse
);

  logic held;

  button_hold_detect #(
      .HOLD_CYCLES(HOLD_CYCLES)
  ) u_detect (
      .clk(clk),
      .button(button),
      .held(held)
  );

  rising_edge_detector u_detector (
      .clk(clk),
      .sig_in(held),
      .rise(pulse)
  );

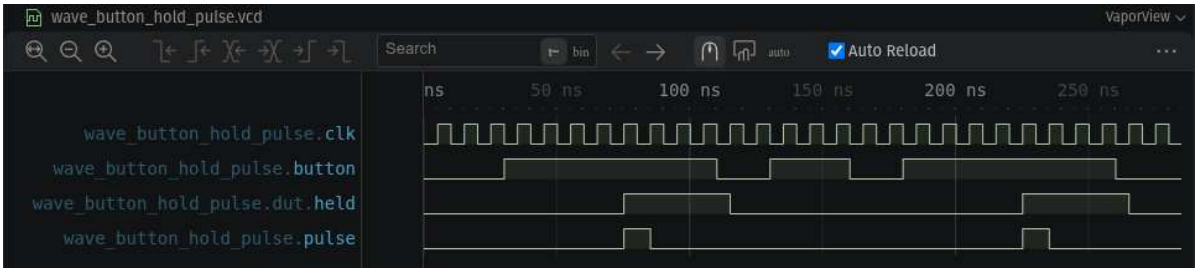
endmodule
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/button_hold_pulse.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/button_hold_pulse.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_button_hold_pulse.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_button_hold_pulse.vcd opened for output.
tb/wave_button_hold_pulse.v:40: $finish called at 280000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```

```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_button_hold_pulse.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_button_hold_pulse.py .                                        [100%]

============================== 1 passed in 0.32s ===============================
(venv) root@4034a70d84cc:/workspace#
```



Report 10

Explain what this module does. Justify your explanation by referring to the generated waveforms, being precise about timing.

Report 11

Explain how `button_hold_pulse` is a Moore FSM despite `rising_edge_detector` being Mealy.

Note Because `rising_edge_detector` is a Mealy FSM, `pulse` asserts on the same clock edge as `held`, rather than one cycle later. A Moore implementation could achieve the same timing by passing `HOLD_CYCLES - 1` to `button_hold_detect`.

Report 10

The Hold Button Detect module take in the button signal and checks for the number of HOLD_CYCLES to pass before releasing the `held` signal. This `held` signal is then passed to the Rising Edge Detector that pulses the `pulse` output and thereafter does nothing as long as the `held` signal is HIGH.

In the waveform, HOLD_CYCLES is 5, and thus `held` becomes HIGH after at least 5 rising edges of the button being held down. After 5 rising edges, the HIGH `held` signal is passed to the Rising Edge Detector which then pulses once as long as the `held` signal remains high, reseting only when the `held` signal is low for at least one rising edge

Report 11

The Button Hold Pulse module is still a Moore FSM as its output `pulse` signal is dependent only on the state given by the Rising Edge Detector module and not on the input state of the button

7.5. Button Auto Repeat

```
module button_auto_repeat #(
    parameter int HOLD_CYCLES   = 50_000_000,
    // REPEAT_CYCLES must be smaller than HOLD_CYCLES
    parameter int REPEAT_CYCLES = 5_000_000
) (
    input  logic clk,
    input  logic button,
    output logic pulse
);

  logic rise;
  logic held;
  logic pulse_train;

  assign pulse = rise | (button & pulse_train);
```

The two internal signals, `rise` and `pulse_train`, are derived as follows.

- `button` → `rising_edge_detector` → `rise`
- `button` → `button_hold_detect` → `restartable_rate_generator` → `pulse_train`

A brief press produces an immediate pulse; holding the button produces a pulse train.

The first pulse of the pulse train occurs after `button` has been sampled high for `HOLD_CYCLES` consecutive rising edges; subsequent pulses repeat every `REPEAT_CYCLES` rising edges.

Rationale When editing the watch, a brief press should advance the time by one step; holding the button should produce rapid repeated advances. This module provides both behaviours.

Implementation Notes Instantiate three modules:

- `rising_edge_detector`
- `button_hold_detect`
- `restartable_rate_generator`

Choose their parameters carefully.

Note The rising edge detector is Mealy, so the overall design is Mealy. There is therefore no reason to avoid gating `pulse_train` with `button`: using `button & pulse_train` eliminates the possibility of a spurious pulse one clock cycle after the button is released.

rtl > button_auto_repeat.sv > ...

```sv
// Button Auto Repeat
// When the button is first pressed, a pulse is sent. As long as the button
// remains pressed for at least HOLD_CYCLES, a pulse is sent every
// REPEAT_CYCLES starting from the end of HOLD_CYCLES.
//
// Parameters:
//    HOLD_CYCLES:   - Number of clock cycles to check against to determine
//                     HIGH (default: 50_000_000)
//    REPEAT_CYCLES: - Number of clock cycles to pass to pulse a HIGH
//                     (default: 5_000_000)
//
// Ports:
//    clk:           - Clock input
//    button:        - Input button signal
//    pulse:         - Output signal, HIGH when button is initially pressed
//                     OR while the button is pressed for at least HOLD_CYCLES
//                     send out a HIGH pulse every REPEAT_CYCLES
//
// Local Variables:
//    rise:          - pulse signal for initial press
//    held:          - signal to signify button being held
//    pulse_train:   - waves of pulses repeating every REPEAT_CYCLES as long as
//                     `held` is HIGH


`timescale 1ns / 1ps

module button_auto_repeat #(
    parameter int HOLD_CYCLES   = 50_000_000,
    // REPEAT_CYCLES must be smaller than HOLD_CYCLES
    parameter int REPEAT_CYCLES = 5_000_000
) (
    input  logic clk,
    input  logic button,
    output logic pulse
);

  logic rise;
  logic held;
  logic pulse_train;

  // Sends out a pulse when the button is initially pressed
  rising_edge_detector u_detector (
      .clk(clk),
      .sig_in(button),
      .rise(rise)
  );

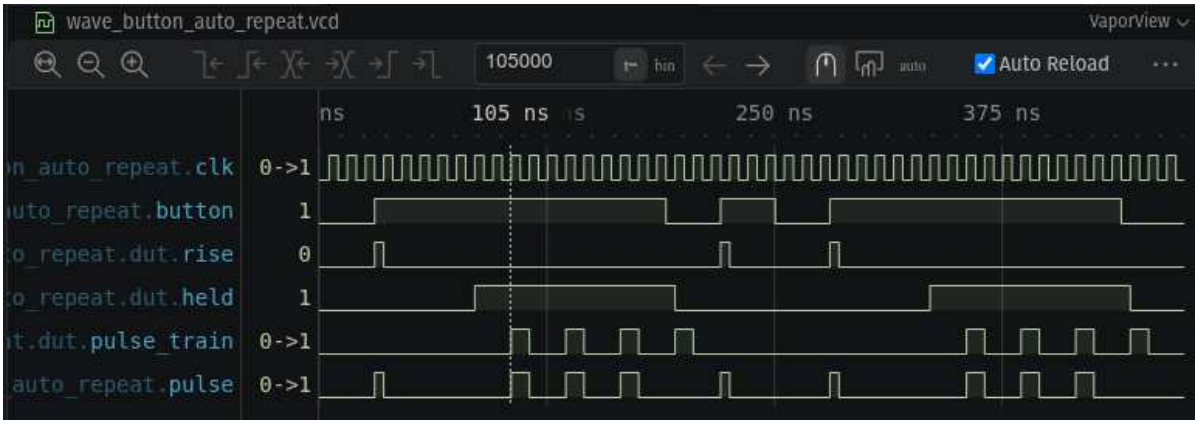
  // Want the pulse to 'tick' upon reaching the HOLD_CYCLE but `pulse_train`
  // only 'ticks' AFTER completing one full REPEAT_CYCLE. So need to offset
  // the starting line by REPEAT_CYCLE
  button_hold_detect #(
      .HOLD_CYCLES(HOLD_CYCLES - REPEAT_CYCLES + 1)
  ) u_detect (
      .clk(clk),
      .button(button),
      .held(held)
  );

  // Repeat Rate = REPEAT_CYCLES
  // pulse_train only ticks when REPEAT_CYCLES is reached only AFTER held
  // is passed in as true
  restartable_rate_generator #(
      .CYCLE_COUNT(REPEAT_CYCLES)
  ) u_1khz_gen (
      .clk (clk),
      .run (held),
      .tick(pulse_train)
  );

  assign pulse = rise | (button & pulse_train);

endmodule
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/button_auto_repeat.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/button_auto_repeat.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_button_auto_repeat.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_button_auto_repeat.vcd opened for output.
tb/wave_button_auto_repeat.v:44: $finish called at 470000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_button_auto_repeat.py
================================= test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_button_auto_repeat.py .                                              [100%]

================================== 1 passed in 0.43s ===================================
(venv) root@4034a70d84cc:/workspace#
```

pulse is dependent on state `button` and input `pulse train` represented by the Restartable Rate Generator module

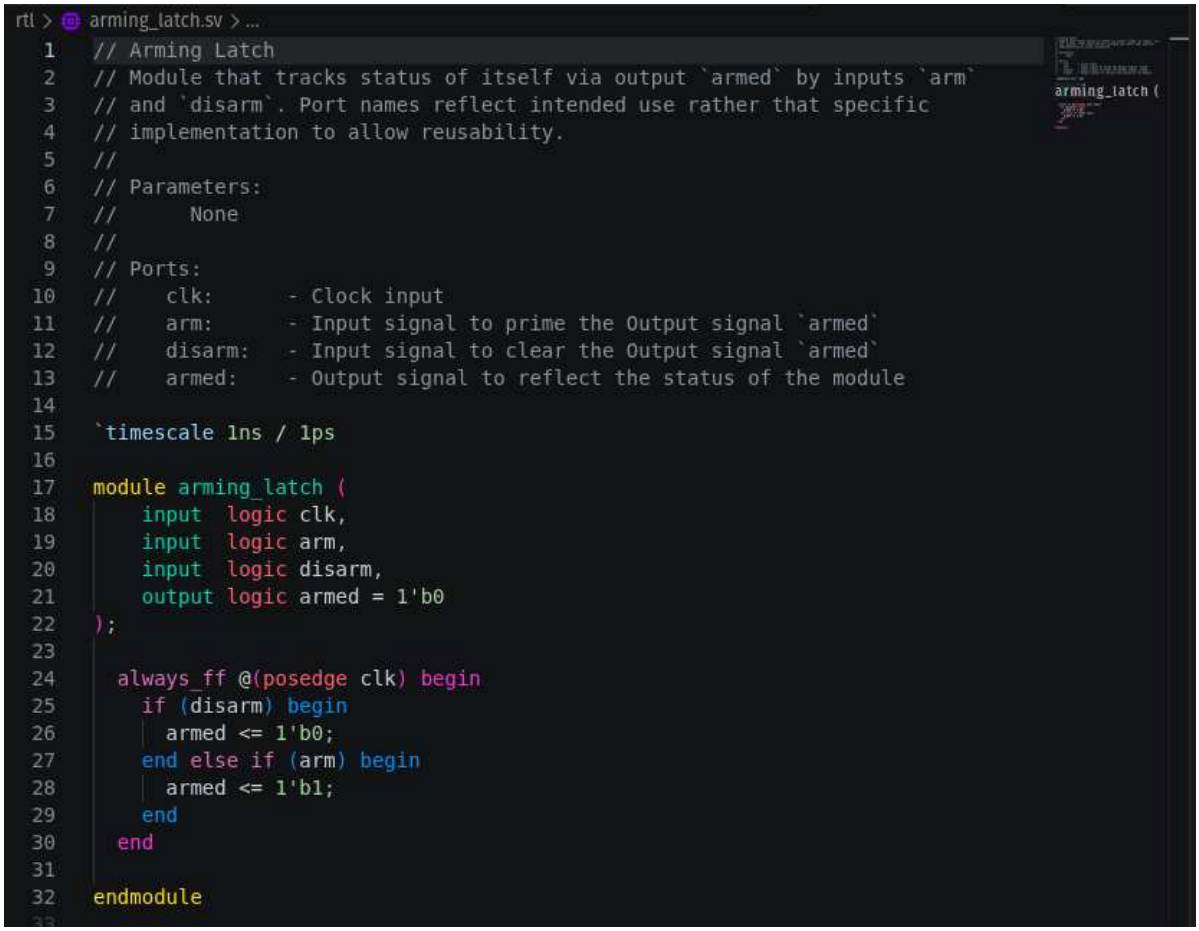
7.6. Arming Latch

```
module arming_latch (
    input  logic clk,
    input  logic arm,
    input  logic disarm,
    output logic armed
);
```

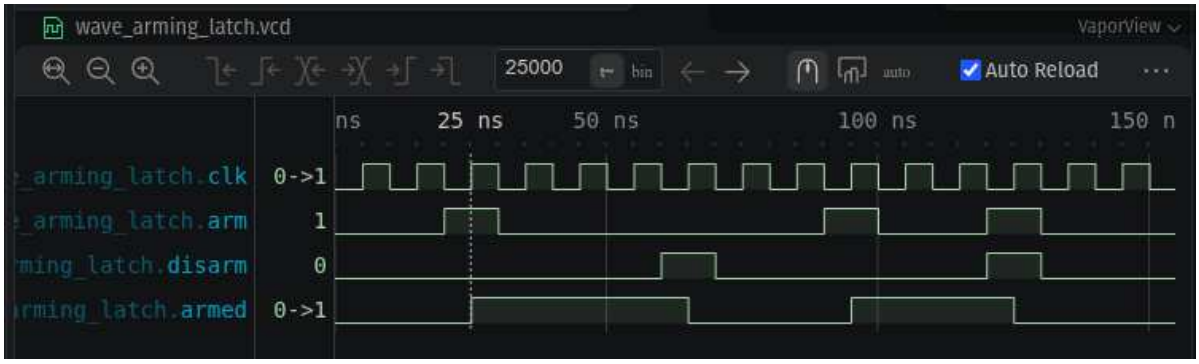
Initially `armed` is low. On each rising clock edge, if `disarm` is high, `armed` is cleared; if only `arm` is high, `armed` is set.

Rationale One instance of this module will track whether the watch is in edit mode.

Note This is simply a flip-flop with synchronous set and clear, where clear takes priority; the port names reflect intended use rather than implementation.



```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/arming_latch.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/arming_latch.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_arming_latch.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_arming_latch.vcd opened for output.
tb/wave_arming_latch.v:45: $finish called at 150000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_arming_latch.py
================================= test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_arming_latch.py .                                                    [100%]

================================== 1 passed in 0.36s ===================================
root@4034a70d84cc:/workspace#
```

State of `armed` depends on inputs `arm` and `disarm` with `disarm` having higher priority

7.7. Edit Mode Selector

When `count` is zero, `button` must be held long enough to assert `long_press`, which arms the latch and allows `count` to advance to one. Thereafter, each brief press of `button` advances `count` by one until it wraps back to zero and the cycle repeats.

Rationale One instance of this module will track which field of the watch is being edited, with `mode_enable` encoding the current state:

`mode_enable`	Meaning
`3'b000`	No field selected
`3'b001`	Edit seconds
`3'b010`	Edit minutes
`3'b100`	Edit hours

Implementation Notes Start with a block diagram of the circuit, then derive the combinational logic for `enable_counter`, `reset_counter`, and `disarm`.

Report 12

Include your block diagram and explain the combinational logic you used for `enable_counter`, `reset_counter`, and `disarm`.

Additional Oral Preparation 4

Be able to explain the output logic.

Read these notes only after completing your design.

- Rather than asserting `reset_counter` only on the disarming transition, it is safer and simpler to hold it high continuously whenever `armed` is low.
- As a further safety measure, `enable_counter` should also be low whenever `armed` is low. This is redundant if `reset_counter` is driven correctly, but it makes the design intent explicit.
- To derive `disarm`, avoid detecting the 2-to-0 transition in `count` directly. Instead, identify the condition under which `u_mod_3_counter` increments `count`, then AND that condition with `count == 2'd2`.
- If you incorporated `reset_counter` into that condition, convince yourself that it is redundant.

Note While a single bespoke FSM would work, good hardware design favours composing behaviour from smaller, reusable components. Doing so frees you from reasoning about the design as a monolithic state diagram; instead, you can think in terms of signal flow: when one signal asserts, a few clock cycles later another asserts, which in turn causes a third to respond. Each component encapsulates one behaviour, and wiring them together produces a design that is easier to reason about, test, and reuse.

rtl > edit_mode_selector.sv > ...

```sv
// Edit Mode Selector
// Long-pressing the button brings the watch into Edit Mode with 'seconds' as
// first editable mode. Short press once brings it to 'minutes' mode and once
// more into 'hours' mode. Subsequent short presses does nothing.
//
// Parameters:
//    HOLD_CYCLES:   - Number of clock cycles to check against to determine
//                     HIGH (default: 50_000_000)
//
// Ports:
//    clk:               - Clock input
//    button:            - Input button signal
//    [2:0] mode_enable: - Output signal as One-Hot Encoding States
//                         3'b000 = Not in Edit Mode
//                         3'b001 = Edit Seconds
//                         3'b010 = Edit Minutes
//                         3'b100 = Edit Hours
//
// Local Variables:
//    long_press:     - A HIGH pulse when the button is held for HOLD_CYCLES
//    press:          - A HIGH pulse when button is pressed, LOW when held
//    armed:          - HIGH when watch is in Edit State
//    disarm:         - HIGH when 'hours' mode and button pressed
//    reset_counter:  - Signal to reset Mod_N_Counter module
//    enable_counter: - Signal to enable Mod_N_Counter module
//    [1:0] count:    - Signal to increment `mode_enable` by shiftwise left

`timescale 1ns / 1ps

module edit_mode_selector #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input logic clk,
    input logic button,
    output logic [2:0] mode_enable
);

  logic long_press;
  button_hold_pulse #(
      .HOLD_CYCLES(HOLD_CYCLES)
  ) u_hold_pulse (
      .clk(clk),
      .button(button),
      .pulse(long_press)
  );

  logic press;
  rising_edge_detector u_detector (
      .clk(clk),
      .sig_in(button),
      .rise(press)
  );

  logic armed;
  logic disarm;
  arming_latch u_latch (
      .clk(clk),
      .arm(long_press),
      .disarm(disarm),
      .armed(armed)
  );

  logic reset_counter;
  logic enable_counter;
  logic [1:0] count;
  mod_n_counter #(
      .N(3),
      .WIDTH(2)
  ) u_mod_3_counter (
      .clk(clk),
      .rst(reset_counter),
      .enable(enable_counter),
      .count(count)
  );

  // Counter runs only while armed; resets when disarmed
  assign enable_counter = (armed && (long_press || press));
  assign reset_counter = disarm;

  // Disarm on the press that steps past the last mode
  assign disarm = (count == 2'b10) && (press);

  // Output logic
  assign mode_enable = armed ? (3'b001 << count) : 3'b000;

endmodule
```

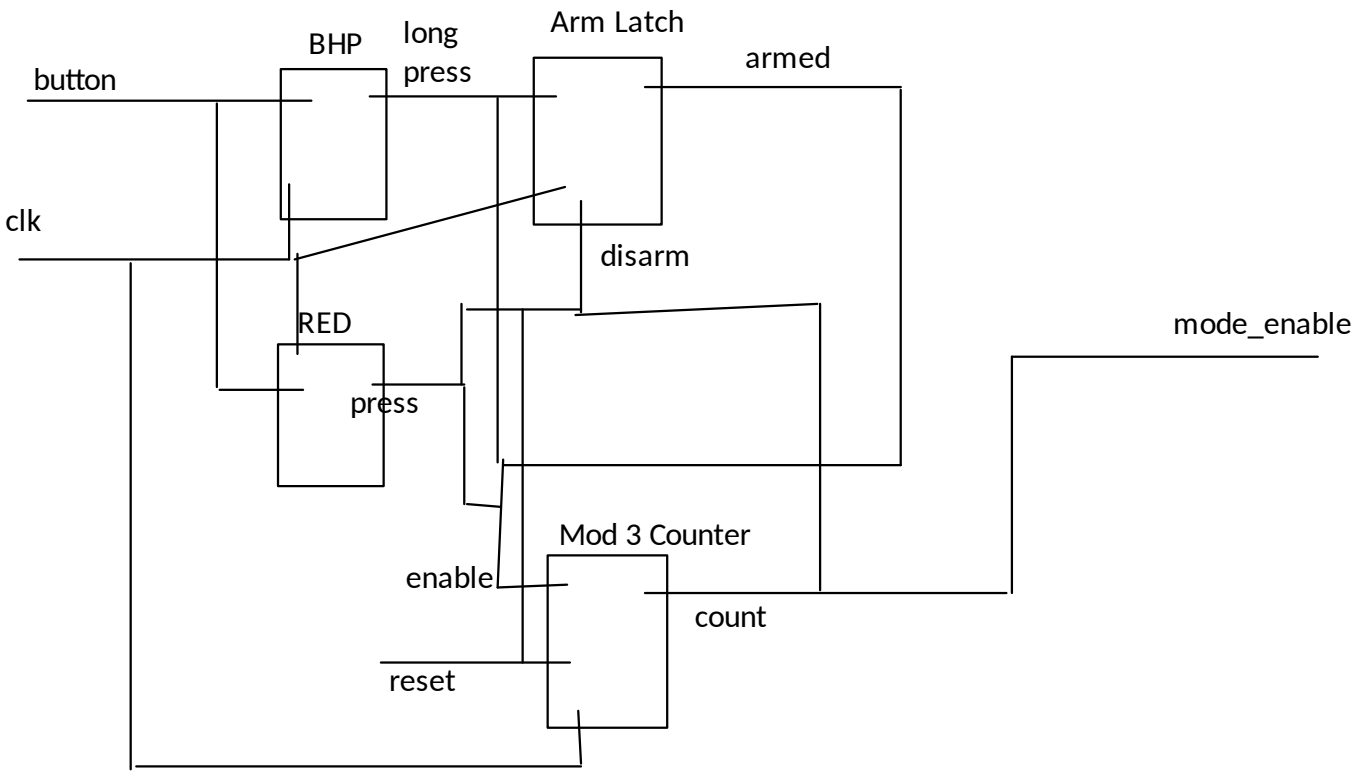
```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/edit_mode_selector.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/edit_mode_selector.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_edit_mode_selector.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_edit_mode_selector.vcd opened for output.
tb/wave_edit_mode_selector.v:66: $finish called at 580000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_edit_mode_selector.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_edit_mode_selector.py .                                       [100%]

============================== 1 passed in 0.35s ===============================
root@4034a70d84cc:/workspace#
```



7.8. Editable Counter

This module wraps an up/down counter with two modes of operation. In normal mode, `count` increments on each `tick` pulse. In edit mode, asserting `inc` increments `count` and asserting `dec` decrements it; if both are asserted simultaneously, `count` does not change.

Rationale Three instances of this module will hold the hours, minutes, and seconds of the watch.

Implementation Notes Assign `up` and `enable` using only the three event signals. If both `inc` and `dec` are high simultaneously, `count` should not change.

Report 13

Explain why `wire` was used instead of `logic` for the three event signals.

rtl > editable_counter.sv > ...

```
// Editable Counter
// This module will be used to represent the Hours, Minutes and Seconds of
// the Watch.
//
// Parameters:
//    N:                - Maximum value for the counter
//    WIDTH:            - Bitwise length of number N
//
// Ports:
//    clk:              - Clock input
//    tick:             - Input signal to pulse the `tick_event` when not in
//                        `edit_mode`
//    edit_mode:        - Input signal to represent State whether module is
//                      - in edit mode
//    inc:              - Input signal to pulse the `inc_event` in `edit_mode`
//    dec:              - Input signal to pulse the `dec_event` in `edit_mode`
//    [WIDTH-1:0] count:- Output signal to count a number between 0 to N-1
//
// Local Variables:
//    inc_event:  - Signal representing incrementing STATE when
//                  `inc` is HIGH AND `dec` is LOW while in `edit_mode`
//    dec_event:  - Signal representing decrementing STATE when
//                  `dec` is HIGH AND `inc` is LOW while in `edit_mode`
//    tick_event: - Signal representing NOT in `edit_mode` running according
//                  to `tick` pulses
//    up:         - Direction of counting: HIGH = upwards, LOW = downwards
//    enable:     - Enable the Up_Down_Counter when correct conditions are met

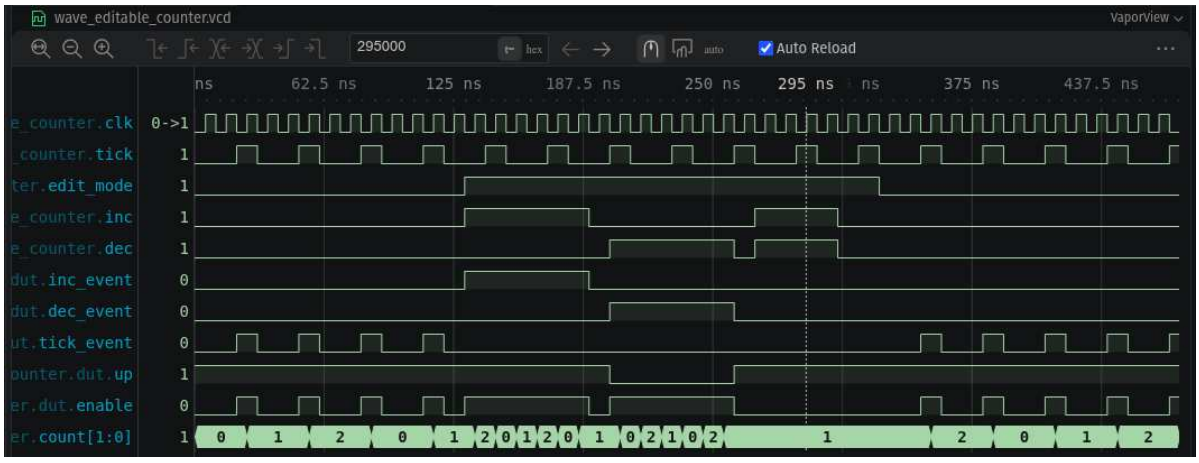
`timescale 1ns / 1ps

module editable_counter #(
    parameter int N = 60,
    parameter int WIDTH = 6
) (
    input logic clk,
    input logic tick,  // Count increments on tick when edit_mode is LOW
    input logic edit_mode,
    input logic inc,  // Count increments by one when edit_mode is HIGH
    input logic dec,  // Count decrements by one when edit_mode is HIGH
    output logic [WIDTH-1:0] count
);

  logic enable;
  logic up;
  up_down_counter #(
      .MAX  (N - 1),
      .WIDTH(WIDTH)
  ) u_counter (
      .clk(clk),
      .enable(enable),
```

```
43    logic enable;
44    logic up;
45    up_down_counter #(
46        .MAX  (N - 1),
47        .WIDTH(WIDTH)
48    ) u_counter (
49        .clk(clk),
50        .enable(enable),
51        .up(up),
52        .count(count)
53    );
54
55    wire inc_event = edit_mode && inc && !dec;
56    wire dec_event = edit_mode && dec && !inc;
57    wire tick_event = !edit_mode && tick;
58
59    assign up = inc_event || !dec_event || tick_event;
60    assign enable = tick_event || (dec_event || inc_event);
61
62  endmodule
63
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/editable_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/editable_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_editable_counter.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_editable_counter.vcd opened for output.
tb/wave_editable_counter.v:69: $finish called at 470000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_editable_counter.py
================================== test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 3 items

tests/test_editable_counter.py ...                                              [100%]

=================================== 3 passed in 0.82s ===================================
root@4034a70d84cc:/workspace#
```

7.9. Key Synchroniser

```
module key_synchroniser (
    input  logic       clk,
    input  logic [3:0] key_n,    // active-low, asynchronous
    output logic [3:0] key_sync  // active-high, synchronised
);
```

This module operates bitwise: each input bit is inverted then synchronised through a two-stage flip-flop cascade, with both stages initialised to zero.

Rationale The DE1-SoC pushbuttons are active-low, so they must be inverted before use. Digital inputs generally also require debouncing and synchronisation; since the DE1-SoC pushbuttons are debounced in hardware, we need only synchronise them.

Without synchronisation, a signal that transitions near a rising clock edge can leave a flip-flop in a metastable state (neither a clean 0 nor a clean 1). Passing the signal through two flip-flops in series mitigates this: even if the first flip-flop enters metastability, it has a full clock cycle to resolve to a clean 0 or 1 before the second samples it, which it will with vanishingly high probability. Crucially, it does not matter which way the first flip-flop resolves: because `key_n` is a single-bit signal, the only consequence is that the button transition may be detected one cycle late. This is far less serious than metastability in a multi-bit signal such as an FSM state register, where the resolved value matters and the system could jump to a completely wrong state.

Implementation Notes Pass each bit of `key_n` through an inverter and then two flip-flops in series to yield the corresponding bit of `key_sync`. Use bitwise operations to avoid repeating the logic four times, and initialise both flip-flops to zero explicitly.

Note Clear naming conventions reduce the risk of polarity errors: `key_n` is active-low, while `key_sync` is active-high, as indicated by the absence of the `_n` suffix.

rtl > key_synchroniser.sv > ...

```
// Key Synchroniser
// Digital Inputs require debouncing and sychronisation. DE1-SoC push buttons
// are debounced in hardware. Only sychronisation is required.
// DE1-SoC push buttons are Active-LOW, meaning LOW when pressed and HIGH
// when released. This module synchronises and inverts them to Active-HIGH
// for use in the rest of the circuit.
//
// Parameters:
//    None
//
// Ports:
//    clk:            - Clock input
//    [3:0] key_n:    - Input Signal from DE1-SoC KEY[3:0] ports, ACTIVE-LOW
//    [3:0] key_sync: - Output Signal for circuit, ACTIVE-HIGH
//
// Local Variables:
//    key_mid: - Intermediary Net to help mitigate metastability
//             initialised with 4'b1111 to initialise `key_sync` with 4'b0000

`timescale 1ns / 1ps

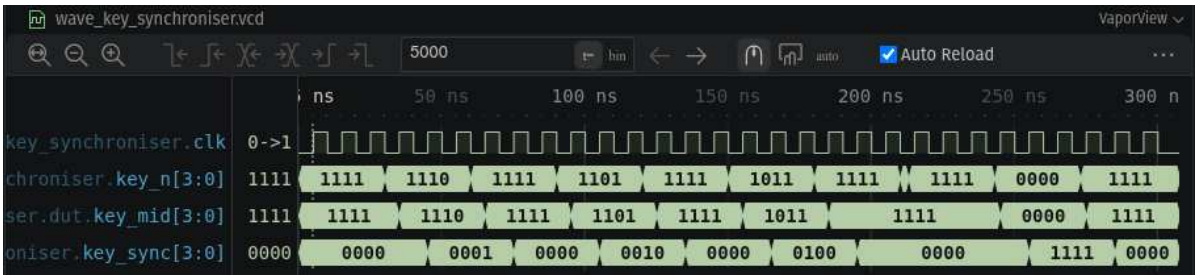
module key_synchroniser (
    input  logic       clk,
    input  logic [3:0] key_n,              // active-low, asynchronous
    output logic [3:0] key_sync = 4'b0000  // active-high, synchronised
);

  logic [3:0] key_mid = 4'b1111;

  always_ff @(posedge clk) begin
    key_mid  <= key_n;
    key_sync <= ~key_mid;
  end

endmodule
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/key_synchroniser.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/key_synchroniser.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_key_synchroniser.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_key_synchroniser.vcd opened for output.
tb/wave_key_synchroniser.v:61: $finish called at 303000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_key_synchroniser.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_key_synchroniser.py .                                         [100%]

============================== 1 passed in 0.43s ===============================
root@4034a70d84cc:/workspace#
```

7.10. Decimal Display Driver

Copy the following template from `templates/decimal_display_driver.sv` to your `rtl/` folder and edit it.

This module accepts three 7-bit decimal values (`value0`–`value2`, range 0–99) and three blanking controls (`blank0`–`blank2`). Each value is displayed on a pair of seven-segment displays: `value0` drives `HEX1` (tens) and `HEX0` (ones), and so on up to `value2` on `HEX5` and `HEX4`. When a blanking control is asserted, both digits of the corresponding value are blanked. The six outputs are active-low.

Rationale This module allows the rest of the design to treat the display as three independent decimal values, hiding the binary-to-BCD conversion, seven-segment decoding, digit blanking, and active-low polarity.

Implementation Notes A `generate` loop is a compile-time construct: the synthesiser unrolls it and produces one copy of the hardware per iteration, just as if you had written the block out three times by hand. Inside the loop body you can declare local wires and instantiate modules, using `i` as an array index to connect each copy to the correct ports. The `begin`–`end` block must be named so that local wires in each iteration occupy distinct namespaces and do not clash with those in other iterations.

The template has already packed the scalar ports into arrays and set up the `generate` loop; your task is simply to fill in the body of the named block. Declare any necessary local wires, then instantiate `binary_to_bcd` and `seven_segment`, using `i` to index the correct array elements.

Notes The automated tests do not cover every possible input. Since this module composes `binary_to_bcd` and `seven_segment`, both of which are tested separately, only a few tests are included to catch basic wiring errors.

Although 15 lines are needed to pack the ports into arrays, each line is straightforward to verify by eye. Repeating the circuit three times by hand would be harder to check carefully, and reader fatigue would make errors easier to miss.

rtl > decimal_display_driver.sv > ...

```sv
// ------------------------------------------------------------------------
// decimal_display_driver
// ------------------------------------------------------------------------
// Board-specific display driver for the DE1-SoC HEX displays.
//
// This module presents three decimal values (0-99) on the six
// seven-segment HEX displays (HEX5..HEX0).
// Each value uses two digits.
//
// The module accepts numeric values and blanking controls.
// Internally, it handles:
//   - binary-to-BCD conversion
//   - seven-segment decoding
//   - digit blanking
//   - active-low segment polarity
// ------------------------------------------------------------------------
`timescale 1ns / 1ps

module decimal_display_driver (
    // Decimal values to be displayed (range: 0-99)
    // value0 -> HEX1 (tens) and HEX0 (ones)
    // value1 -> HEX3 (tens) and HEX2 (ones)
    // value2 -> HEX5 (tens) and HEX4 (ones)
    input logic [6:0] value0,
    input logic [6:0] value1,
    input logic [6:0] value2,

    // Blanking controls for each decimal value.
    // When asserted, both digits of the value are blanked.
    input logic blank0,
    input logic blank1,
    input logic blank2,

    // DE1-SoC seven-segment display outputs.
    // Active-low segments: [g,f,e,d,c,b,a]
    output logic [6:0] HEX0,
    output logic [6:0] HEX1,
    output logic [6:0] HEX2,
    output logic [6:0] HEX3,
    output logic [6:0] HEX4,
    output logic [6:0] HEX5
);

  // Pack scalar ports into arrays for use in the generate loop.
  // value_a[i] drives HEX(2i+1) (tens) and HEX(2i) (ones).
  logic [6:0] value_a[3];
  assign value_a[0] = value0;
  assign value_a[1] = value1;
  assign value_a[2] = value2;

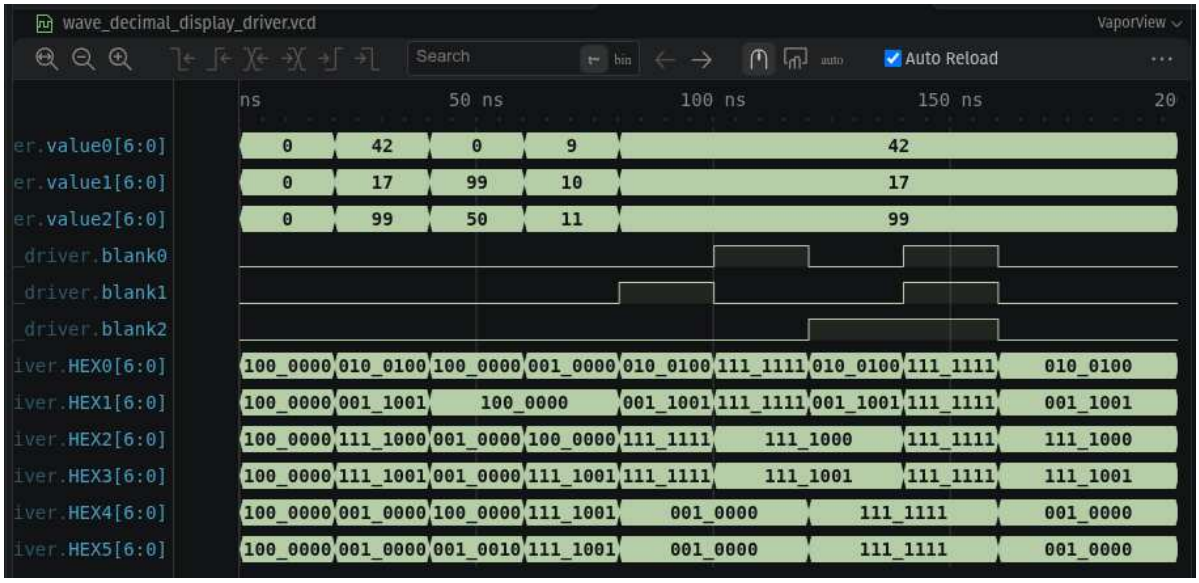
  logic blank_a[3];
  assign blank_a[0] = blank0;
  assign blank_a[1] = blank1;
  assign blank_a[2] = blank2;

  logic [6:0] hex_a[6];
  assign HEX0 = hex_a[0];
  assign HEX1 = hex_a[1];
  assign HEX2 = hex_a[2];
  assign HEX3 = hex_a[3];
  assign HEX4 = hex_a[4];
  assign HEX5 = hex_a[5];

  genvar i;
  generate
    for (i = 0; i < 3; i = i + 1) begin : g_digit_pair
      // Fill this in
      logic [3:0] tens, ones;
      binary_to_bcd u_bcd (
          .bin (value_a[i]),
          .tens(tens),
          .ones(ones)
      );
      seven_segment u_seg_tens (
          .digit(tens),
          .blank(blank_a[i]),
          .segments(hex_a[2*i+1])
      );
      seven_segment u_seg_ones (
          .digit(ones),
          .blank(blank_a[i]),
          .segments(hex_a[2*i])
      );
    end
  endgenerate

endmodule
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/decimal_display_driver.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/decimal_display_driver.sv
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_decimal_display_driver.v
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_decimal_display_driver.vcd opened for output.
tb/wave_decimal_display_driver.v:72: $finish called at 180000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_decimal_display_driver.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_decimal_display_driver.py .                                   [100%]

============================== 1 passed in 0.44s ===============================
root@4034a70d84cc:/workspace#
```

7.11. Board Wrapper

Copy `templates/user_top.sv` and `templates/top_de1_soc.sv` to your `rtl/` directory. Their contents are provided below for reference.

There are no waveforms to generate for this task; go straight to running the automated tests to verify you have copied the files correctly. There is no separate test for `user_top`: it contains the code that enables `top_de1_soc` to be tested, so it is covered by that test.

Rationale Two engineering principles worth following are *separation of concerns* and *don't repeat yourself*. Applied to our watch design, they suggest creating a dedicated *user* top-level module whose ports are meaningful to the design rather than to any particular board.

A second module, the *board wrapper*, then serves purely as the synthesis top-level that Quartus targets. It maps the board's physical pins to the user top-level's ports and sets board-specific parameters such as the clock frequency. Whenever the design is ported to a different board, only the board wrapper needs to change.

Additional Oral Preparation 5

Be able to explain the purpose of a board wrapper and how it is used.

Note Do not modify these files directly; the automated tests depend on them remaining unaltered. To adapt them for a different design, copy both files, rename the copies, and update the module names.

Board Wrapper focusses only on the physical pins and connections of the board

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/top_de1_soc.sv rtl/user_top.sv
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/top_de1_soc.sv rtl/user_top.sv
root@4034a70d84cc:/workspace#
```

```
root@4034a70d84cc:/workspace# pytest tests/test_top_de1_soc.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_top_de1_soc.py .                                              [100%]

============================== 1 passed in 0.39s ===============================
root@4034a70d84cc:/workspace#
```

7.12. Watch Specifications and Feature Layering

With all the building blocks in place, we can now implement the user top-level module for the settable watch. The full specification is below.

Normal Mode

- Time increments once per second.
- Holding `KEY[3]` for ≥ 1 s enters edit mode with the seconds field selected.

Edit Mode The selected digits flash at 2 Hz with an 80 % duty cycle.

`KEY[3]` Advance the selection: seconds → minutes → hours → exit.

`KEY[1]` Increment the selected field.

`KEY[0]` Decrement the selected field.

Hold `KEY[1]` or `KEY[0]` for >0.5 s Repeat at 10 Hz.

Fields below the selected field continue to tick normally. The selected field and all fields above it do not advance while in edit mode.

Before reading on, take a few minutes to draw a block diagram of this design.

You may have found the design difficult: it does not decompose naturally into submodules, because the interdependencies make it hard to identify clean boundaries. This is typical of top-level modules with many interrelated features.

Implementing everything at once is risky: a large body of code accumulates before any of it can be tested, and the more untested code there is, the harder bugs are to locate.

The solution is *feature layering*: begin with the most basic functionality, verify it works, then add features one at a time, testing and debugging each before moving on.

In the following sections, you will implement `user_top_watch` with a version suffix (`_v1`, `_v2`, and so forth); the corresponding test bench files follow the same convention. The automated tests are cumulative: each version must pass all prior tests as well as its own.

Linting and Simulation Commands Do not use wildcards when linting or simulating (see Appendix A.6). **Use the flags below**, which instruct the tools to locate modules in `rtl/`.

```
verilator -Wall --lint-only -y rtl rtl/user_top_watch_v1.sv
iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_user_top_watch_v1.v
```

7.13. Timekeeping (Watch V1)

The base layer is a reimplementation of the time display from Section 6, using three `editable_counter` instances in place of `hms_counter`.

Feature layering requires a different approach from writing independent modules. This section walks through the process in detail; **you are then expected to apply these practices independently.**

In the VS Code Explorer, right-click `rtl/user_top.sv` and select Copy, then right-click the `rtl` folder and select Paste. Rename the resulting file to `user_top_watch_v1.sv`, open it, and change the module name from `user_top` to `user_top_watch_v1`. Remove the test code inside, then add a comment just after the module declaration labelling the block as the basic time display.

```
// ------------------
// Core Functionality
// ------------------
```

We start from the outputs and work backwards: the hours, minutes, and seconds outputs are driven by `editable_counter` instances, so we add the seconds instance first, declaring new `logic` signals for each connection with no driver yet (there is an intentional error below):

```
// Seconds
logic seconds_tick;
logic seconds_edit;
logic seconds_inc;
logic seconds_dec;
editable_counter #(
    .N(60),
    .WIDTH(6)
) u_seconds (
    .clk(clk),
    .tick(seconds_tick),
    .edit_mode(seconds_edit),
    .inc(seconds_inc),
    .dec(seconds_dec),
    .count(seconds_disp)
);
```

We need the same for minutes and hours. A `generate` block is not appropriate here, as naming each field explicitly keeps the code clear. Instead, use VS Code's multi-cursor: copy the seconds block, place the cursor on the first occurrence of `seconds` in the copy, press Ctrl+D to select each subsequent occurrence, type the replacement, press Escape to exit multi-cursor mode, then repeat for hours.

```
// ------------------------------------------------------------------
// WARNING: This file is used by the automated test suite. Do not
// modify it.
//
// This file also serves as a template for your own designs. To use
// it:
//   1. Copy the entire contents into a new file with a descriptive
//      name.
//   2. Delete the test logic below and replace it with your own
//      code.
//   3. In top_de1_soc, change the module name from user_top to your
//      new module name.
//
//   The board wrapper sets CYCLES_PER_SECOND; use this parameter in
//   your design wherever timing is needed.
// ------------------------------------------------------------------
`timescale 1ns / 1ps

module user_top_watch_v1 #(
    /* verilator lint_off UNUSEDPARAM */
    parameter int CYCLES_PER_SECOND = 50_000_000
    /* verilator lint_on UNUSEDPARAM */
) (
    input logic clk,
    /* verilator lint_off UNUSED */
    input logic [3:0] button,
    input logic [9:0] sw,
    /* verilator lint_on UNUSED */
    output logic [9:0] led,
    output logic [6:0] hours_disp,
    output logic [6:0] minutes_disp,
    output logic [6:0] seconds_disp,
    output logic blank_hours,
    output logic blank_minutes,
    output logic blank_seconds
);

  // ------------------
  // Core Functionality
  // ------------------

  // Seconds
  logic seconds_tick;
  logic seconds_edit;
  logic seconds_inc;
  logic seconds_dec;
  logic [5:0] seconds;
  editable_counter #(
      .N(60),
      .WIDTH(6)
  ) u_seconds (
      .clk(clk),
      .tick(seconds_tick),
      .edit_mode(seconds_edit),
      .inc(seconds_inc),
      .dec(seconds_dec),
      .count(seconds)
  );

  // Minutes
  logic minutes_tick;
  logic minutes_edit;
  logic minutes_inc;
  logic minutes_dec;
  logic [5:0] minutes;
  editable_counter #(
      .N(60),
      .WIDTH(6)
  ) u_minutes (
      .clk(clk),
      .tick(minutes_tick),
      .edit_mode(minutes_edit),
      .inc(minutes_inc),
      .dec(minutes_dec),
      .count(minutes)
  );

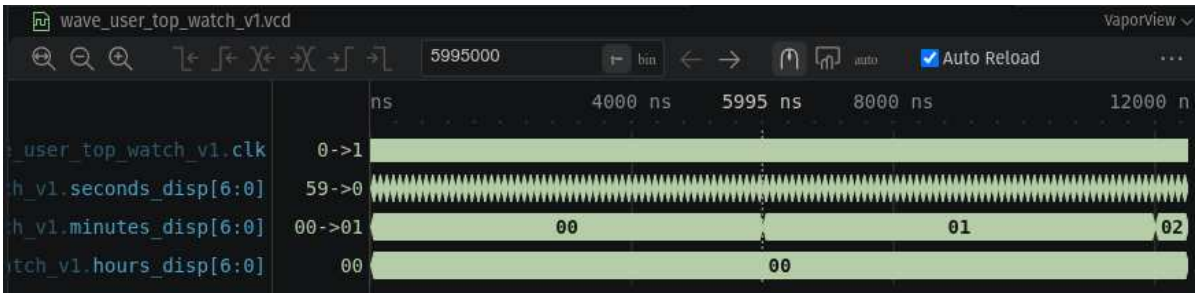
  // Hours
  logic hours_tick;
  logic hours_edit;
  logic hours_inc;
  logic hours_dec;
  logic [4:0] hours;
  editable_counter #(
      .N(24),
      .WIDTH(5)
  ) u_hours (
      .clk(clk),
      .tick(hours_tick),
      .edit_mode(hours_edit),
      .inc(hours_inc),
      .dec(hours_dec),
      .count(hours)
  );

  // Derive 1 Hz tick from system clock
  restartable_rate_generator #(
      .CYCLE_COUNT(CYCLES_PER_SECOND)
  ) u_divider_1_Hz (
      .clk (clk),
      .run (1'b1),
      .tick(seconds_tick)
  );

```

```
104
105   // Derive ticks for Minutes and Hours
106   assign minutes_tick = (seconds_disp == 7'd59) && seconds_tick;
107   assign hours_tick = (minutes_disp == 7'd59) && (seconds_disp == 7'd59) && seconds_tick;
108
109   // Zero-extend counter values to display outputs
110   assign hours_disp = {2'b0, hours};
111   assign minutes_disp = {1'b0, minutes};
112   assign seconds_disp = {1'b0, seconds};
113
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/user_top_watch_v1.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_user_top_watch_v1.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_user_top_watch_v1.vcd opened for output.
tb/wave_user_top_watch_v1.v:45: $finish called at 12500000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_user_top_watch_v1.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_user_top_watch_v1.py .                                        [100%]

============================== 1 passed in 0.81s ===============================
root@4034a70d84cc:/workspace#
```

7.14. Mode Selection (Watch V2)

Start by duplicating `rtl/user_top_watch_v1.sv`, renaming it `user_top_watch_v2.sv`, and changing the module name from `user_top_watch_v1` to `user_top_watch_v2`.

The standard development cycle (Section 5) applies; the test bench and automated tests for this version carry the suffix `_v2`.

At the end of `user_top_watch_v2`, add a comment marking the start of the mode selection logic, then add an instance of `edit_mode_selector` using the same approach as before.

```
// --------------
// Mode Selection
// --------------

logic [2:0] mode_enable;
edit_mode_selector #(
    .HOLD_CYCLES(...) // Fill in, based on CYCLES_PER_SECOND
) u_mode_selector (
    // Fill in
);
```

Also instantiate a `pwm_generator` to produce the waveform that drives the display flash rate and duty cycle, connecting its output to a new `logic` signal `pwm_out`.

Finally, the `assign` statements that tie `blank_hours`, `blank_minutes`, and `blank_seconds` low should be replaced with appropriate expressions at the end of the module.

Note The `verilator` lint-off directive still suppresses warnings for `button` as a whole, even though we now use `button[3]`. Since the directive is just a comment, it is invisible to all other tools and has no effect on the design. Removing it would cause `verilator` to warn that the remaining buttons are unused, and adding dummy logic to silence that warning would be worse than simply acknowledging the unused inputs. We therefore leave the directive in place.

```
// --------------
// Mode Selection
// --------------

logic [2:0] mode_enable;
edit_mode_selector #(
    .HOLD_CYCLES(CYCLES_PER_SECOND)
) u_mode_selector (
    .clk(clk),
    .button(button[3]),
    .mode_enable(mode_enable)
);
assign seconds_edit = (mode_enable == 3'b001);
assign minutes_edit = (mode_enable == 3'b010);
assign hours_edit   = (mode_enable == 3'b100);

// Flashing Edit Mode at 2Hz with 10% Duty Cycle
logic pwm_out;
logic rst;
assign rst = (mode_enable == 3'b000);
pwm_generator #(
    .PERIOD_CYCLES(CYCLES_PER_SECOND / 2),
    .DUTY_CYCLES  (CYCLES_PER_SECOND / 10)
) u_generator (
    .clk(clk),
    .rst(rst),
    .pwm_out(pwm_out)
);
assign blank_hours = pwm_out && (mode_enable == 3'b100);
assign blank_minutes = pwm_out && (mode_enable == 3'b010);
assign blank_seconds = pwm_out && (mode_enable == 3'b001);

```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/user_top_watch_v2.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_user_top_watch_v2.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_user_top_watch_v2.vcd opened for output.
tb/wave_user_top_watch_v2.v:81: $finish called at 5100000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_user_top_watch_v2.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 2 items

tests/test_user_top_watch_v2.py ..                                       [100%]

============================== 2 passed in 2.06s ===============================
root@4034a70d84cc:/workspace#
```

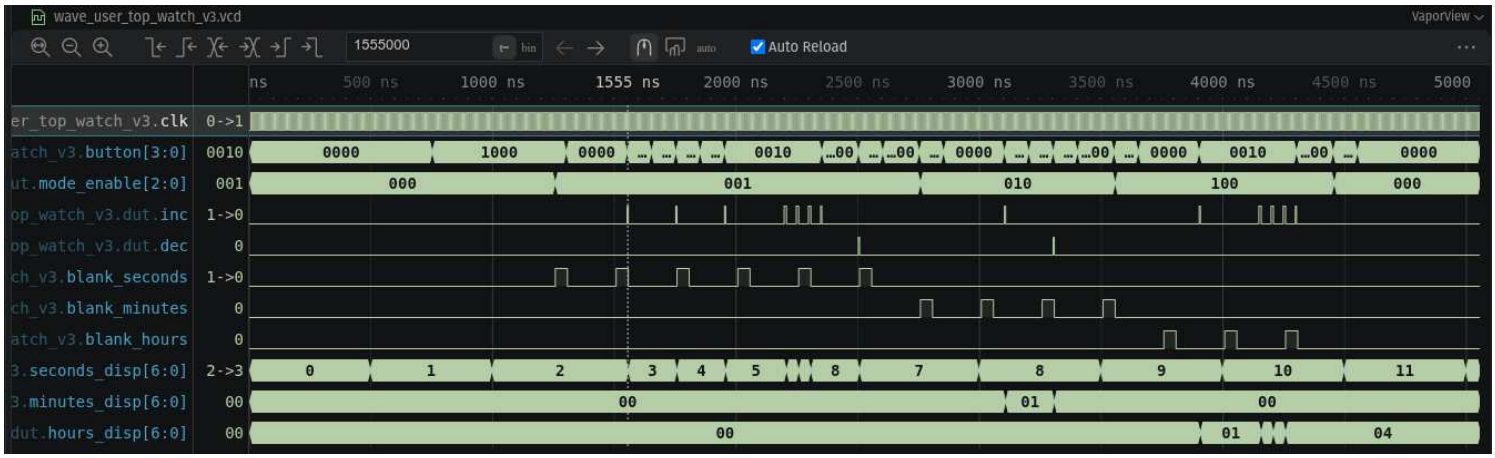
7.15. Settable Watch (V3)

Create `user_top_watch_v3` and add a comment marking the start of the edit logic. From here you are on your own: implement the full specification (Section 7.12), following the development cycle of Section 5 and the feature-layering approach of Section 7.13.

```
104
105 // Derive ticks for Minutes and Hours, runs only in normal mode
106 assign minutes_tick = (seconds_disp == 7'd59) && seconds_tick && (mode_enable == 3'b000);
107 assign hours_tick   = (minutes_disp == 7'd59) && (seconds_disp == 7'd59) && seconds_tick && (mode_enable == 3'b000);
108
```

```
145
146 // --------------
147 // Edit Logic
148 // --------------
149
150 // Increment pulses on button[1] press
151 logic inc;
152 button_auto_repeat #(
153     .HOLD_CYCLES  (CYCLES_PER_SECOND / 2),
154     .REPEAT_CYCLES(CYCLES_PER_SECOND / 10)
155 ) u_inc (
156     .clk(clk),
157     .button(button[1]),
158     .pulse(inc)
159 );
160
161 // Increment pulses on button[0] press
162 logic dec;
163 button_auto_repeat #(
164     .HOLD_CYCLES  (CYCLES_PER_SECOND / 2),
165     .REPEAT_CYCLES(CYCLES_PER_SECOND / 10)
166 ) u_dec (
167     .clk(clk),
168     .button(button[0]),
169     .pulse(dec)
170 );
171
172 // Increment or Decrement only when their respective modes are enabled
173 assign seconds_inc = inc && (mode_enable == 3'b001);
174 assign seconds_dec = dec && (mode_enable == 3'b001);
175 assign minutes_inc = inc && (mode_enable == 3'b010);
176 assign minutes_dec = dec && (mode_enable == 3'b010);
177 assign hours_inc = inc && (mode_enable == 3'b100);
178 assign hours_dec = dec && (mode_enable == 3'b100);
179
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/user_top_watch_v3.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_user_top_watch_v3.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_user_top_watch_v3.vcd opened for output.
tb/wave_user_top_watch_v3.v:131: $finish called at 5050000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_user_top_watch_v3.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 4 items

tests/test_user_top_watch_v3.py ....                                     [100%]

============================== 4 passed in 4.01s ===============================
root@4034a70d84cc:/workspace#
```

7.16. Accurate Time Setting (V4)

Just when you think you are finished, your friend complains that they cannot set the watch precisely: they have no control over when the seconds counter rolls over.

The desired feature is to realign the seconds tick when the user moves from editing seconds to editing minutes, so that the next tick occurs exactly one second later. This is achievable because the seconds counter is driven by a *restartable* rate generator.

Implement this feature in `user_top_watch_v4.sv`.

Report 15

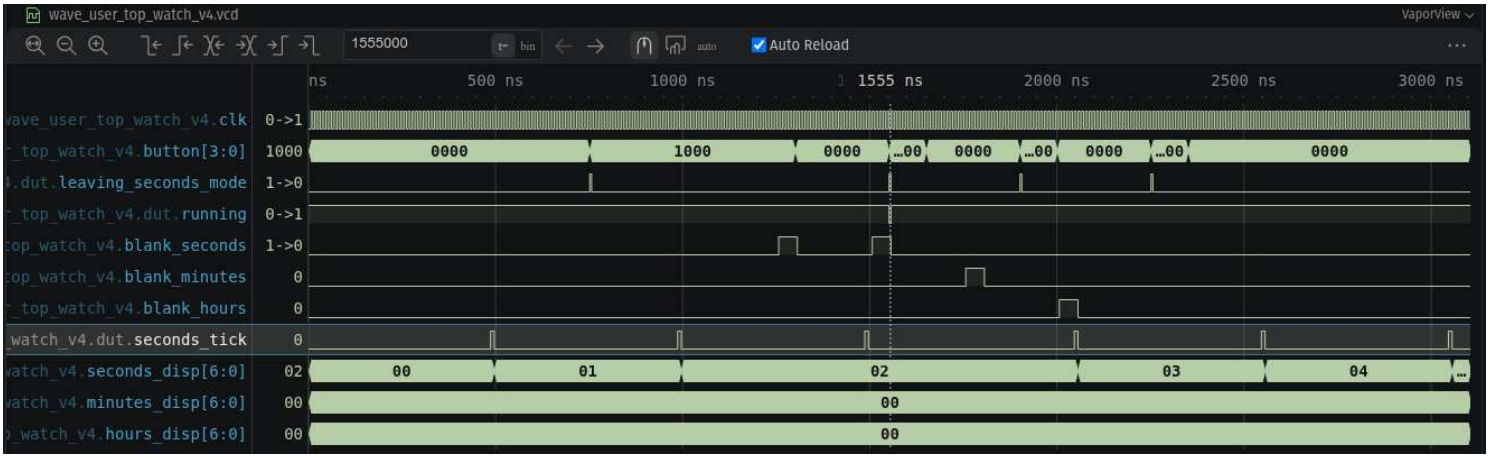
Think carefully about which traces to include in your waveform report. All internal signals are visible, but there are many; include only those relevant to the feature being demonstrated.

Report 16

Does your design cause multiple resets? Why is this not a problem?

```
95
96  // Derive 1 Hz tick from system clock
97  // Reset the Seconds tick when leaving the edit-seconds-mode
98  logic leaving_seconds_mode;
99  logic running;
100 rising_edge_detector u_reset_seconds (
101     .clk(clk),
102     .sig_in(button[3]),
103     .rise(leaving_seconds_mode)
104 );
105 always_comb begin
106   if (mode_enable == 3'b001) begin
107     running = 1'b1 && !leaving_seconds_mode;
108   end else begin
109     running = 1'b1;
110   end
111 end
112
113 restartable_rate_generator #(
114     .CYCLE_COUNT(CYCLES_PER_SECOND)
115 ) u_divider_1_Hz (
116     .clk (clk),
117     .run (running),
118     .tick(seconds_tick)
119 );
120
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/user_top_watch_v4.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_user_top_watch_v4.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_user_top_watch_v4.vcd opened for output.
tb/wave_user_top_watch_v4.v:75: $finish called at 3100000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_user_top_watch_v4.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 5 items

tests/test_user_top_watch_v4.py .....                                    [100%]

============================== 5 passed in 4.45s ===============================
root@4034a70d84cc:/workspace#
```

Report 17

Reflect on your experience in previous lab classes. Were you typically able to get a design working on the first compile and board test? Which development workflow do you prefer: the VS Code simulation environment, or writing code directly in Quartus?

Report 18

Assume the 50 MHz clock on the DE1-SoC board is accurate to within 50 ppm. How many seconds might your watch drift over a 24-hour period?

VS Code preferred, as it allows quick test turnarounds

50 ppm times 86400 seconds = 4_320_000ppm seconds per day
= 4.32 seconds per day

Digital Systems Project Assignment 2
Digital Watch: Stopwatch and Timer

This is the second of two assignments. Building on your settable watch from Assignment 1, you will add a stopwatch, a timer, and a display brightness control. You will also be introduced to formal verification and see first-hand how a synthesiser translates your code into actual hardware. Greater emphasis is placed on explaining the design decisions along the way.

1. Policies and Assessment

These are the same as for Assignment 1.

Back up your work regularly.

2. Automated Tests

The automated test suite, accessed by running `pytest` in the terminal, includes per-module functional tests, the formal verification tests introduced later in this assignment, and checks that no module has linting or compilation warnings or errors.

Run `pytest` now to confirm that all your files from Assignment 1 are in order before proceeding.

For the modules written as part of this assignment, some will have formal verification tests only, some will have cocotb tests only, and some will have both. Formal verification is not a panacea and is generally not the right tool to use for testing responses to sequences of inputs.

```
(venv) root@4034a70d84cc:/workspace# pytest
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 112 items

tests/test_arming_latch.py .                                             [  0%]
tests/test_binary_to_bcd.py .                                            [  1%]
tests/test_button_auto_repeat.py .                                       [  2%]
tests/test_button_hold_detect.py .                                       [  3%]
tests/test_button_hold_pulse.py .                                        [  4%]
tests/test_decimal_display_driver.py .                                   [  5%]
tests/test_edit_mode_selector.py .                                       [  6%]
tests/test_editable_counter.py ...                                       [  8%]
tests/test_formal.py ssssssss                                            [ 16%]
tests/test_hms_counter.py ...                                            [ 18%]
tests/test_key_synchroniser.py .                                         [ 19%]
tests/test_lint.py ..........................................            [ 60%]
tests/test_mod_n_counter.py ..                                           [ 62%]
tests/test_pwm_generator.py .......                                      [ 68%]
tests/test_restartable_rate_generator.py ......                          [ 74%]
tests/test_rising_edge_detector.py .                                     [ 75%]
tests/test_seven_segment.py ..                                           [ 76%]
tests/test_stopwatch_control.py s                                        [ 77%]
tests/test_top_de1_soc.py .                                              [ 78%]
tests/test_top_time_display_v1.py .                                      [ 79%]
tests/test_up_down_counter.py ...                                        [ 82%]
tests/test_user_top_brightness_timepiece.py s                            [ 83%]
tests/test_user_top_brightness_wrapper.py ssss                           [ 86%]
tests/test_user_top_stopwatch_v1.py s                                    [ 87%]
tests/test_user_top_timepiece_v1.py s                                    [ 88%]
tests/test_user_top_timer_v1.py s                                        [ 89%]
tests/test_user_top_watch_v1.py .                                        [ 90%]
tests/test_user_top_watch_v2.py ..                                       [ 91%]
tests/test_user_top_watch_v3.py ....                                     [ 95%]
tests/test_user_top_watch_v4.py .....                                    [100%]

======================== 95 passed, 17 skipped in 22.01s ========================
(venv) root@4034a70d84cc:/workspace#
```

3. Formal Verification

Formal verification is introduced here using an up-down counter with reset as the running example.

3.1. Up-Down Counter with Reset

The up-down counter from Assignment 1 lacked reset capability.

> **Report 1**
>
> When a module needs to be extended mid-design, several approaches are possible: modify the original in place; copy it, modify the copy, and leave the original untouched; or copy it, modify the copy, and rewrite the original as a thin wrapper around the new module. Can you think of any other options? For this specific case, which approach would you take, and how would you name the resulting module or modules? Justify your choices and discuss their trade-offs relative to the alternatives.

SystemVerilog's support for default port values makes modifying the original module particularly attractive here. Adding `input logic rst = 1'b0` to the port list means existing instantiations that do not connect `rst` continue to work unchanged, while new instantiations can drive it explicitly. Any regression introduced during the modification would be caught by the existing test suite; the suite should then be extended to cover the new reset behaviour.

In practice, however, Verilator and Icarus Verilog both lack support for default port values; we therefore leave the existing module untouched and create a new one from a copy.

Task Implement the following module in `rtl/up_down_counter_rst.sv`, following all conventions from Assignment 1, including **elimination of all Verilator linting warnings.**

rtl > up_down_counter_rst.sv > ...

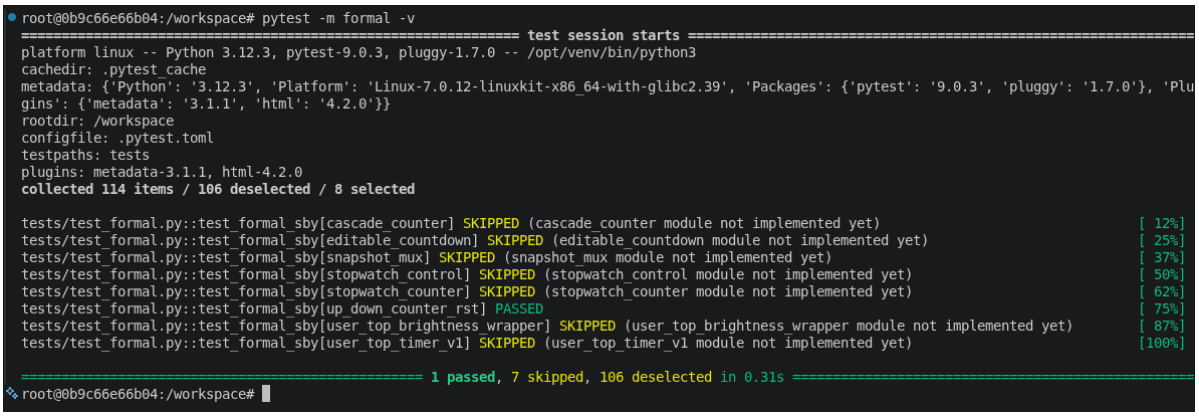
```
1  // Up_Down_Counter_Rst
2  // To act as a standard Counter, whose modulus value is determined by the
3  // MAX parameter argument that was passed into it. The counter will count up
4  // when the up input is high, and count down when the up input is low. The
5  // counter will wrap back to 0 when the maximum value is reached, and wrap
6  // back to the maximum value when the minimum value is reached for up and
7  // down counting, respectively.
8  // The input `rst` signal allows asynchronous reseting of the `count` output
9  //
10 // Parameters:
11 //   MAX    - Maximum count value (default: 2).
12 //   WIDTH  - Width of the count output (default: 2).
13 //
14 // Ports:
15 //   clk    - Clock input.
16 //   rst    - When high, counter becomes and remains 'd0 on every rising edge
17 //   enable - When high, the counter is enabled.
18 //   up     - When high, the counter counts up; when low, counts down.
19 //   count  - Output Signal, returns [WIDTH-1:0] number between 0 and N-1
20 //
21 // Local Variables:
22 //   [WIDTH-1:0] Max:        - Casts `MAX` parameter to local `Max` parameter
23 //   [WIDTH-1:0] next_count: - Intermediary variable to hold value of `count`'s
24 //                            next number
25
26 `timescale 1ns / 1ps
27
28 module up_down_counter_rst #(
29     parameter int MAX   = 2,
30     parameter int WIDTH = 2
31 ) (
32     input logic clk,
33     input logic rst,
34     input logic enable,
35     input logic up,
36     output logic [WIDTH-1:0] count = WIDTH'(0)
37 );
38
39   // Declare local parameters and signals
40   // Cast input `MAX` parameter to local `Max` parameter
41   localparam logic [WIDTH-1:0] Max = WIDTH'(MAX);
42   logic [WIDTH-1:0] next_count;
43
44   // State, and Output Logic
45   always_ff @(posedge clk) begin
46     if (rst) count <= WIDTH'(0);
47     else if (enable) count <= next_count;
48   end
49
50   // Next state logic
51   always_comb begin
52     if (up) begin  // up=1 -> Count up
53       if (count < Max) next_count = count + WIDTH'(1);
54       else next_count = WIDTH'(0);  // Wrap to 0 when Max
55     end else begin  // else up=0 -> Count down
56       if (count > WIDTH'(0)) next_count = count - WIDTH'(1);
57       else next_count = Max;  // Wrap to Max when 0
58     end
59   end
60
61 endmodule
62
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/up_down_counter_rst.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/up_down_counter_rst.sv
(venv) root@4034a70d84cc:/workspace#
```

```
root@0b9c66e66b04:/workspace# sby -f sby/up_down_counter_rst.sby
SBY  5:50:35 [sby/up_down_counter_rst_max_4] Removing directory '/workspace/sby/up_down_counter_rst_max_4'.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] Copy '/workspace/rtl/up_down_counter_rst.sv' to '/workspace/sby/up_down_counter_rst_max_4/src/up_down_counter_rst.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] Copy '/workspace/sby/up_down_counter_rst_assert.sv' to '/workspace/sby/up_down_counter_rst_max_4/src/up_down_counter_rst_assert.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0: smtbmc z3
SBY  5:50:35 [sby/up_down_counter_rst_max_15] Removing directory '/workspace/sby/up_down_counter_rst_max_15'.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] Copy '/workspace/rtl/up_down_counter_rst.sv' to '/workspace/sby/up_down_counter_rst_max_15/src/up_down_counter_rst.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] Copy '/workspace/sby/up_down_counter_rst_assert.sv' to '/workspace/sby/up_down_counter_rst_max_15/src/up_down_counter_rst_assert.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0: smtbmc z3
SBY  5:50:35 [sby/up_down_counter_rst_max_60] Removing directory '/workspace/sby/up_down_counter_rst_max_60'.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] Copy '/workspace/rtl/up_down_counter_rst.sv' to '/workspace/sby/up_down_counter_rst_max_60/src/up_down_counter_rst.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] Copy '/workspace/sby/up_down_counter_rst_assert.sv' to '/workspace/sby/up_down_counter_rst_max_60/src/up_down_counter_rst_assert.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0: smtbmc z3
SBY  5:50:35 [sby/up_down_counter_rst_max_4] base: starting process "cd sby/up_down_counter_rst_max_4/src; yosys -ql ../model/design.log ../model/design.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_15] base: starting process "cd sby/up_down_counter_rst_max_15/src; yosys -ql ../model/design.log ../model/design.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] base: starting process "cd sby/up_down_counter_rst_max_60/src; yosys -ql ../model/design.log ../model/design.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] base: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] base: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] prep: starting process "cd sby/up_down_counter_rst_max_4/model; yosys -ql design_prep.log design_prep.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] base: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] prep: starting process "cd sby/up_down_counter_rst_max_15/model; yosys -ql design_prep.log design_prep.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] prep: starting process "cd sby/up_down_counter_rst_max_60/model; yosys -ql design_prep.log design_prep.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] prep: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] smt2: starting process "cd sby/up_down_counter_rst_max_4/model; yosys -ql design_smt2.log design_smt2.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_15] prep: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] smt2: starting process "cd sby/up_down_counter_rst_max_15/model; yosys -ql design_smt2.log design_smt2.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] prep: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] smt2: starting process "cd sby/up_down_counter_rst_max_60/model; yosys -ql design_smt2.log design_smt2.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] smt2: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_4; yosys-smtbmc -s z3 --presat --noprogress -t 3  --append 0 --dump-vcd
 engine_0/trace.vcd --dump-yw engine_0/trace.yw --dump-vlogtb engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_15] smt2: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: starting process "cd sby/up_down_counter_rst_max_4; yosys-smtbmc -s z3 --presat -i --noprogress -t 3  --append 0 --dump
-vcd engine_0/trace_induct.vcd --dump-yw engine_0/trace_induct.yw --dump-vlogtb engine_0/trace_induct_tb.v --dump-smtc engine_0/trace_induct.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] smt2: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_15; yosys-smtbmc -s z3 --presat --noprogress -t 3  --append 0 --dump-v
cd engine_0/trace.vcd --dump-yw engine_0/trace.yw --dump-vlogtb engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: starting process "cd sby/up_down_counter_rst_max_15; yosys-smtbmc -s z3 --presat -i --noprogress -t 3  --append 0 --du
mp-vcd engine_0/trace_induct.vcd --dump-yw engine_0/trace_induct.yw --dump-vlogtb engine_0/trace_induct_tb.v --dump-smtc engine_0/trace_induct.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: starting process "cd sby/up_down_counter_rst_max_60; yosys-smtbmc -s z3 --presat -i --noprogress -t 3  --append 0 --du
mp-vcd engine_0/trace_induct.vcd --dump-yw engine_0/trace_induct.yw --dump-vlogtb engine_0/trace_induct_tb.v --dump-smtc engine_0/trace_induct.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_60; yosys-smtbmc -s z3 --presat --noprogress -t 3  --append 0 --dump-v
cd engine_0/trace.vcd --dump-yw engine_0/trace.yw --dump-vlogtb engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: Status returned by engine for basecase: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
```

```
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: Status returned by engine for basecase: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: Status returned by engine for basecase: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: Status returned by engine for induction: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: Status returned by engine for induction: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: engine_0 did not produce any traces
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: successful proof by k-induction.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] DONE (PASS, rc=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: Status returned by engine for induction: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: engine_0 did not produce any traces
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: successful proof by k-induction.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] DONE (PASS, rc=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: finished (returncode=0)
```

```
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: Status returned by engine for basecase: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: engine_0 did not produce any traces
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: successful proof by k-induction.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```


root@0b9c66e66b04:/workspace# pytest -m formal -v
test session starts
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.12-linuxkit-x86_64-with-glibc2.39', 'Packages': {'pytest': '9.0.3', 'pluggy': '1.7.0'}, 'Plu
gins': {'metadata': '3.1.1', 'html': '4.2.0'}}
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 114 items / 106 deselected / 8 selected
tests/test_formal.py::test_formal_sby[cascade_counter] SKIPPED (cascade_counter module not implemented yet) [12%]
tests/test_formal.py::test_formal_sby[editable_countdown] SKIPPED (editable_countdown module not implemented yet) [25%]
tests/test_formal.py::test_formal_sby[snapshot_mux] SKIPPED (snapshot_mux module not implemented yet) [37%]
tests/test_formal.py::test_formal_sby[stopwatch_control] SKIPPED (stopwatch_control module not implemented yet) [50%]
tests/test_formal.py::test_formal_sby[stopwatch_counter] SKIPPED (stopwatch_counter module not implemented yet) [62%]
tests/test_formal.py::test_formal_sby[up_down_counter_rst] PASSED [75%]
tests/test_formal.py::test_formal_sby[user_top_brightness_wrapper] SKIPPED (user_top_brightness_wrapper module not implemented yet) [87%]
tests/test_formal.py::test_formal_sby[user_top_timer_v1] SKIPPED (user_top_timer_v1 module not implemented yet) [100%]
1 passed, 7 skipped, 106 deselected in 0.31s
root@0b9c66e66b04:/workspace#

3.2. Assertion File

The assertion file wraps the design under verification in a module that instantiates it and adds the formal properties to be checked. Because SVA concurrent assertions (which use the `property` and `sequence` keywords to express temporal properties) require the commercial Verific front-end and are therefore not available in open-source Yosys flows, we use immediate assertions instead: `assert` statements placed directly inside procedural blocks.

When a design passes formal verification, it is worth reflecting on what has actually been proved. The proof establishes not that this module *is* an up-down counter with reset, but that it satisfies the properties captured by the assertions above. Whether those properties fully capture the concept of an up-down counter is a question about language, not logic; a lawyer might ask whether any reasonable person would consider the specification complete. Engineers can afford to be more pragmatic: provided nothing more is asked of this module than what the assertions guarantee, it is fit for purpose.

Report 2

Do the assertions above fully capture the behaviour of an up-down counter with reset? If not, identify any behaviours left unspecified, or any properties a reasonable person might expect that the assertions do not guarantee.

Report 3

The configuration file verifies the counter for three specific values of `MAX`. Since this does not cover all possible values, how would you formally prove correctness when this module is used in a larger design?

reset has two states: HIGH and LOW
are there any input sequence involving `reset` that makes any asserts fail the current cycle?

enumerates all possible combination of signal states and runs the transitions

use BMC and k-induction

3.3. Assertions, Valid States and Verification Checks

The assertions are checked once per clock cycle. The formal verification tools assume ideal logic: signal values update just after a rising edge and remain constant until the next.

A state at cycle n is *valid* if no assertions fail at that cycle.

The two checks use this definition differently, as we now describe.

The formal verification tools run two checks: a bounded model check (BMC), which verifies that no assertion fails in the first k cycles, and a k-induction check, which verifies that any sequence of k consecutive valid states must be followed by another valid state. When both pass at the same depth k, together they constitute a complete proof of correctness for all time: the BMC establishes that the first k steps are assertion-free, and the induction check extends this guarantee indefinitely.

We use assertions in three block types: `initial`, `always @(*)`, and `always @(posedge clk)`. Each produces a counterexample when it fails; interpreting these correctly requires understanding how the two checks treat each block type. Since signals are constant within a cycle, it might seem that `always @(*)` and `always @(posedge clk)` are interchangeable. They are not.

An assertion in an `initial` block applies only to cycle 0. The induction checker ignores it, since induction requires assertions that can apply starting from any point in time, not just the initial state.

An assertion in an `always @(*)` block applies to all cycles, including cycle 0. Both checkers consider a state at cycle 0 to be invalid if it fails any such assertion, just as one would expect.

An assertion in an `always @(posedge clk)` block is attributed to the cycle being entered: the first time it triggers, it is attributed to cycle 1. Two things are natural: it is natural for the checker to attribute the assertion to the new cycle, and it is equally natural that the assertion checks the state of the previous cycle, as explained below. This mismatch does not compromise the validity of formal verification, but readers should keep it in mind when interpreting counterexample output.

Rising edges demarcate cycles: each edge marks the end of one cycle and the beginning of the next. The period before an edge is the *previous cycle*; the period after it is the *current cycle*. The first time `always @(posedge clk)` triggers, the circuit enters cycle 1. The checker adopts this labelling, which is entirely natural.

> Because flip-flop outputs are assigned via non-blocking assignment, they take effect only after all procedural blocks have finished. Inside an `always @(posedge clk)` block, signal values are therefore those from just before the rising edge, that is, from the previous cycle. In particular, at cycle 1 the assertions check the state at cycle 0.

To make this concrete, suppose the induction checker initialises the state at cycle 0 to `start=1` and `count=3`. No assertions are violated because no rising edge has yet occurred. When the first rising edge arrives and line 45 executes, the prover records a violation at cycle 1, but the condition it checks concerns cycle 0. Someone reading "invalid at step 1" would naturally examine cycle 1, while the actual problem lies in cycle 0.

The k-induction check for our counter fails when $k = 1$, an artefact of the mismatched labelling. From the counter's logic alone, if the state at cycle k is valid then the state at cycle $k + 1$ will also be valid, so one would expect the counter to pass 1-induction. The subtlety is that 1-induction is genuine 1-induction for `always @(*)` assertions, but behaves like 0-induction for `always @(posedge clk)` assertions, since those check the previous cycle's state. The counter therefore requires 2-induction to pass.

To see why, consider the 1-induction counterexample. The checker places `start=1` and `count=3` at cycle 0. No `always @(*)` assertions fail, so cycle 0 is deemed valid. At the first rising edge, labelled cycle 1, line 45 executes and finds `count` non-zero at cycle 0, recording

a violation at cycle 1. A valid cycle 0 has thus led to a violation at cycle 1, so 1-induction fails, even though the counter is correct.

This artefact does not compromise the soundness of formal verification. The looser constraint on cycle 0 gives the induction checker more freedom to construct an adversarial initial state, making induction *harder* to pass, not easier. Any design that does pass is genuinely correct; the artefact merely forces a higher depth. For our counter, $k = 2$ suffices: the 2-induction hypothesis covers two consecutive valid states, which is enough to exclude the spurious counterexample above.

3.4. Configuration File

The configuration file `sby/up_down_counter_rst.sby` is shown below, followed by a discussion of each part keyed to the line numbers.

Lines 1–3 set the proof depth to 3, which governs both checkers. The base-case checker verifies that no assertion is violated in cycles 0, 1, or 2. The induction checker searches for a different kind of counterexample: does there exist a run of three consecutive cycles with all assertions satisfied, immediately followed by a cycle that violates one? If no such run exists, the property holds for all time.

3.5. Expected Output

Because `sby` cannot verify over all possible parameter values, we select a representative sample of `MAX` values and define each as a separate task in the configuration file `up_down_counter_rst.sby`: `max_4`, `max_15`, and `max_60`. When run without a task name, the command above launches all tasks in parallel, producing interleaved output; the third column of each line identifies which task produced it.

The configuration file specifies a depth of 3, which governs both checks. The base-case checker verifies that no assertion fails at clock cycles 0, 1, or 2, starting from the known initial state. The induction checker runs independently, without constraining the starting state to be the initial state: it asks whether there exist any three consecutive cycles in which all assertions hold, but the successor cycle violates one. If no such sequence exists, the inductive step holds, and together with the base case, k-induction guarantees the assertions hold for all time.

The induction checker's output was interleaved with the previous block but finishes here. Its goal is to prove that no assertion is ever violated: for all n, no assertion fails at cycle n. Unlike the base-case checker, it imposes no constraint on the starting state, so the proof must cover all possible mid-execution states, not just those reachable immediately from startup.

In summary, each step widens the window of required valid states before the potential violation, restricting the set of starting states the checker must consider. Once that set is narrow enough that no violating run survives, the inductive step is proved. Together with the base-case result, the k-induction proof is complete.

3.6. Examining Counterexamples

To see what a failing verification looks like, consider an up-down counter in which the count-down wrap is omitted: rather than wrapping to `MAX` when `count` reaches zero, the code subtracts one unconditionally and lets the bit-vector underflow. Excerpts of the `sby` output for the `max_4` task are shown below in three parts, with prefixes and irrelevant lines removed.

The failed assertion is the range check on line 38 (`count <= MAX`). The trace is the counterexample `sby` found: the inputs `enable = 1`, `rst = 0`, and `up = 0` are applied while `count = 0`, and since the wrap is missing, `count` underflows to $2^{\texttt{WIDTH}} - 1$, which exceeds `MAX`.

The induction checker fails for the same reason. It searches for an arbitrary starting state and a sequence of inputs such that no assertion is violated in the first three cycles, but one is violated on the fourth; `count` can begin at any value the solver finds convenient.

Its counterexample is saved to `trace_induct.vcd`: opening it in VaporView shows `count` taking the values 3, 4, 0, then 7. Starting from `count = 3`, three up-count steps give 3, 4, 0, all within range (the wrap is correctly implemented); a final down-count step then causes `count` to underflow to 7, violating the range assertion.

The final summary confirms that both the base-case and induction checks failed, identifies the paths to the two counterexample trace files, and reports the specific assertion that was violated in each case, including its source location (`up_down_counter_rst_assert.sv`, line 38). The overall result is `FAIL` with return code 2, and `sby` lists the tasks that failed. The return code is useful when `sby` is run as part of an automated flow: a non-zero exit code signals failure to the calling script or build system.

Finding Counterexample Waveforms. Whenever `sby` reports a failure, you will need to open the counterexample trace files in VaporView. The paths to these files are printed in `sby`'s output, on lines labelled `counterexample trace`. For this run they are `sby/up_down_counter_rst_max_4/engine_0/trace.vcd` (base case) and `sby/up_down_counter_rst_max_4/engine_0/trace_induct.vcd` (induction). In future runs the filenames will differ, so read them directly from the output.

Valid Versus Reachable. The discussion below of k-induction's limitations hinges on the distinction between these two notions. The following example makes it concrete.

```
1 always_comb
2   if (count < 10) next_count = (count == 9) ? 0 : count + 1;
3   else next_count = (count == 19) ? 10 : count + 1;
```

This defines two independent counters: one cycling through 0–9 and the other through 10–19. No sequence of inputs can move the system from one range to the other. If `count` is initialised to 0, only the states 0–9 are reachable; the states 10–19 are not. Formally, a state is *reachable* if the design can arrive at it from the initial state defined by Verilog `initial` statements. A state is *valid* if it violates no assertion (Section 3.3). A *valid sequence* is a sequence of valid states in which each consecutive pair is connected by a transition that the design permits under some input. Not every valid state need be reachable.

Limitations of k-induction. Because the induction checker starts from an arbitrary state, its counterexamples are not guaranteed to be reachable. A *spurious* counterexample is one whose starting state is valid but unreachable from the true initial state of the design. When a spurious counterexample is present, the induction step fails even if the design is correct, and the proof cannot succeed until it is ruled out.

The simplest approach is to try increasing the depth: a longer lead-in of valid states is harder to construct, so some spurious counterexamples are ruled out as the depth grows.

However, this does not always work, particularly for designs with an `enable` input. A spurious counterexample found at one depth can be adapted to any greater depth by prepending additional cycles with `enable` held low (since nothing changes while `enable` is low, each such cycle remains valid). This exact problem arises in our counter if the following assertion is removed:

```
1 always @(*) assert (count <= MAX);
```

Without it, the induction checker cannot succeed at any depth. It can initialise `count` to a value greater than `MAX`, then hold `enable` low indefinitely without violating any assertion. Setting `enable` and `up` high then causes `count` to wrap to zero, but because it wraps from a value other than `MAX`, the transition violates the count-up assertion.

Task Temporarily comment out the `assert (count <= MAX)` line (shortcut: Ctrl+/), run `sby`, and observe that the induction step now fails. The paths to the counterexample trace files are printed in `sby`'s output; open them in VaporView. **Restore the assertion file before continuing.**

Report 4

Include the counterexample waveform. The general reason for this failure was explained above; use the waveform to identify the specific counterexample the checker found and explain why it is one. Then explain why restoring the assertion allows the induction step to succeed.

```
11  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_4; yosys-smtbmc -s z3 --presat --noprogress -t 3  --appen
12  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: starting process "cd sby/up_down_counter_rst_max_4; yosys-smtbmc -s z3 --presat -i --noprogress -t 3  --ap
13  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Solver: z3
14  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
15  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
16  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
17  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
18  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
19  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
20  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Status: passed
21  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: finished (returncode=0)
22  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: Status returned by engine for basecase: pass
23  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Solver: z3
24  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 3..
25  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 2..
26  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 1..
27  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 0..
28  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Temporal induction failed!
29  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Assert failed in up_down_counter_rst_assert: a_count_up
30  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Writing trace to VCD file: engine_0/trace_induct.vcd
31  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Writing trace to Verilog testbench: engine_0/trace_induct_tb.v
32  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Writing trace to constraints file: engine_0/trace_induct.smtc
33  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Writing trace to Yosys witness file: engine_0/trace_induct.yw
34  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Status: failed
35  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: finished (returncode=1)
36  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: Status returned by engine for induction: FAIL
37  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
38  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
39  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: engine_0 (smtbmc z3) returned pass for basecase
40  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: engine_0 (smtbmc z3) returned FAIL for induction
41  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: counterexample trace [induction]: sby/up_down_counter_rst_max_4/engine_0/trace_induct.vcd
42  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary:   failed assertion up_down_counter_rst_assert.a_count_up at up_down_counter_rst_assert.sv:63.7-63.65
43  SBY 11:28:03 [sby/up_down_counter_rst_max_4] DONE (UNKNOWN, rc=4)
44
```

```
sby > up_down_counter_rst_max_4 > UNKNOWN
1  Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
2  Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
3  engine_0 (smtbmc z3) returned pass for basecase
4  engine_0 (smtbmc z3) returned FAIL for induction
5  counterexample trace [induction]: engine_0/trace_induct.vcd
6  counterexample trace [induction]: engine_0/trace_induct.yw
7    failed assertion up_down_counter_rst_assert.a_count_up at up_down_counter_rst_assert.sv:63.7-63.65
8
```

```
SBY 11:28:03 [sby/up_down_counter_rst_max_60] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY 11:28:03 [sby/up_down_counter_rst_max_60] summary: engine_0 (smtbmc z3) returned FAIL for induction
SBY 11:28:03 [sby/up_down_counter_rst_max_60] summary: counterexample trace [induction]: sby/up_down_counter_rst_max_60/engine_0/trace_induct.vcd
SBY 11:28:03 [sby/up_down_counter_rst_max_60] summary:   failed assertion up_down_counter_rst_assert.a_count_up at up_down_counter_rst_assert.sv:63.7-63.65
SBY 11:28:03 [sby/up_down_counter_rst_max_60] DONE (UNKNOWN, rc=4)
SBY 11:28:03 The following tasks failed: ['max_4', 'max_60']
root@0b9c66e66b04:/workspace#
```

sby > up_down_counter_rst_max_4 > engine_0 > logfile_induction.txt

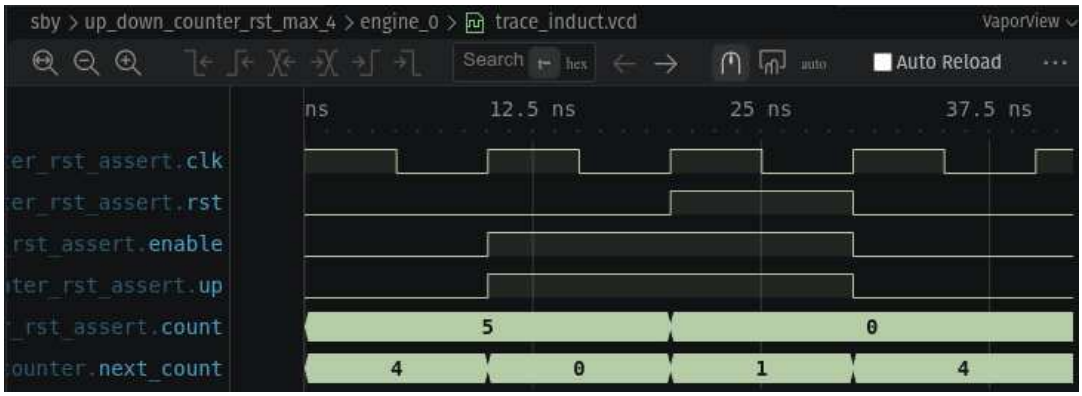
```
##   0:00:00  Solver: z3
##   0:00:00  Trying induction in step 3..
##   0:00:00  Trying induction in step 2..
##   0:00:00  Trying induction in step 1..
##   0:00:00  Trying induction in step 0..
##   0:00:00  Temporal induction failed!
##   0:00:00  Assert failed in up_down_counter_rst_assert: a_count_up
##   0:00:00  Writing trace to VCD file: engine_0/trace_induct.vcd
##   0:00:00  Writing trace to Verilog testbench: engine_0/trace_induct_tb.v
##   0:00:00  Writing trace to constraints file: engine_0/trace_induct.smtc
##   0:00:00  Writing trace to Yosys witness file: engine_0/trace_induct.yw
##   0:00:00  Status: failed
```

rtl > up_down_counter_rst.sv > ...

```
51  always_comb begin
52    if (up) begin  // up=1 -> Count up
53      if (count < Max) next_count = count + WIDTH'(1);
54      else next_count = WIDTH'(0);  // Wrap to 0 when Max
55    end else begin  // else up=0 -> Count down
56      if (count > WIDTH'(0)) next_count = count - WIDTH'(1);
57      else next_count = Max;  // Wrap to Max when 0
58    end
59  end
```

sby > up_down_counter_rst_assert.sv > ...

```
58  // Check count up
59  logic wrap_up;
60  always @(posedge clk) begin
61    wrap_up = ($past(count == MAX) && count == 0);
62    if (!start && $past(up && enable && !rst))
63      a_count_up : assert (count == $past(count) + 1 || wrap_up);
64  end
```





MAX_4
- [2:0] count = 5 is a possible state (3'b101)
- assert `a_count_up` expects count to be next reachable 6 at k-2 (step 2/cycle 2) but is 0 due to next_count = 0 at k-1 (step 1/cycle 1) by the next stage logic in the DUT module. Since 0 =/= 5+1, assert fails. Although the next state 0 is reachable, the transition is invalid and thus count 5 to count 0 is an invalid sequence.
- If assert (count<=MAX) exists, the starting states of `count` will be restricted to 0-4

Feasibility of Formal Verification. In principle, formal verification of finite-state designs such as digital circuits is decidable: for a correct design, there exists an inductive invariant that proves the property. In practice, however, two barriers limit its applicability. First, the search space grows exponentially with the number of state bits, so large designs may be computationally intractable. Second, finding the necessary assertion strengthening for k-induction cannot in general be automated: the auxiliary assertions needed to rule out spurious counterexamples may be non-obvious and require human insight. At the scale of this project, a complete proof is typically achievable. For industrial-scale designs, formal verification remains an active research area, and engineers sometimes accept bounded model checking as a practical substitute.

4. Stopwatch

In Assignment 1 we developed a settable watch; by the end of this assignment it will be extended with a stopwatch, a timer, and display brightness control. Because these features are loosely coupled, we begin by setting the settable watch aside and focusing purely on the stopwatch.

The board wrapper from Assignment 1 is reused here, so we need only concern ourselves with the interface defined by the `user_top` module created there.

4.1. Specification

The stopwatch displays minutes, seconds, and centiseconds on the seven-segment displays. The two buttons operate as follows.

Button	Function
`button[0]`	Start / Stop
`button[1]`	Lap (while running) / Reset (while stopped)

Start/Stop (`button[0]`) Pressing `button[0]` toggles the stopwatch between running and stopped. It does not affect the display state (live or frozen). The first centisecond increment must occur 10 ms after the button press, not sooner.

Lap and Reset (`button[1]`) The behaviour of `button[1]` depends on the current state:

- While running with a live display, pressing `button[1]` freezes the display at the current time; the stopwatch continues running in the background.
- While running with a frozen display, pressing `button[1]` unfreezes the display.
- While stopped with a frozen display, pressing `button[1]` unfreezes the display; the time is not reset.
- While stopped with a live display, pressing `button[1]` resets the time to zero; the stopwatch remains stopped and the display remains live.

Simultaneous Presses If both buttons are pressed on the same clock edge, both are ignored.

4.2. Design Approach

While this sounds straightforward, particularly given that modules from Assignment 1 can be reused, implementing the stopwatch as a single monolithic module is harder than it appears. A well-decomposed design keeps each module focused on at most one or two non-trivial decisions.

Report 5

Before reading further, sketch how you would approach the design.

Report 6

Using a feature layering approach, how would you decompose the stopwatch into layers? Describe the responsibility of each layer.

The resulting design has a natural structure. The *data path* (the chain of modules through which the time value flows toward the display) runs from the stopwatch counter through the snapshot multiplexer to the display driver in the board wrapper. Alongside it sits a small set of *control signals*: reset and enable for the counter, and a freeze signal for the multiplexer. A separate module will drive these via a simple FSM that responds to button presses.

Report 7

Draw a block diagram of the design described above, clearly showing the data path and control signals. Compare it with your initial sketch from Section 4.2 and comment on any differences.

Because `stopwatch_counter` has a non-trivial timing constraint, we keep the implementation concise by factoring out a cascade counter as a separate module. It will be driven by the `restartable_rate_generator` from Assignment 1.

4.3. Cascade Counter

All outputs are initialised to 0.

When `rst` is high, all outputs are set to 0 on the next rising clock edge, regardless of `enable`.

When `rst` is low and `enable` is low, all outputs are unchanged.

When `rst` is low and `enable` is high, `count0` increments on each rising clock edge, wrapping from `N0-1` to 0. When `count0` wraps, `count1` increments, wrapping from `N1-1` to 0. When `count1` wraps, `count2` increments, wrapping from `N2-1` to 0.

Dependencies Only the following module is to be directly instantiated.

- `mod_n_counter`

rtl > cascade_counter.sv > ...

```sv
// Cascade Counter
// Consists of three cascading Modulo N Counters that syncs three sets of
// counters together, where the rollover of initial counter performs the tick
// for the next counter.
//
// Parameters:
//    N2:    - Max 'length' value for Counter N2
//    N1:    - Max 'length' value for Counter N1
//    N0:    - Max 'length' value for Counter N0
//    W2:    - Output port bit width of `N2-1` value
//    W1:    - Output port bit width of `N1-1` value
//    W0:    - Output port bit width of `N0-1` value
//
// Ports:
//    clk:              - Clock input
//    rst:              - Asynchronous reset input
//    enable:           - When HIGH, module is enabled
//    [W2-1:0] count2:  - Output Signal of counter N2 (0 to N2-1)
//    [W1-1:0] count1:  - Output Signal of counter N1 (0 to N1-1)
//    [W0-1:0] count0:  - Output Signal of counter N0 (0 to N0-1)
//
// Local Variables:
//    CountN2, CountN1, CountN1: - To hold their respective parameter values
//                                 (N2-1), (N1-1), (N0-1) with their correct
//                                 bit widths
//    enable1:  - HIGH only when N0 rolls over and enable is HIGH
//    enable2:  - HIGH only when N1 rolls over and enable1 is HIGH

`timescale 1ns / 1ps

module cascade_counter #(
    parameter int N2 = 3,
    parameter int N1 = 4,
    parameter int N0 = 5,

    // Output port widths
    parameter int W2 = 2,
    parameter int W1 = 2,
    parameter int W0 = 3
) (
    input logic clk,
    input logic rst,
    input logic enable,
    output logic [W2-1:0] count2,
    output logic [W1-1:0] count1,
    output logic [W0-1:0] count0
);

  localparam logic [W2-1:0] CountN2 = (W2)'(N2) - 1;
  localparam logic [W1-1:0] CountN1 = (W1)'(N1) - 1;
```

```
    localparam logic [W1-1:0] CountN1 = (W1)'(N1) - 1;
    localparam logic [W0-1:0] CountN0 = (W0)'(N0) - 1;
    logic enable1, enable2;

    mod_n_counter #(
        .N(CountN0 + 1),
        .WIDTH(W0)
    ) u_count_N0 (
        .clk(clk),
        .rst(rst),
        .enable(enable),
        .count(count0)
    );

    mod_n_counter #(
        .N(CountN1 + 1),
        .WIDTH(W1)
    ) u_count_N1 (
        .clk(clk),
        .rst(rst),
        .enable(enable1),
        .count(count1)
    );

    mod_n_counter #(
        .N(CountN2 + 1),
        .WIDTH(W2)
    ) u_count_N2 (
        .clk(clk),
        .rst(rst),
        .enable(enable2),
        .count(count2)
    );

    always_comb begin
      if (count0 == CountN0 && enable) enable1 = 1'b1;
      else enable1 = 1'b0;
      if (count1 == CountN1 && enable1) enable2 = 1'b1;
      else enable2 = 1'b0;
    end

  endmodule
```

```
root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/cascade_counter.sv
root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/cascade_counter.sv
root@0b9c66e66b04:/workspace#
```

```
SBY  5:30:38 [sby/cascade_counter_default] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:30:38 [sby/cascade_counter_default] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:30:38 [sby/cascade_counter_default] summary: engine_0 did not produce any traces
SBY  5:30:38 [sby/cascade_counter_default] summary: successful proof by k-induction.
SBY  5:30:38 [sby/cascade_counter_default] DONE (PASS, rc=0)
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Solver: z3
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Status: passed
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: finished (returncode=0)
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: Status returned by engine for induction: pass
SBY  5:30:38 [sby/cascade_counter_large] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:30:38 [sby/cascade_counter_large] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:30:38 [sby/cascade_counter_large] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:30:38 [sby/cascade_counter_large] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:30:38 [sby/cascade_counter_large] summary: engine_0 did not produce any traces
SBY  5:30:38 [sby/cascade_counter_large] summary: successful proof by k-induction.
SBY  5:30:38 [sby/cascade_counter_large] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```

```
root@0b9c66e66b04:/workspace# pytest -m formal -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0 -- /opt/venv/bin/pyth
on3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.12-linuxkit-x86_64-with-gli
bc2.39', 'Packages': {'pytest': '9.0.3', 'pluggy': '1.7.0'}, 'Plugins': {'metadat
a': '3.1.1', 'html': '4.2.0'}}
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 116 items / 108 deselected / 8 selected

tests/test_formal.py::test_formal_sby[cascade_counter] PASSED             [ 12%]
tests/test_formal.py::test_formal_sby[editable_countdown] SKIPPED (ed...) [ 25%]
tests/test_formal.py::test_formal_sby[snapshot_mux] SKIPPED (snapshot...) [ 37%]
tests/test_formal.py::test_formal_sby[stopwatch_control] SKIPPED (sto...) [ 50%]
tests/test_formal.py::test_formal_sby[stopwatch_counter] SKIPPED (sto...) [ 62%]
tests/test_formal.py::test_formal_sby[up_down_counter_rst] PASSED         [ 75%]
tests/test_formal.py::test_formal_sby[user_top_brightness_wrapper] SKIPPED [ 87%]
tests/test_formal.py::test_formal_sby[user_top_timer_v1] SKIPPED (use...) [100%]

================ 2 passed, 6 skipped, 108 deselected in 0.71s ==================
root@0b9c66e66b04:/workspace#
```

4.4. Stopwatch Counter

All outputs are initialised to 0.

When `rst` is high, all outputs are set to 0 on the next rising clock edge, regardless of `enable`.

When `rst` is low and `enable` is low, all outputs are unchanged.
When `rst` is low and `enable` is high, `centiseconds` increments once per centisecond, wrapping from 99 to 0. When `centiseconds` wraps, `seconds` increments, wrapping from 59 to 0. When `seconds` wraps, `minutes` increments, wrapping from 99 to 0. The first increment must occur one centisecond after `enable` goes high, not sooner.

rtl > stopwatch_counter.sv > ...

```sv
// Stopwatch Counter
// A module that counts (99) minutes, (59) seconds and (99) centiseconds using
// the Restartable Rate Generator that ticks per 1/10th of CYCLES_PER_SECOND
// and the Cascade Counter that drives the Output Signals.
//
// Parameters:
//    CYCLES_PER_SECOND: - Number of cycles per second
//
// Ports:
//    clk:                - Clock input
//    rst:                - Asynchronous reset input
//    enable:             - When HIGH, module is enabled
//    [6:0] minutes:      - Output Signal for 7'd99 minutes
//    [5:0] seconds:      - Output Signal for 6'd59 seconds
//    [6:0] centiseconds: - Output Signal for 7'd99 centiseconds
//
// Local Variables:
//    gen_run:  - HIGH *only* when `rst`:LOW and `enable`:HIGH
//    tick:     - HIGH when u_centisecond_counter completes a cycle
//    tick_centisecond: - HIGH only when `tick`:HIGH and `enable`:HIGH

`timescale 1ns / 1ps

module stopwatch_counter #(
    parameter int CYCLES_PER_SECOND = 50_000_000
) (
    input logic clk,
    input logic rst,  // Takes priority over enable
    input logic enable,
    output logic [6:0] minutes,
    output logic [5:0] seconds,
    output logic [6:0] centiseconds  // hundredths of a second
);

  logic gen_run;
  logic tick;
  logic tick_centisecond;

  restartable_rate_generator #(
      .CYCLE_COUNT(CYCLES_PER_SECOND / 100)
  ) u_centisecond_counter (
      .clk (clk),
      .run (gen_run),
      .tick(tick)
  );

  cascade_counter #(
      .N0(100),
      .N1(60),
      .N2(100),
      .W0(7),
      .W1(6),
      .W2(7)
  ) u_stopwatch_counter (
      .clk(clk),
      .rst(rst),
      .enable(tick_centisecond),
      .count0(centiseconds),
      .count1(seconds),
      .count2(minutes)
  );

  always_comb begin
    if (rst || !enable) begin
      gen_run = 1'b0;
      tick_centisecond = 1'b0;
    end else begin
      gen_run = 1'b1;
      tick_centisecond = tick;
    end
  end

endmodule

```

```
root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/stopwatch_counter.sv
root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/stopwatch_counter.sv
root@0b9c66e66b04:/workspace#
```

```
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:01 (1)
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: engine_0 did not produce any traces
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: successful proof by k-induction.
SBY  8:49:38 [sby/stopwatch_counter_div_7] DONE (PASS, rc=0)
SBY  8:49:38 [sby/stopwatch_counter_div_3] engine_0.basecase: ##   0:00:00  Status: passed
SBY  8:49:38 [sby/stopwatch_counter_div_3] engine_0.basecase: finished (returncode=0)
SBY  8:49:38 [sby/stopwatch_counter_div_3] engine_0.basecase: Status returned by engine for basecase: pass
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:02 (2)
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: engine_0 did not produce any traces
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: successful proof by k-induction.
SBY  8:49:38 [sby/stopwatch_counter_div_3] DONE (PASS, rc=0)
SBY  8:49:38 [sby/stopwatch_counter_div_8] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  8:49:38 [sby/stopwatch_counter_div_8] engine_0.induction: ##   0:00:00  Status: passed
SBY  8:49:38 [sby/stopwatch_counter_div_8] engine_0.induction: finished (returncode=0)
SBY  8:49:38 [sby/stopwatch_counter_div_8] engine_0.induction: Status returned by engine for induction: pass
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:02 (2)
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: engine_0 did not produce any traces
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: successful proof by k-induction.
SBY  8:49:38 [sby/stopwatch_counter_div_8] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```

```
root@0b9c66e66b04:/workspace# pytest -m formal -v
================================= test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.12-linuxkit-x86_64-with-glibc2.39',
'Packages': {'pytest': '9.0.3', 'pluggy': '1.7.0'}, 'Plugins': {'metadata': '3.1.1', 'html
': '4.2.0'}}
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 118 items / 110 deselected / 8 selected

tests/test_formal.py::test_formal_sby[cascade_counter] PASSED                    [ 12%]
tests/test_formal.py::test_formal_sby[editable_countdown] SKIPPED (editable_co...) [ 25%]
tests/test_formal.py::test_formal_sby[snapshot_mux] SKIPPED (snapshot_mux modu...) [ 37%]
tests/test_formal.py::test_formal_sby[stopwatch_control] SKIPPED (stopwatch_co...) [ 50%]
tests/test_formal.py::test_formal_sby[stopwatch_counter] PASSED                  [ 62%]
tests/test_formal.py::test_formal_sby[up_down_counter_rst] PASSED                [ 75%]
tests/test_formal.py::test_formal_sby[user_top_brightness_wrapper] SKIPPED (us...) [ 87%]
tests/test_formal.py::test_formal_sby[user_top_timer_v1] SKIPPED (user_top_tim...) [100%]

====================== 3 passed, 5 skipped, 110 deselected in 1.11s ====================
root@0b9c66e66b04:/workspace#
```

Report 8

Explain how your implementation satisfies the first-tick timing constraint: the first centisecond increment must occur one centisecond after `enable` goes high, not sooner.

The Restartable Rate Generator (RRG) module does not have an input `reset` port to reset it's clock, it does so by taking the invert of the input `run` port: Disabling the RRG module resets it's internal count. It is tempting to wire the RRG's output `tick` port to the Cascade Counter (CC) module input `enable` port. This is incorrect as in the edge case of `tick`=HIGH and `enable`=LOW, the CC counter will tick which is not what we want.

We thus have to

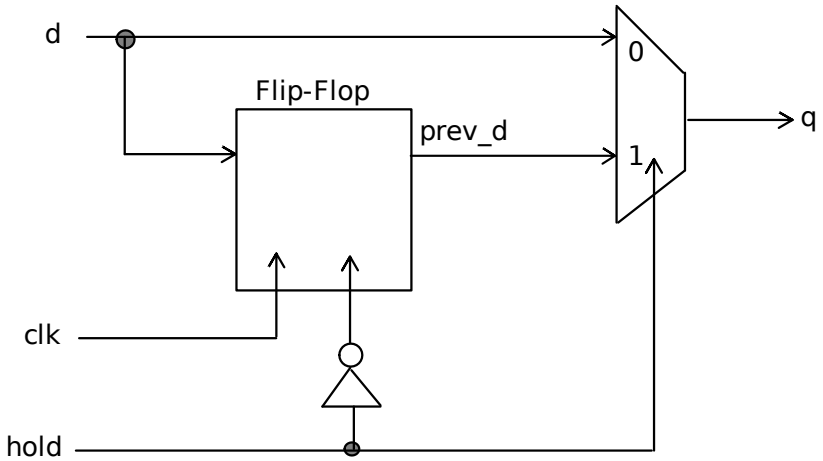
4.5. Snapshot Multiplexer

When `hold` is low, `q` equals `d` combinatorially — there is no clock edge involved; any change in `d` propagates to `q` immediately.

When `hold` is high, `q` is frozen at the value `d` held on the last rising clock edge before `hold` went high.

Report 9

Draw a diagram showing how this is implemented in hardware.



rtl > snapshot_mux.sv > SystemVerilog - Language Support > snapshot_mux

```sv
// Snapshot Multiplexer
// Output `q` takes `d` when `hold` is LOW. When `hold` is HIGH, q takes last
// known `d` during `hold`'s last LOW that crossed the rising edge.
//
// Parameters:
//    WIDTH:  - Bit width of input `d` and output `q`.
//
// Ports:
//    clk:            - Clock input
//    hold:           - Input Signal representing the Mux Switch's `sel`.
//    [WIDTH-1:0] d:  - Input Signal, D-FlipFlop `d` input
//    [WIDTH-1:0] q:  - Output Signal, D-FlipFlop `q` output
//
// Local Variables:
//    [WIDTH-1:0] prev_d: - Intermediary Signal, stores `d` at `hold`'s last
//                          LOW signal


module snapshot_mux #(
    parameter int WIDTH = 1
) (
    input logic clk,
    input logic hold,
    input logic [WIDTH-1:0] d,
    output logic [WIDTH-1:0] q
);

  logic [WIDTH-1:0] prev_d = WIDTH'(0);

  always_ff @(posedge clk) begin
    if (!hold) prev_d <= d;
  end

  always_comb begin
    if (!hold) q = d;
    else q = prev_d;
  end

endmodule

```

```
root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/snapshot_mux.sv
root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/snapshot_mux.sv
root@0b9c66e66b04:/workspace#
```

```
SBY 10:11:27 [sby/snapshot_mux_width_1] engine_0.induction: Status returned by engine for induction: pass
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: engine_0 (smtbmc z3) returned pass for induction
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: engine_0 did not produce any traces
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: successful proof by k-induction.
SBY 10:11:27 [sby/snapshot_mux_width_1] DONE (PASS, rc=0)
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Solver: z3
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Status: passed
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: finished (returncode=0)
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: Status returned by engine for basecase: pass
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: engine_0 (smtbmc z3) returned pass for induction
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: engine_0 did not produce any traces
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: successful proof by k-induction.
SBY 10:11:27 [sby/snapshot_mux_width_20] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```

```
root@0b9c66e66b04:/workspace# pytest -m formal -v
================================== test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.12-linuxkit-x86_64-with-glibc2.39', '
Packages': {'pytest': '9.0.3', 'pluggy': '1.7.0'}, 'Plugins': {'metadata': '3.1.1', 'html':
 '4.2.0'}}
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 120 items / 112 deselected / 8 selected

tests/test_formal.py::test_formal_sby[cascade_counter] PASSED                    [ 12%]
tests/test_formal.py::test_formal_sby[editable_countdown] SKIPPED (editable_cou...) [ 25%]
tests/test_formal.py::test_formal_sby[snapshot_mux] PASSED                       [ 37%]
tests/test_formal.py::test_formal_sby[stopwatch_control] SKIPPED (stopwatch_con...) [ 50%]
tests/test_formal.py::test_formal_sby[stopwatch_counter] PASSED                  [ 62%]
tests/test_formal.py::test_formal_sby[up_down_counter_rst] PASSED                [ 75%]
tests/test_formal.py::test_formal_sby[user_top_brightness_wrapper] SKIPPED (use...) [ 87%]
tests/test_formal.py::test_formal_sby[user_top_timer_v1] SKIPPED (user_top_time...) [100%]

================== 4 passed, 4 skipped, 112 deselected in 1.44s ==================
root@0b9c66e66b04:/workspace#
```

4.6. Stopwatch Control

It is assumed that the instantiating module will use rising-edge detectors, so the two inputs are single-cycle pulses, each high for exactly one clock cycle when the corresponding button is pressed.

The module implements the FSM described implicitly in Section 4.1. The following additional constraints apply.

All outputs are initialised to 0.

The `counter_rst` output is high for exactly one clock cycle when asserted: as soon as `rise_lap` falls, the FSM exits the Reset state.

Report 10

Draw the FSM state diagram before implementing it. After completing the implementation, note any differences between your diagram and the final design, and reflect on what caused them.

The formal verification tool relies on `yosys` to compile the SystemVerilog code. However, `yosys` can promote `logic x=y;` into `assign x=y;`, and so may not detect that the former is an initialisation rather than a continuous assignment. An automated test bench has therefore also been provided: run `pytest`.

Hints

- Using `{counter_rst, counter_enable, lap_hold}` as the state variable makes the code easy to read and to write.
- Define the next-state logic for `counter_rst`, `counter_enable`, and `lap_hold` in three separate blocks, one per signal: two require only a single `assign` statement each, and the third requires an `always_comb` block.

Additional Oral Preparation 1

The hint above suggests encoding the FSM state directly as its output signals rather than using a separate state variable. Be prepared to explain why this works and what advantage it offers.

rtl > stopwatch_control.sv > ...

```
// Stopwatch Control
// When in Stopwatch mode, input buttons KEY[1:0] control the state of
// the stopwatch
//
// Parameters:
//    None
//
// Ports:
//    clk:              - Clock input
//    rise_start_stop: - Input signal from KEY[0], assumed passed in as a
//                       pulse from a Rising Edge Detector module
//    rise_lap:         - Input signal from KEY[1], same assumption as above
//    counter_rst:      - Output signal, single one-cycle pulse of RESET state
//    counter_enable:   - Output signal, `enabled` state
//    counter_lap_hold:- Output signal, 'frozen display' state
//
// Local Variables:
//    state:       - Intermediary Variable to store state for output signals
//    next_state: - Intermediary Variable to store state for next rising edge
//    ss_only:     - Combinational Logic to ensure that if both buttons are
//    lap_only:        pressed, nothing happens

`timescale 1ns / 1ps

module stopwatch_control (
    input  logic clk,
    input  logic rise_start_stop,  // Button[0]
    input  logic rise_lap,         // Button[1]
    output logic counter_rst,      // Reset Pulse
    output logic counter_enable,   // Running(1) or Stopped(0)
    output logic lap_hold          // Freeze(1) or Live(0)
);

  logic [2:0] state, next_state;
  initial state = 3'b000;

  wire ss_only = rise_start_stop && !rise_lap;
  wire lap_only = !rise_start_stop && rise_lap;

  always_ff @(posedge clk) begin
    state <= next_state;
  end

  assign next_state[1] = ss_only ? !state[1] : state[1];
  assign next_state[0] = ((counter_enable || lap_hold) && lap_only) ? !state[0] : state[0];

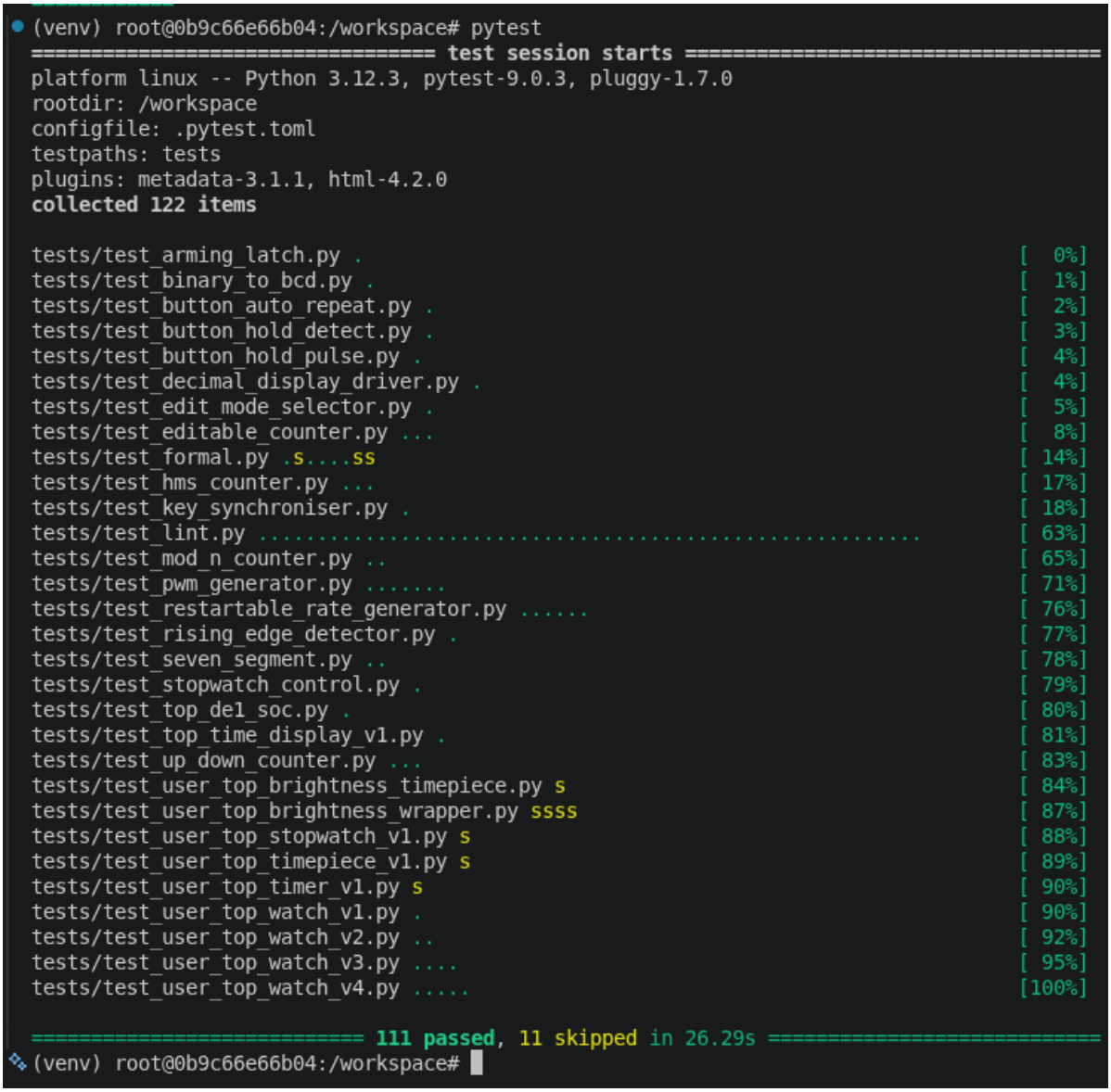
  // TODO: Fix this Latch!!!
  always_latch begin
    if (!(counter_enable || lap_hold)) begin
      next_state[2] = lap_only;
    end
  end

  assign counter_rst = state[2];
  assign counter_enable = state[1];
  assign lap_hold = state[0];

endmodule
```

```
(venv) root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/stopwatch_control.sv
(venv) root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/stopwatch_control.sv
(venv) root@0b9c66e66b04:/workspace#
```

```
SBY  2:43:45 [sby/stopwatch_control] engine_0.induction: finished (returncode=0)
SBY  2:43:45 [sby/stopwatch_control] engine_0.induction: Status returned by engine for induction: pass
SBY  2:43:45 [sby/stopwatch_control] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  2:43:45 [sby/stopwatch_control] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  2:43:45 [sby/stopwatch_control] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  2:43:45 [sby/stopwatch_control] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  2:43:45 [sby/stopwatch_control] summary: engine_0 did not produce any traces
SBY  2:43:45 [sby/stopwatch_control] summary: successful proof by k-induction.
SBY  2:43:45 [sby/stopwatch_control] DONE (PASS, rc=0)
(venv) root@0b9c66e66b04:/workspace#
```


(venv) root@0b9c66e66b04:/workspace# pytest
test session starts
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 122 items
tests/test_arming_latch.py . [0%]
tests/test_binary_to_bcd.py . [1%]
tests/test_button_auto_repeat.py . [2%]
tests/test_button_hold_detect.py . [3%]
tests/test_button_hold_pulse.py . [4%]
tests/test_decimal_display_driver.py . [4%]
tests/test_edit_mode_selector.py . [5%]
tests/test_editable_counter.py ... [8%]
tests/test_formal.py .s....ss [14%]
tests/test_hms_counter.py ... [17%]
tests/test_key_synchroniser.py . [18%]
tests/test_lint.py ..... [63%]
tests/test_mod_n_counter.py .. [65%]
tests/test_pwm_generator.py ..... [71%]
tests/test_restartable_rate_generator.py ..... [76%]
tests/test_rising_edge_detector.py . [77%]
tests/test_seven_segment.py .. [78%]
tests/test_stopwatch_control.py . [79%]
tests/test_top_de1_soc.py . [80%]
tests/test_top_time_display_v1.py . [81%]
tests/test_up_down_counter.py ... [83%]
tests/test_user_top_brightness_timepiece.py s [84%]
tests/test_user_top_brightness_wrapper.py ssss [87%]
tests/test_user_top_stopwatch_v1.py s [88%]
tests/test_user_top_timepiece_v1.py s [89%]
tests/test_user_top_timer_v1.py s [90%]
tests/test_user_top_watch_v1.py . [90%]
tests/test_user_top_watch_v2.py .. [92%]
tests/test_user_top_watch_v3.py .... [95%]
tests/test_user_top_watch_v4.py ..... [100%]
111 passed, 11 skipped in 26.29s
(venv) root@0b9c66e66b04:/workspace#

4.7. Stopwatch Integration

Task Duplicate `rtl/user_top.sv`, rename the file to `rtl/user_top_stopwatch_v1.sv`, and change the module name to `user_top_stopwatch_v1`. Then implement the stopwatch in accordance with Section 4.1, following all conventions from Assignment 1, including elimination of all Verilator linting warnings. Then run `pytest` to test it.

Dependencies Only the following modules are to be directly instantiated.

- `rising_edge_detector`
- `snapshot_mux`
- `stopwatch_control`
- `stopwatch_counter`

Note Formal verification is not used for `user_top_stopwatch_v1`. Integration modules are less amenable to it for two reasons. First, writing a complete formal specification at this level is as complex as writing the implementation itself. Second, what we want to check at the top level is that long sequences of inputs — press start, wait, press lap, press stop, check the display — produce the correct outputs; formal verification is not naturally suited to this, excelling instead at proving invariants and simple cause-and-effect properties of the form "event A implies event B on the next clock edge," which is why it works well for individual submodules. Simulation, by contrast, can directly execute arbitrary input sequences, making it the right tool for integration testing.

Additional Oral Preparation 2

Be prepared to explain why formal verification is well suited to individual submodules but less suited to integration-level testing, and why simulation is preferred at the integration level.

4.8. Programming the Board

Open the Quartus project from Assignment 1, copy your `rtl/*.sv` files into the project directory (overwriting the existing ones), then go to Project ⟩ Add/Remove Files and add any new `*.sv` files.

In `top_de1_soc.sv`, replace `user_top_watch_v4` with `user_top_stopwatch_v1` and save.

Compile and program the board.

```
`timescale 1ns / 1ps

module user_top_stopwatch_v1 #(
    /* verilator lint_off UNUSEDPARAM */
    parameter int CYCLES_PER_SECOND = 50_000_000
    /* verilator lint_on UNUSEDPARAM */
) (
    input logic clk,
    /* verilator lint_off UNUSED */
    input logic [3:0] button,
    input logic [9:0] sw,
    /* verilator lint_on UNUSED */
    output logic [9:0] led,
    output logic [6:0] hours_disp,
    output logic [6:0] minutes_disp,
    output logic [6:0] seconds_disp,
    output logic blank_hours,
    output logic blank_minutes,
    output logic blank_seconds
);

  assign led = clk ? sw : ~sw;

  logic ss, lap;
  logic counter_rst, counter_enable, lap_hold;
  logic [6:0] minutes, minutes_hold;
  logic [5:0] seconds, seconds_hold;
  logic [6:0] centiseconds, centiseconds_hold;

  rising_edge_detector u_red_ss (
      .clk(clk),
      .sig_in(button[0]),
      .rise(ss)
  );

  rising_edge_detector u_red_lap (
      .clk(clk),
      .sig_in(button[1]),
      .rise(lap)
  );

  stopwatch_control u_stopwatch_control (
      .clk(clk),
      .rise_start_stop(ss),
      .rise_lap(lap),
      .counter_rst(counter_rst),
      .counter_enable(counter_enable),
      .lap_hold(lap_hold)
  );

  snapshot_mux #(
      .WIDTH(7)
  ) u_snapshot_minutes_mux (
      .clk(clk),
      .hold(lap_hold),
      .d(minutes),
      .q(minutes_hold)
  );

  snapshot_mux #(
      .WIDTH(6)
  ) u_snapshot_seconds_mux (
      .clk(clk),
      .hold(lap_hold),
      .d(seconds),
      .q(seconds_hold)
  );

  snapshot_mux #(
      .WIDTH(7)
  ) u_snapshot_centiseconds_mux (
      .clk(clk),
      .hold(lap_hold),
      .d(centiseconds),
      .q(centiseconds_hold)
  );

  stopwatch_counter #(
      .CYCLES_PER_SECOND(CYCLES_PER_SECOND)
  ) u_stopwatch_counter (
      .clk(clk),
      .rst(counter_rst),
      .enable(counter_enable),
      .minutes(minutes),
      .seconds(seconds),
      .centiseconds(centiseconds)
  );

  assign hours_disp = (lap_hold) ? minutes_hold : minutes;
  assign minutes_disp = (lap_hold) ? {1'b0, seconds_hold} : {1'b0, seconds};
  assign seconds_disp = (lap_hold) ? centiseconds_hold : centiseconds;

  assign blank_hours = 1'b0;
  assign blank_minutes = 1'b0;
  assign blank_seconds = 1'b0;

endmodule
```

```
(venv) root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/user_top_stopwatch_v1.sv
(venv) root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/user_top_stopwatch_v1.sv
(venv) root@0b9c66e66b04:/workspace#
```

```
(venv) root@0b9c66e66b04:/workspace# pytest
================================= test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 124 items

tests/test_arming_latch.py .                                                 [  0%]
tests/test_binary_to_bcd.py .                                                [  1%]
tests/test_button_auto_repeat.py .                                           [  2%]
tests/test_button_hold_detect.py .                                           [  3%]
tests/test_button_hold_pulse.py .                                            [  4%]
tests/test_decimal_display_driver.py .                                       [  4%]
tests/test_edit_mode_selector.py .                                           [  5%]
tests/test_editable_counter.py ...                                           [  8%]
tests/test_formal.py .s....ss                                                [ 14%]
tests/test_hms_counter.py ...                                                [ 16%]
tests/test_key_synchroniser.py .                                             [ 17%]
tests/test_lint.py ..........................................................  [ 64%]
tests/test_mod_n_counter.py ..                                               [ 66%]
tests/test_pwm_generator.py .......                                          [ 71%]
tests/test_restartable_rate_generator.py ......                              [ 76%]
tests/test_rising_edge_detector.py .                                         [ 77%]
tests/test_seven_segment.py ..                                               [ 79%]
tests/test_stopwatch_control.py .                                            [ 79%]
tests/test_top_de1_soc.py .                                                  [ 80%]
tests/test_top_time_display_v1.py .                                          [ 81%]
tests/test_up_down_counter.py ...                                            [ 83%]
tests/test_user_top_brightness_timepiece.py s                                [ 84%]
tests/test_user_top_brightness_wrapper.py ssss                               [ 87%]
tests/test_user_top_stopwatch_v1.py .                                        [ 88%]
tests/test_user_top_timepiece_v1.py s                                        [ 89%]
tests/test_user_top_timer_v1.py s                                            [ 90%]
tests/test_user_top_watch_v1.py .                                            [ 91%]
tests/test_user_top_watch_v2.py ..                                           [ 92%]
tests/test_user_top_watch_v3.py ....                                         [ 95%]
tests/test_user_top_watch_v4.py .....                                        [100%]

========================== 114 passed, 10 skipped in 25.66s ============================
(venv) root@0b9c66e66b04:/workspace#
```

5. Timer

With the stopwatch complete, we now design and implement a timer.

5.1. Specification

The timer uses the same time-setting interface as the settable watch from Assignment 1. The timer has three modes: set, stopped, and running.

Stopped Mode. The display shows the current timer value. Pressing `button[0]` starts the timer if the count is greater than zero; otherwise it has no effect. The first decrement occurs not sooner than one second after the button is pressed. Holding `button[3]` for at least one second enters set mode.

Set Mode. The seconds digits flash at 2 Hz with an 80% duty cycle, indicating the currently selected field. The flashing digits are incremented by `button[1]` and decremented by `button[0]`; holding either button for more than half a second repeats at 10 Hz. A second press of `button[3]` moves to the minutes digits, a third to the hours digits, and a fourth exits set mode and returns to stopped mode with the newly set time.

Running Mode. The display counts down once per second. Pressing `button[0]` pauses the timer. When the count reaches zero, the timer stops automatically and the display holds at zero. Set mode cannot be entered while the timer is running.

5.2. Design Approach

The time-setting procedure is identical to that in Assignment 1, so the same modules can be instantiated. In fact, apart from three small differences, the timer is equivalent to a settable watch: time ticks down rather than up; time stops at zero instead of wrapping; and time can be paused and resumed.

We consider each difference in turn.

Time Ticks Down. The underlying counters simply need their direction reversed.

Time Stops at Zero. A combinational check determines whether the current count is all zeros; this produces a signal that qualifies the counter enable, preventing any further decrement once zero is reached.

Pausing and Resuming. The tick signal that drives the counters is qualified by a running/stopped flag. When the timer is paused, the tick is suppressed and the counters hold their value.

Since the required changes are small, the overall design closely mirrors that of the settable watch from Assignment 1. Try to derive it independently; if you get stuck, the settable watch implementation is there as a reference.

5.3. Editable Countdown with Borrow Out

Report 11

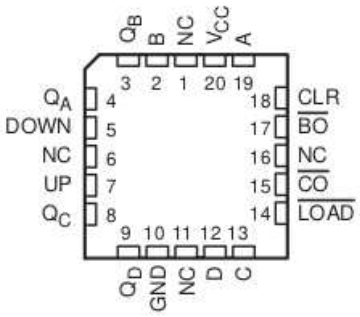
Find the datasheet for the 74LS193 synchronous 4-bit up/down counter. Using the schematic diagram in the datasheet, explain how the borrow output is generated and why it behaves as it does.

description

The 'ALS193A are synchronous, reversible, 4-bit up/down binary counters. Synchronous counting operation is provided by having all flip-flops clocked simultaneously so that the outputs change coincident with each other when instructed by the steering logic. This mode of operation eliminates the output counting spikes normally associated with asynchronous (ripple-clock) counters.

The outputs of the four flip-flops are triggered on a low-to-high-level transition of either count/clock (UP or DOWN) input. The direction of the count is determined by which count input is pulsed while the other count input is high.

SN54ALS193A . . . FK PACKAGE
(TOP VIEW)



NC – No internal connection

All four counters are fully programmable; that is, each output may be preset to either level by placing a low on the load ($\overline{LOAD}$) input and entering the desired data at the data inputs. The output changes to agree with the data inputs independently of the count pulses. This feature allows the counters to be used as modulo-N dividers by simply modifying the count length with the preset inputs.

A high level applied to the clear (CLR) input forces all outputs to the low level. The clear function is independent of the count and $\overline{LOAD}$ inputs. The UP, DOWN, and $\overline{LOAD}$ inputs are buffered to lower the drive requirement, which significantly reduces the loading on, or current required by, clock drivers, etc., for long parallel words.

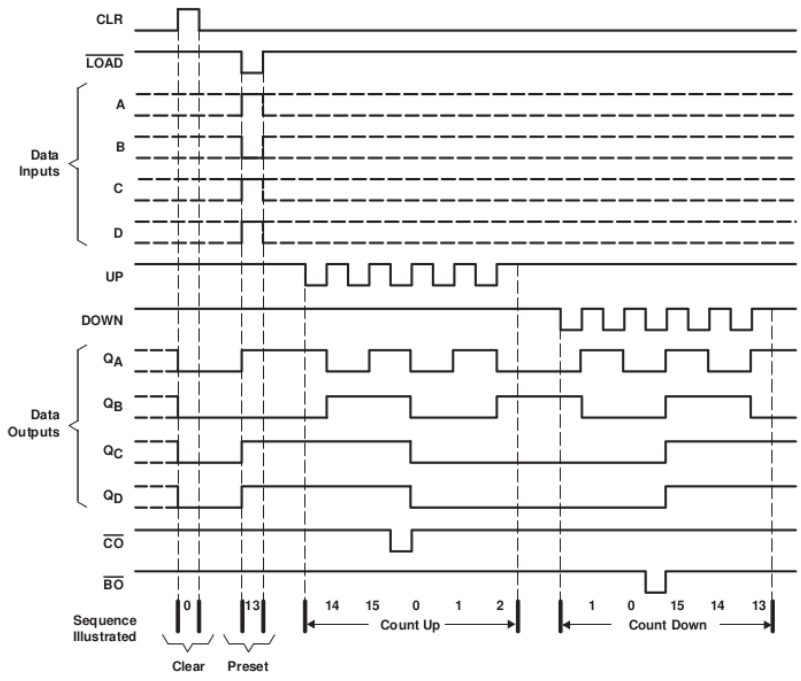
These counters are designed to be cascaded without the need for external circuitry. The borrow ($\overline{BO}$) output produces a low-level pulse while the count is zero (all Q outputs low) and the DOWN input is low. Similarily, the carry ($\overline{CO}$) output produces a low-level pulse while the count is 9 or 15 (all Q outputs high) and the UP input is low. The counters can then be easily cascaded by feeding $\overline{BO}$ and $\overline{CO}$ to the count-down and count-up inputs, respectively, of the succeeding counter.

The SN54ALS193A is characterized for operation over the full military temperature range of −55°C to 125°C. The SN74ALS193A is characterized for operation from 0°C to 70°C.

typical clear, load, and count sequence

the following sequence is illustrated below:

1. Clear outputs to zero
2. Load (preset) to binary 13
3. Count up to 14, 15 (carry), 0, 1, and 2
4. Count down to 1, 0 (borrow), 15, 14, and 13



NOTES: A. Clear overrides load, data, and count inputs.
B. When counting up, count-down input must be high; when counting down, count-up input must be high.

Instead of using CLOCK_50 directly for pulses, pair with UP/DOWN states as the 'CLK' for both directions

The 4-bit counter can be loaded initially with a 4-bit value

If UP is counting and the counter reaches (Moore) state 4'b1111, an output CARRY signal is generated

Likewise, when DOWN is counting and counter reaches (Moore) state 4'b0000, an output BORROW signal is generated

This module behaves like `editable_counter` from Assignment 1, with one change and two additions: `count` decrements on each `tick` pulse (wrapping from 0 to `MAX`); `clr` sets `count` to 0 on the next rising clock edge, taking priority over all other inputs; and `borrow_out` is a new combinational output mirroring the borrow output of the 74LS193, except that it is also low when `edit_mode` or `clr` is high. The `count` output is initialised to 0.

Dependency Only the following module is to be directly instantiated.

- `up_down_counter_rst`

rtl > editable_countdown.sv > ...

```
// Editable Countdown
// Same as the Editable Counter module but with `clr`, counts down and extra
// output signal `borrow_out`
//
// Parameters:
//    MAX
//    WIDTH
//
// Ports:
//    clk:              - Clock input
//    clr:              - Asynchronous reset
//    tick:             - Input signal to pulse the `tick_event` when not in
//                        `edit_mode`
//    edit_mode:        - Input signal to represent State whether module is
//                      - in edit mode
//    inc:              - Input signal to pulse the `inc_event` in `edit_mode`
//    dec:              - Input signal to pulse the `dec_event` in `edit_mode`
//    [WIDTH-1:0] count:- Output signal to count a number between 0 to N-1
//    borrow_out:       - Output signal to pulse rollovers
//
// Local Variables:
//    inc_event:  - Signal representing incrementing STATE when
//                  `inc` is HIGH AND `dec` is LOW while in `edit_mode`
//    dec_event:  - Signal representing decrementing STATE when
//                  `dec` is HIGH AND `inc` is LOW while in `edit_mode`
//    tick_event: - Signal representing NOT in `edit_mode` running according
//                  to `tick` pulses
//    up:         - Direction of counting: HIGH = upwards, LOW = downwards
//    enable:     - Enable the Up_Down_Counter when correct conditions are met

`timescale 1ns / 1ps

module editable_countdown #(
    parameter int MAX   = 59,
    parameter int WIDTH = 6
) (
    input logic clk,
    input logic clr,
    input logic tick,
    input logic edit_mode,
    input logic inc,
    input logic dec,
    output logic [WIDTH-1:0] count,
    output logic borrow_out
);

  logic enable;
  logic up;
  up_down_counter_rst #(
      .MAX  (MAX),
      .WIDTH(WIDTH)
  ) u_up_down_counter_rst (
      .clk(clk),
      .rst(clr),
      .enable(enable),
      .up(up),
      .count(count)
  );

  wire inc_event = edit_mode && inc && !dec && !clr;
  wire dec_event = edit_mode && dec && !inc && !clr;
  wire tick_event = !edit_mode && tick && !clr;

  always_comb begin
    if (edit_mode) up = inc_event || !dec_event;
    else up = !tick_event;
  end

  assign enable = tick_event || (dec_event || inc_event);

  assign borrow_out = tick_event && (count == 0);

endmodule
```

```
root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/editable_countdown.sv
root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/editable_countdown.sv
root@0b9c66e66b04:/workspace#
```

```
SBY  7:32:21 [sby/editable_countdown_max_4] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_4] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_4] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  7:32:21 [sby/editable_countdown_max_4] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  7:32:21 [sby/editable_countdown_max_4] summary: engine_0 did not produce any traces
SBY  7:32:21 [sby/editable_countdown_max_4] summary: successful proof by k-induction.
SBY  7:32:21 [sby/editable_countdown_max_4] DONE (PASS, rc=0)
SBY  7:32:21 [sby/editable_countdown_max_59] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_59] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_59] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  7:32:21 [sby/editable_countdown_max_59] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  7:32:21 [sby/editable_countdown_max_59] summary: engine_0 did not produce any traces
SBY  7:32:21 [sby/editable_countdown_max_59] summary: successful proof by k-induction.
SBY  7:32:21 [sby/editable_countdown_max_59] DONE (PASS, rc=0)
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Solver: z3
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Status: passed
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: finished (returncode=0)
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: Status returned by engine for induction: pass
SBY  7:32:21 [sby/editable_countdown_max_15] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_15] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:01 (1)
SBY  7:32:21 [sby/editable_countdown_max_15] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  7:32:21 [sby/editable_countdown_max_15] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  7:32:21 [sby/editable_countdown_max_15] summary: engine_0 did not produce any traces
SBY  7:32:21 [sby/editable_countdown_max_15] summary: successful proof by k-induction.
SBY  7:32:21 [sby/editable_countdown_max_15] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```

```
root@0b9c66e66b04:/workspace# pytest
==================================== test session starts ====================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 126 items

tests/test_arming_latch.py .                                                          [  0%]
tests/test_binary_to_bcd.py .                                                         [  1%]
tests/test_button_auto_repeat.py .                                                    [  2%]
tests/test_button_hold_detect.py .                                                    [  3%]
tests/test_button_hold_pulse.py .                                                     [  3%]
tests/test_decimal_display_driver.py .                                                [  4%]
tests/test_edit_mode_selector.py .                                                    [  5%]
tests/test_editable_counter.py ...                                                    [  7%]
tests/test_formal.py ......ss                                                         [ 14%]
tests/test_hms_counter.py ...                                                         [ 16%]
tests/test_key_synchroniser.py .                                                      [ 17%]
tests/test_lint.py ....................................................................  [ 65%]
tests/test_mod_n_counter.py ..                                                        [ 66%]
tests/test_pwm_generator.py .......                                                   [ 72%]
tests/test_restartable_rate_generator.py ......                                       [ 76%]
tests/test_rising_edge_detector.py .                                                  [ 77%]
tests/test_seven_segment.py ..                                                        [ 79%]
tests/test_stopwatch_control.py .                                                     [ 80%]
tests/test_top_de1_soc.py .                                                           [ 80%]
tests/test_top_time_display_v1.py .                                                   [ 81%]
tests/test_up_down_counter.py ...                                                     [ 84%]
tests/test_user_top_brightness_timepiece.py s                                         [ 84%]
tests/test_user_top_brightness_wrapper.py ssss                                        [ 88%]
tests/test_user_top_stopwatch_v1.py .                                                 [ 88%]
tests/test_user_top_timepiece_v1.py s                                                 [ 89%]
tests/test_user_top_timer_v1.py s                                                     [ 90%]
tests/test_user_top_watch_v1.py .                                                     [ 91%]
tests/test_user_top_watch_v2.py ..                                                    [ 92%]
tests/test_user_top_watch_v3.py ....                                                  [ 96%]
tests/test_user_top_watch_v4.py .....                                                 [100%]

============================== 117 passed, 9 skipped in 26.53s ==============================
root@0b9c66e66b04:/workspace#
```

5.4. Timer Integration

Task Implement `user_top_timer_v1` in `rtl/user_top_timer_v1.sv` in accordance with Section 5.1, with no Verilator linting warnings. Develop and test one feature at a time using test benches you write yourself.

To facilitate formal verification, add two ports to your module: